
PySCIPOpt

Zuse Institute Berlin <scip@zib.de>

Nov 29, 2025

CONTENTS

1	Contents	3
1.1	Installation Guide	3
1.2	Building From Source	4
1.3	User Guide (Tutorials)	8
1.4	Why SCIP?	59
1.5	Extending the Interface	62
1.6	Similar Software	63
1.7	Frequently Asked Questions	65
1.8	Contributors	66
1.9	API Reference	67
	Python Module Index	151
	Index	153

PySCIPOpt is the Python interface to [SCIP](#) (Solving Constraint Integer Programs).

CONTENTS

1.1 Installation Guide

This page will detail all methods for installing PySCIOpt via package managers, which come with their own versions of SCIP. For building PySCIOpt against your own custom version of SCIP, or for building PySCIOpt from source, visit [this page](#).

Contents

- *Installation Guide*
 - *PyPI (pip)*
 - *Conda*

1.1.1 PyPI (pip)

Pre-built binary wheels are uploaded to PyPI (Python Package Index) for each release. Supported platforms are Linux (x86_64), Windows (x86_64) and MacOS (x86_64, Apple Silicon).

To install PySCIOpt simply run the command:

```
pip install pyscipopt
```

To avoid interfering with system packages, it's best to use a *virtual environment* <<https://docs.python.org/3/library/venv.html>>.

Warning

Using a virtual environment is **mandatory** in some newer Python configurations to avoid permission and package conflicts.

```
python3 -m venv venv source venv/bin/activate # On Windows use: venvScriptsactivate pip install pyscipopt
```

Remember to activate the environment (source venv/bin/activate) in each terminal session where you use PySCIOpt.

Note

For Linux users: PySCIPOpt versions newer than 5.1.1 installed via PyPI now require glibc 2.28+

For our build infrastructure we use [manylinux](#). As CentOS 7 is no longer supported, we have migrated from [manylinux2014](#) to [manylinux_2_28](#).

TLDR: Older linux distributions may not work for newer versions of PySCIPOpt installed via pip.

Note

For Mac users: PySCIPOpt versions newer than 5.1.1 installed via PyPI now only support MACOSX 13+ for users running x86_64 architecture, and MACOSX 14+ for users running newer Apple silicon.

Note

For versions older than 4.4.0 installed via PyPI SCIP is not automatically installed. This means that SCIP must be installed yourself. If it is not installed globally, then the `SCIPOPTDIR` environment flag must be set, see [this page](#) for more details.

Note

Some Mac configurations require adding the library installation path to `DYLD_LIBRARY_PATH` when using a locally installed version of SCIP.

1.1.2 Conda

It is also possible to use the Conda package manager to install PySCIPOpt. Conda will install SCIP automatically, hence everything can be installed in a single command:

```
conda install --channel conda-forge pyscipopt
```

Note

Do not use the Conda base environment to install PySCIPOpt.

1.2 Building From Source

When building PySCIPOpt from source, one must have their own installation of [SCIP](#). To download SCIP please either use the pre-built SCIP Optimization Suite available [here](#) (recommended) or download SCIP and build from source itself from [here](#). A more minimal and experimental pre-built SCIP is available [here](#).

Contents

- [Building From Source](#)

- *Download Source Code*
- *Requirements*
- *Environment Variables*
- *Build Instructions*
- *Build with Debug*
- *Testing the Installation*
- *Building Documentation Locally*

Note

The latest PySCIPOpt version is usually only compatible with the latest major release of the SCIP Optimization Suite. The following table summarizes which version of PySCIPOpt is required for a given SCIP version:

Table 1: Supported SCIP versions for each PySCIPOpt version

SCIP	PySCIPOpt
10.0.0	6.0
9.2	5.3, 5.4, 5.5, 5.6, 5.7
9.1	5.1, 5.2.x
9.0	5.0.x
8.0	4.x
7.0	3.x
6.0	2.x
5.0	1.4, 1.3
4.0	1.2, 1.1
3.2	1.0

Note

If you install SCIP yourself and are not using the pre-built packages, you need to install the SCIP Optimization Suite using CMake. The Makefile system is not compatible with PySCIPOpt!

1.2.1 Download Source Code

To download the source code for PySCIPOpt we recommend cloning the repository using Git. The two methods for doing so are via SSH (recommended) and HTTPS.

```
git clone git@github.com:scipopt/PySCIPOpt.git
```

```
git clone https://github.com/scipopt/PySCIPOpt.git
```

One can also download the repository itself from GitHub [here](#).

1.2.2 Requirements

When building from source you must have the packages `setuptools` and `Cython` installed in your Python environment. These can be installed via PyPI:

```
pip install setuptools
pip install Cython
```

Note

Since the introduction of Cython 3 we recommend building using `Cython>=3`.

Furthermore, you must have the Python development files installed on your system. Not having these files will produce an error similar to: (error message "Python.h not found"). To install these development files on Linux use the following command (change according to your distributions package manager):

```
sudo apt-get install python3-dev # Linux
```

Note

For other operating systems this may not be necessary as it comes with many Python installations.

1.2.3 Environment Variables

When installing PySCIOpt from source, Python must be able to find your installation of SCIP. If SCIP is installed globally then this is not an issue, although we still encourage users to explicitly use such an environment variable. If SCIP is not installed globally, then the user must set the appropriate environment variable that points to the installation location of SCIP. The environment variable that must be set is `SCIOPTDIR`.

For Linux and MacOS systems set the variable with the following command:

```
export SCIOPTDIR=<path_to_install_dir>
```

Note

For macOS users, to ensure that the SCIP dynamic library can be found at runtime by PySCIOpt, you should add your SCIP installation path to the `DYLD_LIBRARY_PATH` environment variable by running:

```
export DYLD_LIBRARY_PATH="<path_to_install_dir>/lib:$DYLD_LIBRARY_PATH"
```

For Windows use the following command:

```
set SCIOPTDIR=<path_to_install_dir> # This is done for command line interfaces (cmd,
↪ Cmler, WSL)
$Env:SCIOPTDIR = "<path_to_install_dir>" # This is done for command line interfaces
↪ (powershell)
```

`SCIOPTDIR` should be a directory. It needs to have a subdirectory `lib` that contains the library, e.g. `libscip.so` (for Linux) and a subdirectory `include` that contains the corresponding header files:

```

SCIPOPTDIR
> lib
> libscip.so ...
> include
> scip
> lpi
> ...

```

Note

It is always recommended to use virtual environments for Python, see [here](#).

A virtual environment allows one to have multiple environments with different packages installed in each. To install a virtual environment simply run the command:

```
python -m venv <venv_name e.g. venv>
```

1.2.4 Build Instructions

After setting up the environment variables SCIPOPTDIR (see above) and installing all requirements (see above), you can now install PySCIPOpt from source. To do so run the following command from the main directory of PySCIPOpt (one with `setup.py`, `pyproject.toml` and `README.md`):

```
# Set environment variable SCIPOPTDIR if not yet done
python -m pip install .
```

For recompiling the source in the current directory use the command:

```
python -m pip install --compile .
```

Note

Building PySCIPOpt from source can be slow. This is normal.

If you want to build it quickly and unoptimised, which will affect performance (highly discouraged if running any meaningful time dependent experiments), you can set the environment variable `export CFLAGS="-O0 -ggdb"` (Linux example command)

1.2.5 Build with Debug

To use debug information in PySCIPOpt you need to build it with the following command:

Note

Be aware that you will need the debug library of the SCIP Optimization Suite for this to work (`cmake .. -DCMAKE_BUILD_TYPE=Debug`).

1.2.6 Testing the Installation

To test your brand-new installation of PySCIPOpt you need to install some dependencies.

```
pip install -r requirements/test.txt
```

Tests can be run in the PySCIPOpt directory with the commands:

```
pytest # Will run all the available tests
pytest tests/test_name.py # Will run a specific tests/test_name.py (Unix)
pytest -nauto # Will run tests in parallel using all available cores
```

Ideally, the status of your tests must be passed or skipped. Running tests with pytest creates the `__pycache__` directory in tests and, occasionally, a model file in the working directory. They can be removed harmlessly.

1.2.7 Building Documentation Locally

You can build the documentation locally with the command:

```
pip install -r docs/requirements.txt
sphinx-build docs docs/_build
```

1.3 User Guide (Tutorials)

This section contains official tutorials (examples) for PySCIPOpt. Please keep in mind that PySCIPOpt's primary purpose is as a wrapper for SCIP. Therefore, for sometimes more detailed information see [this page](#).

1.3.1 Introduction (Model Object, Solution Information, Parameter Settings)

The `Model` object is the central Python object that you will interact with. To use the `Model` object simply import it from the package directly.

```
from pyscipopt import Model

scip = Model()
```

Contents

- *Introduction (Model Object, Solution Information, Parameter Settings)*
 - *Create a Model, Variables, and Constraints*
 - *Query the Model for Solution Information*
 - *Set / Get a Parameter*
 - *Set Plugin-wide Parameters (Aggressiveness)*
 - *Set Solver Emphasis*
 - *Copy a SCIP Model*

Create a Model, Variables, and Constraints

While an empty Model is still something, we ultimately want a non-empty optimization problem. Let's consider the basic optimization problem:

$$\begin{aligned}
 &\min \\
 &2x + 3y - 5z \\
 &\text{s.t.} \\
 &x + y \leq 5 \\
 &x + z \geq 3 \\
 &y + z = 4 \\
 &(x, y, z) \in \mathbb{R}_{\geq 0}
 \end{aligned}$$

We can construct the optimization problem as follows:

```

scip = Model()
x = scip.addVar(vtype='C', lb=0, ub=None, name='x')
y = scip.addVar(vtype='C', lb=0, ub=None, name='y')
z = scip.addVar(vtype='C', lb=0, ub=None, name='z')
cons_1 = scip.addCons(x + y <= 5, name="cons_1")
cons_2 = scip.addCons(x + z >= 3, name="cons_2")
cons_3 = scip.addCons(y + z == 4, name="cons_3")
scip.setObjective(2 * x + 3 * y - 5 * z, sense="minimize")
scip.optimize()

```

That's it! We've built the optimization problem defined above and we've optimized it. For how to read a Model from file see [this page](#) and for best practices on how to create more variables see [this page](#).

Note

`vtype='C'` here refers to a continuous variable. Providing the `lb`, `ub` was not necessary as they default to `(0, None)` for continuous variables. Providing the `name` attribute is not necessary either but is good practice. Providing the objective sense was also not necessary as it defaults to "minimize".

Note

An advantage of SCIP is that it can handle general non-linearities. See [this page](#) for more information on this.

Query the Model for Solution Information

Now that we have successfully optimized our model, let's see some examples of what information we can query. For example, the solving time, number of nodes, optimal objective value, and the variable solution values in the optimal solution.

```

solve_time = scip.getSolvingTime()
num_nodes = scip.getNTotalNodes() # Note that getNNodes() is only the number of nodes_
↳ for the current run (resets at restart)

```

(continues on next page)

(continued from previous page)

```
obj_val = scip.getObjVal()
for scip_var in [x, y, z]:
    print(f"Variable {scip_var.name} has value {scip.getVal(scip_var)}")
```

Set / Get a Parameter

SCIP has an absolutely giant amount of parameters (see [here](#)). There is one easily accessible function for setting individual parameters. For example, if we want to set a time limit of 20s on the solving process then we would execute the following code:

```
scip.setParam("limits/time", 20)
```

To get the value of a parameter there is also one easily accessible function. For instance, we could now check if the time limit has been set correctly with the following code.

```
time_limit = scip.getParam("limits/time")
```

A user can set multiple parameters at once by creating a dictionary with keys corresponding to the parameter names and values corresponding to the desired parameter values.

```
param_dict = {"limits/time": 20}
scip.setParams(param_dict)
```

To get the values of all parameters in a dictionary use the following command:

```
param_dict = scip.getParams()
```

Finally, if you have a `.set` file (common for using SCIP via the command-line) that contains all the parameter values that you wish to set, then one can use the command:

```
scip.readParams(path_to_file)
```

Set Plugin-wide Parameters (Aggressiveness)

We can influence the behavior of some of SCIP's plugins using `SCIP_PARAMSETTING`. This can be applied to the heuristics, to the presolvers, and to the separators (respectively with `setHeuristics`, `setPresolve`, and `setSeparating`).

```
from pyscipopt import Model, SCIP_PARAMSETTING

scip = Model()
scip.setHeuristics(SCIP_PARAMSETTING.AGGRESSIVE)
```

There are four parameter settings:

Table 2: A list of the different options and the result

Option	Result
DEFAULT	set to the default values of all the plugin's parameters
FAST	the time spend for the plugin is decreased
AGGRESSIVE	such that the plugin is called more aggressively
OFF	turn off the plugin

Note

This is important to get dual information, as it's necessary to disable presolving and heuristics. For more information, see the tutorial on getting *constraint information*.

Set Solver Emphasis

One can also instruct SCIP to focus on different aspects of the search process. To do this, import `SCIP_PARAMEMPHASIS` from `pyscipopt` and set the appropriate value. For example, if the goal is just to find a feasible solution, then we can do the following:

```
from pyscipopt import Model, SCIP_PARAMEMPHASIS

scip = Model()
scip.setEmphasis(SCIP_PARAMEMPHASIS.FEASIBILITY)
```

You can find below a list of the available options, alongside their meaning.

Table 3: Parameter emphasis summary

Setting	Meaning
<code>PARAMEMPHASIS.DEFAULT</code>	to use default values
<code>PARAMEMPHASIS.COUNTER</code>	to get feasible and “fast” counting process
<code>PARAMEMPHASIS.CPSOLVER</code>	to get CP-like search (e.g. no LP relaxation)
<code>PARAMEMPHASIS.EASYCIP</code>	to solve easy problems fast
<code>PARAMEMPHASIS.FEASIBILITY</code>	to detect feasibility fast
<code>PARAMEMPHASIS.HARDLP</code>	to be capable to handle hard LPs
<code>PARAMEMPHASIS.OPTIMALITY</code>	to prove optimality fast
<code>PARAMEMPHASIS.PHASEFEAS</code>	to find feasible solutions during a 3 phase solution process
<code>PARAMEMPHASIS.PHASEIMPROVE</code>	to find improved solutions during a 3 phase solution process
<code>PARAMEMPHASIS.PHASEPROOF</code>	to proof optimality during a 3 phase solution process
<code>PARAMEMPHASIS.NUMERICS</code>	to solve problems which cause numerical issues

Copy a SCIP Model

A SCIP Model can also be copied. This can be done with the following logic:

```
scip_alternate_model = Model(sourceModel=scip) # Assuming scip is a pyscipopt Model
```

This model is completely independent of the source model. The data has been duplicated. That is, calling `scip.optimize()` at this point will have no effect on `scip_alternate_model`.

Note

After optimizing users often struggle with reoptimization. To make changes to an already optimized model, one must first do the following:

```
scip.freeTransform()
```

Without calling this function the Model can only be queried in its post optimized state. This is because the transformed problem and all the previous solving information is not automatically deleted, and thus stops a new opti-

mization call.

i Note

To completely remove the SCIP model from memory use the following command:

```
scip.freeProb()
```

This command is potentially useful if there are memory concerns and one is creating a large amount of different SCIP models.

1.3.2 Variables in SCIP

In this overview of variables in PySCIPOpt we'll walk through best practices for modelling them and the various information that can be extracted from them.

For the following let us assume that a Model object is available, which is created as follows:

```
from pyscipopt import Model, quicksum  
  
scip = Model()
```

i Note

In general, you want to keep track of your variable objects. They can always be obtained from the model after they are added, but then the responsibility falls to the user to match them, e.g. by name or constraints they feature in.

Contents

- *Variables in SCIP*
 - *Variable Types*
 - *Dictionary of Variables*
 - *List of Variables*
 - *Numpy array of Variables*
 - *Get Variables*
 - *Variable Information*
 - *What is a Column?*
 - *What is a Transformed Variable?*

Variable Types

SCIP has four different types of variables:

Table 4: Variable Types

Variable Type	Abbreviation	Description
Continuous	C	A continuous variable belonging to the reals with some lower and upper bound
Integer	I	An integer variable unable to take fractional values in a solution with some lower and upper bound
Binary	B	A variable restricted to the values 0 or 1.
Implicit Integer	M	A variable that is continuous but can be inferred to be integer in any valid solution

The variable type can be queried from the Variable object.

```
x = scip.addVar(vtype='C', name='x')
assert x.vtype() == "CONTINUOUS"
```

Dictionary of Variables

Here we will store PySCIPOpt variables in a standard Python dictionary

```
var_dict = {}
n = 5
m = 5
for i in range(n):
    var_dict[i] = {}
    for j in range(m):
        var_dict[i][j] = scip.addVar(vtype='B', name=f"x_{i}_{j}")

example_cons = {}
for i in range(n):
    example_cons[i] = scip.addCons(quicksum(var_dict[i][j] for j in range(m)) == 1,
    name=f"cons_{i}")
```

List of Variables

Here we will store PySCIPOpt variables in a standard Python list

```
n, m = 5, 5
var_list = [[None for i in range(m)] for i in range(n)]
for i in range(n):
    for j in range(m):
        var_list[i][j] = scip.addVar(vtype='B', name=f"x_{i}_{j}")

example_cons = []
for i in range(n):
    example_cons.append(scip.addCons(quicksum(var_list[i][j] for j in range(m)) == 1,
    name=f"cons_{i}"))
```

Numpy array of Variables

Here we will store PySCIPOpt variables in a numpy ndarray

```
import numpy as np
n, m = 5, 5
var_array = np.zeros((n, m), dtype=object) # dtype object allows arbitrary storage
for i in range(n):
    for j in range(m):
        var_array[i][j] = scip.addVar(vtype='B', name=f"x_{i}_{j}")

example_cons = np.zeros((n,), dtype=object)
for i in range(n):
    example_cons[i] = scip.addCons(quicksum(var_dict[i][j] for j in range(m)) == 1,
    ↪ name=f"cons_{i}")
```

Note

An advantage of using numpy array storage is that you can then use numpy operators on the array of variables, e.g. reshape and stacking functions. It also means that you can form PySCIPOpt expressions in bulk, similar to matrix variables in some other packages. That is something like:

```
a = np.random.uniform(size=(n,m))
c = a @ var_array
```

Get Variables

Given a Model object, all added variables can be retrieved with the function:

```
scip_vars = scip.getVars()
```

Variable Information

In this subsection we'll walk through some functionality that is possible with the variable objects.

First, we can easily obtain the objective coefficient of a variable.

```
scip.setObjective(2 * x)
assert x.getObj() == 2.0
```

Assuming we have a solution to our problem, we can obtain the variable solution value in the current best solution with the command:

```
var_val = scip.getVal(x)
```

An alternate way to obtain the variable solution value (can be done from whatever solution you wish) is to query the solution object with the SCIP expression (potentially just the variable)

```
if scip.getNSols() >= 1:
    scip_sol = scip.getBestSol()
    var_val = scip_sol[x]
```

What is a Column?

We can also obtain the LP solution of a variable. This would be used when you have included your own plugin, and are querying specific information for a given LP relaxation at some node. This is not the variable solution value in the final optimal solution!

The LP solution value brings up an interesting feature of SCIP. Is the variable even in the LP? We can easily check this.

```
is_in_lp = x.isInLP()
if is_in_lp:
    print("Variable is in LP!")
    print(f"Variable value in LP is {x.getLPSol()}")
else:
    print("Variable is not in LP!")
```

When you solve an optimization problem with SCIP, the problem is first transformed. This process is called presolve, and is done to accelerate the subsequent solving process. Therefore, a variable that was originally created may have been transformed to another variable, or may have just been removed from the transformed problem entirely. The variable may also not exist because you are currently doing some pricing, and the LP only contains a subset of the variables. The summary is: It should not be taken for granted that your originally created variable is in an LP.

Now to some additional confusion. When you're solving an LP do you actually want a variable object? The variable object contains a lot of unnecessary information that is not needed to strictly solve the LP. This information will also have to be sent to the LP solver because SCIP is a plugin based solver and can use many different LP solvers. Therefore, if the variable is in the LP, it is represented by a column. The column object is the object that is actually used when solving the LP. The column for a variable can be found with the following code:

```
col = x.getCol()
```

Information that is LP specific can be queried by the column directly. This includes the objective value coefficient, the LP solution value, lower and upper bounds, and of course the variable that it represents.

```
obj_coeff = col.getObjCoeff()
lp_val = col.getPrimsol()
lb = col.getLb()
ub = col.getUb()
x = col.getVar()
```

What is a Transformed Variable?

In the explanation of a column we touched on the transformed problem. Naturally, in the transformed space we now have transformed variables instead of the original variables. To access the transformed variables one can use the command:

```
scip_vars = scip.getVars(transformed=True)
```

A variable can be checked for whether it belongs to the original space or the transformed space with the command:

```
is_original = scip_vars[0].isOriginal()
```

This difference is often important and should be kept in mind. For instance, in general the user is not interested in the solution values of the transformed variables at the end of the solving process, rather they are interested in the solution values of the original variables. This is because they can be interpreted easily as they belong to some user defined formulation.

Note

By default, SCIP places a `t_` in front of all transformed variable names.

1.3.3 Constraints in SCIP

In this overview of constraints in PySCIPOpt we'll walk through best practices for modelling them and the various information that they can be extracted from them.

For the following let us assume that a Model object is available, which is created as follows:

```
from pyscipopt import Model, quicksum

scip = Model()
```

Note

In general you want to keep track of your constraint objects. They can always be obtained from the model after they are added, but then the responsibility falls to the user to match them, e.g. by name.

Contents

- *Constraints in SCIP*
 - *What is a Constraint?*
 - *Quicksum*
 - *Constraint Information*
 - *Constraint Types*
 - *What is a Row?*

What is a Constraint?

A constraint in SCIP is likely much more broad in definition than you are familiar with. For more information we recommend reading [this page](#).

To create a standard linear or non-linear constraint use the command:

```
x = scip.addVar(vtype='B', name='x')
y = scip.addVar(vtype='B', name='y')
z = scip.addVar(vtype='B', name='z')
# Linear constraint
linear_cons = scip.addCons(x + y + z == 1, name="lin_cons")
# Non-linear constraint
nonlinear_cons = scip.addCons(x * y + z == 1, name="nonlinear_cons")
```

Quicksum

It is very common that when constructing constraints one wants to use the inbuilt `sum` function in Python. For example, consider the common scenario where we have a set of binary variables.

```
x = [scip.addVar(vtype='B', name=f"x_{i}") for i in range(1000)]
```

A standard constraint in this example may be that exactly one binary variable can be active. To sum these variables we recommend using the custom `quicksum` function, as it avoids intermediate data structure and adds terms inplace. For example:

```
scip.addCons(quicksum(x[i] for i in range(1000)) == 1, name="sum_cons")
```

Note

While this is often unnecessary for smaller models, for larger models it can have a substantial improvement on time spent in model construction.

Note

For prod there also exists an equivalent `quickprod` function.

Constraint Information

The Constraint object can be queried like any other object. Some of the information a Constraint object contains is the name of the constraint handler responsible for the constraint, and many boolean properties of the constraint, e.g., is it linear.

```
linear_conshdlr_name = linear_cons.getConshdlrName()
assert linear_cons.isLinear()
```

As constraints are broader than the standard linear constraints most users are familiar with, many of the functions that obtain constraint information are callable from the Model object. These include the activity of the constraint, the slack of the constraint, and adding or deleting coefficients.

```
if scip.getNSols() >= 1:
    scip_sol = scip.getBestSol()
    activity = scip.getActivity(linear_cons, scip_sol)
    slack = scip.getSlack(linear_cons, scip_sol)
# Check current coefficients with scip.getValsLinear(linear_cons)
scip.chgCoefLinear(linear_cons, x, 7) # Change the coefficient to 7
```

Currently not mentioned w.r.t. the constraints and rows is the dual information. This is frustratingly complicated. SCIP has a plugin based LP solver, which offers many choices for LP solvers, but makes getting information from them more complicated. Getting dual values from constraints or rows will work, but to be confident that they are returning the correct information we encourage doing three different things:

- Disable presolving and propagation to ensure that the LP solver - which is providing the dual information - actually solves the unmodified problem.
- Disable heuristics to avoid that the problem is solved before the LP solver is called.
- Ensure there are no bound constraints, i.e., constraints with only one variable.

To accomplish this one can apply the following settings to the Model.

```
from pyscipopt import SCIP_PARAMSETTING
scip.setPresolve(SCIP_PARAMSETTING.OFF)
scip.setHeuristics(SCIP_PARAMSETTING.OFF)
scip.disablePropagation()
```

We stress again that when accessing such values you should be confident that you know which LP is being referenced. This information for instance is unclear or difficult to derive a meaningful interpretation from when the solution process has ended. The dual value of a constraint can be obtained with the following code:

```
dual_sol = scip.getDualsolLinear(linear_cons)
```

Constraint Types

In the above we presented examples of only linear constraints and a non-linear constraint. SCIP however can handle many different types of constraints. Some of these that are likely familiar are SOS constraints, Indicator constraints, and AND / OR / XOR constraints. These constraint handlers have custom methods for improving the solving process of optimization problems that they feature in. To add such a constraint, e.g., an SOS and indicator constraint, you'd use the code:

```
sos_cons = scip.addConsSOS1([x, y, z], name="example_sos")
indicator_cons = scip.addConsIndicator(x + y <= 1, binvar=z, name="example_indicator")
```

SCIP also allows the creation of custom constraint handlers. These could be empty and just there to record data, there to provide custom handling of some user defined function, or they could be there to enforce a constraint that is incredibly inefficient to enforce via linear constraints. An example of such a constraint handler is presented in the lazy constraint tutorial for modelling the subtour elimination constraints [here](#)

What is a Row?

In a similar fashion to Variables with columns, see [this page](#), constraints bring up an interesting feature of SCIP when used in the context of an LP. The context of an LP here means that we are after the LP relaxation of the optimization problem at some node. Is the constraint even in the LP? When you solve an optimization problem with SCIP, the problem is first transformed. This process is called presolve, and is done to accelerate the subsequent solving process. Therefore, a constraint that was originally created may have been transformed entirely, as the original variables that featured in the constraint have also been changed. Additionally, maybe the constraint was found to be redundant, i.e., trivially true, and was removed. The constraint is also much more general than necessary, containing information that is not strictly necessary for solving the LP, and may not even be representable by linear constraints. Therefore, when representing a constraint in an LP, we use Row objects. Be warned however, that this is not necessarily a simple one-to-one matching. Some more complicated constraints may either have no Row representation in the LP or have multiple such rows necessary to best represent it in the LP. For a standard linear constraint the Row that represents the constraint in the LP can be found with the code:

```
row = scip.getRowLinear(linear_cons)
```

Note

Remember that such a Row representation refers only to the latest LP, and is best queried when access to the current LP is clear, e.g. when branching.

From a Row object one can easily obtain information about the current LP. Some quick examples are the lhs, rhs, constant shift, the columns with non-zero coefficient values, the matching coefficient values, and the constraint handler that created the Row.

```

lhs = row.getLhs()
rhs = row.getRhs()
constant = row.getConstant()
cols = row.getCols()
vals = row.getVals()
origin_cons_name = row.getConsOriginConshdlrtype()

```

1.3.4 Non-Linear Expressions

One of the big advantages of SCIP is that it handles arbitrary constraints. Arbitrary here is not an exaggeration, see [the constraint tutorial](#). An advantage of this generality is that it has inbuilt support for many non-linear functions. These non-linear expressions can be arbitrarily composed and SCIP will still find a globally optimal solution within tolerances of the entire constraint. Below we will outline many of the supported non-linear expressions.

For the following let us assume that a Model object is available, which is created as follows:

```

from pyscipopt import Model

scip = Model()

```

Contents

- *Non-Linear Expressions*
 - *Non-Linear Objectives*
 - *Polynomials*
 - *Square Root (sqrt)*
 - *Absolute (Abs)*
 - *Exponential (exp) and Log*
 - *Sin and Cosine (cos)*

Non-Linear Objectives

While SCIP supports general non-linearities, it only supports linear objective functions. With some basic reformulation this is not a restriction, however. Let's consider the general optimization problem:

$$\begin{aligned}
 &\min \\
 &f(x) \\
 &\text{s.t.} \\
 &g(x) \leq b \\
 \\
 &x \in \mathbb{Z}^{|\mathcal{J}|} \times \mathbb{R}^{[n]/\mathcal{J}}, \quad \mathcal{J} \subseteq [n] \\
 \\
 &f : \mathbb{R}^n \rightarrow \mathbb{R}, g : \mathbb{R}^n \rightarrow \mathbb{R}
 \end{aligned}$$

Let's consider the case where $f(x)$ is a non-linear function. This problem can be equivalently reformulated as:

$$\begin{aligned}
 &\min \\
 &y \\
 &\text{s.t.} \\
 &g(x) \leq b \\
 &y \geq f(x) \\
 &x \in \mathbb{Z}^{|\mathcal{J}|} \times \mathbb{R}^{[n]/\mathcal{J}}, \quad \mathcal{J} \subseteq [n] \\
 &y \in \mathbb{R} \\
 &f : \mathbb{R}^n \rightarrow \mathbb{R}, g : \mathbb{R}^n \rightarrow \mathbb{R}
 \end{aligned}$$

We've now obtained an equivalent problem with a linear objective function! The same process can be performed with a maximization problem, albeit by introducing a $\leq$ constraint for the introduced variable.

Let's see an example of how this would work when programming. Consider the simple problem:

$$\begin{aligned}
 &\min \\
 &x^2 + y \\
 &\text{s.t.} \\
 &x + y \geq 5 \\
 &x + 1.3y \leq 10 \\
 &(x, y) \in \mathbb{Z}^2
 \end{aligned}$$

One can program an equivalent optimization problem with linear objective function as follows:

```

x = scip.addVar(vtype='I', name='x')
y = scip.addVar(vtype='I', name='y')
z = scip.addVar(vtype='I', name='z') # This will be our replacement objective variable
cons_1 = scip.addCons(x + y >= 5, name="cons_1")
cons_2 = scip.addCons(x + 1.3 * y <= 10, name="cons_2")
cons_3 = scip.addCons(z >= x * x + y, name="cons_3")
scip.setObjective(z)

```

Polynomials

Polynomials can be constructed directly from using Python operators on created variables. Let's see an example of constructing the following constraint:

$$\frac{3x^2 + y^3z^2 + (2x + 3z)^2}{2(xz)} \leq xyz$$

The code for the following constraint can be written as follows:

```

x = scip.addVar(vtype='C', name='x')
y = scip.addVar(vtype='C', name='y')

```

(continues on next page)

(continued from previous page)

```

z = scip.addVar(vtype='C', name='z')
# Build the expression slowly (or do it all in the addCons call)
lhs = 3 * (x ** 2) + ((y ** 3) * (z ** 2)) + ((2 * x) + (3 * z)) ** 2
lhs = lhs / (2 * x * z)
cons_1 = scip.addCons(lhs <= x * y * z, name="poly_cons")

```

Square Root (sqrt)

There is native support for the square root function. Let's see an example for constructing the following constraint:

$$\sqrt{x} \leq y$$

The code for the following constraint can be written as follows:

```

from pyscipopt import sqrt
x = scip.addVar(vtype='C', name='x')
y = scip.addVar(vtype='C', name='y')
cons_1 = scip.addCons(sqrt(x) <= y, name="sqrt_cons")

```

Absolute (Abs)

Absolute values of expressions is supported by overloading how `__abs__` function of SCIP expression objects. Therefore, one does not need to import any functions. Let's see an example for constructing the following constraint:

$$|x| \leq y + 5$$

The code for the following constraint can be written as follows:

```

x = scip.addVar(vtype='C', lb=None, name='x')
y = scip.addVar(vtype='C', name='y')
cons_1 = scip.addCons(abs(x) <= y + 5, name="abs_cons")

```

Note

In general many constraints containing abs functions can be reformulated to linear constraints with the introduction of some binary variables. We recommend reformulating when it is easily possible, as it will in general improve solver performance.

Exponential (exp) and Log

There is native support for the exp and log functions. Let's see an example for constructing the following constraints:

$$\frac{1}{1 + e^{-x}} = y$$

$$\log(x) \leq z$$

The code for the following constraint can be written as follows:

```

from pyscipopt import exp, log
x = scip.addVar(vtype='C', name='x')
y = scip.addVar(vtype='C', name='y')

```

(continues on next page)

(continued from previous page)

```
z = scip.addVar(vtype='C', name='z')
cons_1 = scip.addCons( (1 / (1 + exp(-x))) == y, name="exp_cons")
cons_2 = scip.addCons(log(x) <= z, name="log_cons")
```

Sin and Cosine (cos)

There is native support for the sin and cos functions. Let's see an example for constructing the following constraints:

$$\sin(x) = y$$

$$\cos(y) \leq 0.5$$

The code for the following constraint can be written as follows:

```
from pyscipopt import cos, sin
x = scip.addVar(vtype='C', name='x')
y = scip.addVar(vtype='C', name='y')
cons_1 = scip.addCons(sin(x) == y, name="sin_cons")
cons_2 = scip.addCons(cos(y) <= 0.5, name="cos_cons")
```

1.3.5 Matrix API

In this overview of the matrix variable and constraint API in PySCIPOpt we'll walk through best practices for modelling them and the various information that can be extracted from them.

For the following let us assume that a Model object is available, which is created as follows:

```
from pyscipopt import Model, quicksum

scip = Model()
```

This tutorial should only be read after having an understanding of both the Variable object (see [the constraint tutorial](#)) and the Constraint object (see [the constraint tutorial](#)).

i Note

The matrix API is built heavily on `numpy`. This means that users can use all standard `numpy` operations that they are familiar with when handling matrix variables and expressions. For example, using the `@`, `matmul`, `*`, `+`, `hstack`, `vstack`, and `**` operations work exactly as they do when handling any standard `numpy` array.

Contents

- *Matrix API*
 - *What is a Matrix API?*
 - *Matrix Variables*
 - *Matrix Constraints*
 - *Class Properties*
 - *Accessing Variables and Constraints*

What is a Matrix API?

The standard approach explained in the variable and constraint tutorials, is to build each variable yourself (storing them in some data structure, e.g., a list or dict, with some loop), and to construct each constraint in a similar manner. That means building up each constraint yourself term by term. This approach is flexible, and still remains the standard, but an increasingly common trend is to view the modelling approach from a vector, matrix, and tensor perspective. That is, directly operate on larger sets of variables and expressions, letting python handle the interaction for each term. For such cases, it is encouraged that users now use the new matrix API!

Matrix Variables

Matrix variables are added via a single function call. It is important beforehand to know the shape of the new set of variables you want to create, where shape is some tuple or int. Below is an example for creating a 2x2 matrix variable of type continuous with an ub of 8.

```
x = scip.addMatrixVar(shape, vtype='C', name='x', ub=8)
```

Note

The name of each variable in the example above becomes `x_(indices)`

In the case of each kwarg, e.g., `vtype` and `ub`, a `np.array` of explicit values can be passed. In the example above, this means that each variable within the matrix variable can have its own custom information. For example:

```
x = scip.addMatrixVar(shape, vtype='C', name='x', ub=np.array([[5, 6], [2, 8]]))
```

Matrix Constraints

Matrix constraints follow the same logic as matrix variables. They can be constructed quickly and added all at once. The standard variable operators, like `sin`, `exp`, `sqrt`, etc., are applied element-wise. Some examples are provided below (these examples are nonsensical, and there to purely understand the API):

```
x = scip.addMatrixVar(shape=(2, 2), vtype="B", name="x")
y = scip.addMatrixVar(shape=(2, 2), vtype="C", name="y", ub=5)
z = scip.addVar(vtype="C", name="z", ub=7)

scip.addMatrixCons(x + y <= z)
scip.addMatrixCons(exp(x) + sin(sqrt(y)) == z + y)
scip.addMatrixCons(y <= x @ y)
scip.addMatrixCons(x @ y <= x)
scip.addCons(x.sum() <= 2) # Matrix variables can also appear in standard constraints,
↪ if the result expression is type Expr
```

Note

When creating constraints, one can mix standard variables and values in the same expressions. `numpy` will then handle this, and broadcast the correct operations. In general this can be viewed as creating an imaginary `np.array` of the appropriate shape and populating it with the variable / value.

Class Properties

A `MatrixVariable` and `MatrixConstraint` object have all the same getter functions that are in general available for the standard equivalent. An example is provided below for `vtype`.

```
x = scip.addVar()
matrix_x = scip.addMatrixVar(shape=(2,2))

x.vtype()
matrix_x.vtype()
```

The objects are not interchangeable however, when being passed into functions derived from the `Model` class. That is, there is currently no global support, that the following code runs:

```
scip.imaginary_function(x) # will always work
scip.imaginary_function(matrix_x) # may have to access each variable manually
```

Accessing Variables and Constraints

After creating the matrix variables and matrix constraints, one can always access the individual variables or constraints via their index.

```
x = scip.addMatrixVar(shape=(2, 2))
assert(isinstance(x, MatrixVariable))
assert(isinstance(x[0][0], Variable))
cons = x <= 2
assert(isinstance(cons, MatrixExprCons))
assert(isinstance(cons[0][0], ExprCons))
```

Accessing Solution Values

After optimizing a model, the solution values of a matrix variable can be accessed in an identical manner to the standard variant. There are two recommended ways to do this.

```
matrix_var_vals = scip.getVal(x)
```

```
sol = scip.getBestSol()
matrix_var_vals = sol[x] # returns a numpy array of values
```

1.3.6 Read and Write Files

While building your own optimization problem is fun, at some point you often need to share it, or read an optimization problem written by someone else. For this you're going to need to write the optimization problem to some file or read it from some file.

Contents

- *Read and Write Files*
 - *Model File Formats*
 - *Write a Model*
 - *Read a Model*

Model File Formats

SCIP has extensive support for a wide variety of file formats. The table below outlines what formats those are and the model types they're associated with.

Table 5: Supported File Formats

Extension	Model Type
CIP	SCIP's constraint integer programming format
CNF	DIMACS CNF (conjunctive normal form) format used for example for SAT problems
DIFF	reading a new objective function for mixed-integer programs
FZN	FlatZinc is a low-level solver input language that is the target language for MiniZinc
GMS	mixed-integer nonlinear programs (GAMS) [reading requires compilation with GAMS=true and a working GAMS system]
LP	mixed-integer (quadratically constrained quadratic) programs (CPLEX)
MPS	mixed-integer (quadratically constrained quadratic) programs
OPB	pseudo-Boolean optimization instances
OSiL	mixed-integer nonlinear programs
PIP	mixed-integer polynomial programming problems
SOL	solutions; XML-format (read-only) or raw SCIP format
WBO	weighted pseudo-Boolean optimization instances
ZPL	ZIMPL models, i.e., mixed-integer linear and nonlinear programming problems [read only]

Note

In general we recommend sharing files using the `.mps` extension when possible.

For a more human-readable format for equivalent problems we recommend the `.lp` extension.

For general non-linearities that are to be shared with others we recommend the `.osil` extension.

For general constraint types that will only be used by other SCIP users we recommend the `.cip` extension.

Note

Some of these file formats may only have a reader programmed and not a writer. Additionally, some of these readers may require external plug-ins that are not shipped by default via PyPI.

Write a Model

To write a SCIP Model to a file one simply needs to run the command:

```
from pyscipopt import Model
scip = Model()
scip.writeProblem(filename="example_file.mps", trans=False, genericnames=False)
```

Note

Both `trans` and `genericnames` are there as their default values. The `trans` option is available if you want to print the transformed problem (post presolve) instead of the model originally created. The `genericnames` option is there if you want to overwrite the variable and constraint names provided.

Read a Model

To read in a file to a SCIP model one simply needs to run the command:

```
from pyscipopt import Model
scip = Model()
scip.readProblem(filename="example_file.mps")
```

This will read in the file and you will now have a SCIP model that matches the file. Variables and constraints can be queried, with their names matching those in the file.

1.3.7 SCIP Log Files

For the following let us assume that we have called `optimize()` on a SCIP Model. When running, SCIP outputs a constant stream of information on the current state of the optimization process.

Contents

- *SCIP Log Files*
 - *How to Read SCIP Output*
 - * *Presolve Information*
 - * *Branch-and-Bound Information*
 - * *Final Summarised Information*
 - *How to Redirect SCIP Output*
 - *SCIP Statistics*

How to Read SCIP Output

Let's consider the example complete output below:

```
presolving:
(round 1, fast)      136 del vars, 0 del conss, 2 add conss, 0 chg bounds, 0 chg sides, 0
→0 chg coeffs, 0 upgd conss, 0 impls, 0 clqs
(round 2, fast)      136 del vars, 1 del conss, 2 add conss, 0 chg bounds, 132 chg
→sides, 0 chg coeffs, 0 upgd conss, 0 impls, 0 clqs
(round 3, exhaustive) 136 del vars, 2 del conss, 2 add conss, 0 chg bounds, 133 chg
→sides, 0 chg coeffs, 0 upgd conss, 0 impls, 0 clqs
(round 4, exhaustive) 136 del vars, 2 del conss, 2 add conss, 0 chg bounds, 133 chg
→sides, 0 chg coeffs, 131 upgd conss, 0 impls, 0 clqs
(0.0s) probing cycle finished: starting next cycle
(0.0s) symmetry computation started: requiring (bin +, int +, cont +), (fixed: bin -,
→int -, cont -)
```

(continues on next page)

(continued from previous page)

```

(0.0s) no symmetry present (symcode time: 0.00)
presolving (5 rounds: 5 fast, 3 medium, 3 exhaustive):
  136 deleted vars, 2 deleted constraints, 2 added constraints, 0 tightened bounds, 0
  ↳ added holes, 133 changed sides, 0 changed coefficients
  231 implications, 0 cliques
presolved problem has 232 variables (231 bin, 0 int, 1 impl, 0 cont) and 137 constraints
  53 constraints of type <knapsack>
   6 constraints of type <linear>
  78 constraints of type <logicor>
transformed objective value is always integral (scale: 1)
Presolving Time: 0.01

```

time	node	left	LP iter	LP it/n	mem/heur	mdpt	vars	cons	rows	cuts
↳ sepa	↳ confs	↳ strbr	dualbound	primalbound	gap	compl.				
0.0s	1	0	409	-	5350k	0	232	156	137	0 0 19
↳	0	7.649866e+03	--	Inf	unknown					
o 0.0s	1	0	1064	-	feasump	0	232	156	137	0 0 19
↳	0	7.650000e+03	8.267000e+03	8.07%	unknown					
0.0s	1	0	1064	-	5368k	0	232	156	137	0 0 19
↳	0	7.650000e+03	8.267000e+03	8.07%	unknown					
0.0s	1	0	1064	-	5422k	0	232	156	137	0 0 19
↳	0	7.650000e+03	8.267000e+03	8.07%	unknown					
0.0s	1	0	1067	-	5422k	0	232	156	137	0 0 19
↳	0	7.650000e+03	8.267000e+03	8.07%	unknown					
0.1s	1	0	1132	-	9912k	0	232	156	138	1 1 19
↳	0	7.659730e+03	8.267000e+03	7.93%	unknown					
0.1s	1	0	1133	-	9924k	0	232	157	138	1 1 20
↳	0	7.660000e+03	8.267000e+03	7.92%	unknown					
0.1s	1	0	1134	-	9924k	0	232	157	138	1 1 20
↳	0	7.660000e+03	8.267000e+03	7.92%	unknown					
0.1s	1	0	1210	-	15M	0	232	157	141	4 2 20
↳	0	7.671939e+03	8.267000e+03	7.76%	unknown					
0.1s	1	0	1213	-	15M	0	232	159	141	4 2 22
↳	0	7.672000e+03	8.267000e+03	7.76%	unknown					
0.1s	1	0	1280	-	18M	0	232	157	143	6 3 22
↳	0	7.685974e+03	8.267000e+03	7.56%	unknown					
0.1s	1	0	1282	-	18M	0	232	157	143	6 3 22
↳	0	7.686000e+03	8.267000e+03	7.56%	unknown					
0.2s	1	0	1353	-	21M	0	232	156	145	8 4 22
↳	0	7.701524e+03	8.267000e+03	7.34%	unknown					
0.2s	1	0	1355	-	21M	0	232	156	145	8 4 22
↳	0	7.702000e+03	8.267000e+03	7.34%	unknown					
0.2s	1	0	1435	-	24M	0	232	156	147	10 5 22
↳	0	7.706318e+03	8.267000e+03	7.28%	unknown					
time	node	left	LP iter	LP it/n	mem/heur	mdpt	vars	cons	rows	cuts
↳ sepa	↳ confs	↳ strbr	dualbound	primalbound	gap	compl.				
0.2s	1	0	1438	-	24M	0	232	158	147	10 5 24
↳	0	7.707000e+03	8.267000e+03	7.27%	unknown					
0.2s	1	0	1520	-	30M	0	232	158	149	12 6 24
↳	0	7.711108e+03	8.267000e+03	7.21%	unknown					
0.2s	1	0	1521	-	30M	0	232	158	149	12 6 24
↳	0	7.712000e+03	8.267000e+03	7.20%	unknown					

(continues on next page)

(continued from previous page)

0.2s	1	0	1658	-	34M	0	232	158	151	14	7	24	↩
↩	0	7.715238e+03	8.267000e+03	7.15%	unknown								
0.2s	1	0	1659	-	34M	0	232	158	151	14	7	24	↩
↩	0	7.716000e+03	8.267000e+03	7.14%	unknown								
0.3s	1	0	1770	-	40M	0	232	158	153	16	8	24	↩
↩	0	7.717854e+03	8.267000e+03	7.12%	unknown								
0.3s	1	0	1771	-	40M	0	232	158	153	16	8	24	↩
↩	0	7.718000e+03	8.267000e+03	7.11%	unknown								
0.3s	1	0	1883	-	40M	0	232	157	154	17	9	24	↩
↩	0	7.730185e+03	8.267000e+03	6.94%	unknown								
0.3s	1	0	1884	-	40M	0	232	157	154	17	9	24	↩
↩	0	7.731000e+03	8.267000e+03	6.93%	unknown								
0.3s	1	0	1925	-	46M	0	232	157	156	19	10	24	↩
↩	0	7.734301e+03	8.267000e+03	6.89%	unknown								
0.3s	1	0	1926	-	46M	0	232	157	152	19	10	24	↩
↩	0	7.735000e+03	8.267000e+03	6.88%	unknown								
0.3s	1	0	1946	-	46M	0	232	157	154	21	11	24	↩
↩	0	7.735000e+03	8.267000e+03	6.88%	unknown								
0.4s	1	0	1972	-	46M	0	232	157	156	23	12	24	↩
↩	0	7.735275e+03	8.267000e+03	6.87%	unknown								
0.4s	1	0	1973	-	46M	0	232	158	156	23	12	25	↩
↩	0	7.736000e+03	8.267000e+03	6.86%	unknown								
0.4s	1	0	2007	-	46M	0	232	158	157	24	13	25	↩
↩	0	7.736000e+03	8.267000e+03	6.86%	unknown								
time node left LP iter LP it/n mem/heur mdpt vars cons rows cuts													
↩ sepa confs strbr dualbound primalbound gap compl.													
0.4s	1	0	2057	-	46M	0	232	155	158	25	14	27	↩
↩	0	7.737403e+03	8.267000e+03	6.84%	unknown								
0.4s	1	0	2058	-	46M	0	232	155	153	25	14	27	↩
↩	0	7.738000e+03	8.267000e+03	6.84%	unknown								
0.4s	1	0	2086	-	46M	0	232	155	154	26	15	27	↩
↩	0	7.738004e+03	8.267000e+03	6.84%	unknown								
0.4s	1	0	2093	-	46M	0	232	155	156	28	16	27	↩
↩	0	7.738165e+03	8.267000e+03	6.83%	unknown								
0.4s	1	0	2094	-	46M	0	232	156	156	28	16	28	↩
↩	0	7.739000e+03	8.267000e+03	6.82%	unknown								
0.5s	1	0	2146	-	46M	0	232	156	157	29	17	28	↩
↩	0	7.739168e+03	8.267000e+03	6.82%	unknown								
0.5s	1	0	2147	-	46M	0	232	156	157	29	17	28	↩
↩	0	7.740000e+03	8.267000e+03	6.81%	unknown								
0.5s	1	0	2178	-	46M	0	232	156	157	30	18	28	↩
↩	0	7.740000e+03	8.267000e+03	6.81%	unknown								
0.5s	1	0	2223	-	46M	0	232	157	159	32	19	29	↩
↩	0	7.740575e+03	8.267000e+03	6.80%	unknown								
0.5s	1	0	2224	-	46M	0	232	157	159	32	19	29	↩
↩	0	7.741000e+03	8.267000e+03	6.79%	unknown								
0.5s	1	0	2259	-	46M	0	232	157	160	33	20	29	↩
↩	0	7.741000e+03	8.267000e+03	6.79%	unknown								
0.5s	1	0	2282	-	46M	0	232	157	161	34	21	29	↩
↩	0	7.741495e+03	8.267000e+03	6.79%	unknown								
0.5s	1	0	2283	-	46M	0	232	157	161	34	21	29	↩
↩	0	7.742000e+03	8.267000e+03	6.78%	unknown								

(continues on next page)

(continued from previous page)

```

0.5s| 1 | 0 | 2300 | - | 46M | 0 | 232 | 157 | 159 | 35 | 22 | 29
↳| 0 | 7.742000e+03 | 8.267000e+03 | 6.78%| unknown
0.6s| 1 | 0 | 2342 | - | 46M | 0 | 232 | 157 | 160 | 36 | 23 | 29
↳| 0 | 7.742000e+03 | 8.267000e+03 | 6.78%| unknown
time | node | left | LP iter|LP it/n|mem/heur|mdpt |vars |cons |rows |cuts
↳|sepa|confs|strbr| dualbound | primalbound | gap | compl.
0.6s| 1 | 0 | 2355 | - | 46M | 0 | 232 | 159 | 162 | 38 | 24 | 31
↳| 0 | 7.742000e+03 | 8.267000e+03 | 6.78%| unknown
0.6s| 1 | 0 | 2368 | - | 46M | 0 | 232 | 159 | 163 | 39 | 25 | 31
↳| 0 | 7.742000e+03 | 8.267000e+03 | 6.78%| unknown
0.6s| 1 | 0 | 2376 | - | 46M | 0 | 232 | 160 | 164 | 40 | 26 | 32
↳| 0 | 7.742000e+03 | 8.267000e+03 | 6.78%| unknown
L 0.8s| 1 | 0 | 2713 | - | rens| 0 | 232 | 165 | 164 | 40 | 27 | 37
↳| 0 | 7.742000e+03 | 8.135000e+03 | 5.08%| unknown
0.8s| 1 | 0 | 2713 | - | 46M | 0 | 232 | 165 | 164 | 40 | 27 | 37
↳| 0 | 7.742000e+03 | 8.135000e+03 | 5.08%| unknown
0.8s| 1 | 0 | 2713 | - | 46M | 0 | 232 | 165 | 164 | 40 | 27 | 37
↳| 0 | 7.742000e+03 | 8.135000e+03 | 5.08%| unknown
(run 1, node 1) restarting after 26 global fixings of integer variables

```

(restart) converted 26 cuts from the global cut pool into linear constraints

presolving:

```

(round 1, fast)      26 del vars, 9 del conss, 1 add conss, 0 chg bounds, 0 chg sides,
↳4 chg coeffs, 0 upgd conss, 231 impls, 0 clqs
(round 2, fast)      26 del vars, 9 del conss, 1 add conss, 0 chg bounds, 119 chg sides,
↳123 chg coeffs, 0 upgd conss, 231 impls, 0 clqs
(round 3, exhaustive) 26 del vars, 9 del conss, 1 add conss, 0 chg bounds, 119 chg sides,
↳123 chg coeffs, 13 upgd conss, 231 impls, 0 clqs
(round 4, exhaustive) 26 del vars, 9 del conss, 1 add conss, 0 chg bounds, 119 chg sides,
↳143 chg coeffs, 13 upgd conss, 231 impls, 0 clqs

```

presolving (5 rounds: 5 fast, 3 medium, 3 exhaustive):

```

26 deleted vars, 9 deleted constraints, 1 added constraints, 0 tightened bounds, 0
↳added holes, 119 changed sides, 143 changed coefficients
231 implications, 0 cliques

```

presolved problem has 206 variables (205 bin, 0 int, 1 impl, 0 cont) and 182 constraints

65 constraints of type <knapsack>

20 constraints of type <linear>

97 constraints of type <logicor>

transformed objective value is always integral (scale: 1)

Presolving Time: 0.02

```

time | node | left | LP iter|LP it/n|mem/heur|mdpt |vars |cons |rows |cuts
↳|sepa|confs|strbr| dualbound | primalbound | gap | compl.
0.9s| 1 | 0 | 3076 | - | 42M | 0 | 206 | 182 | 164 | 0 | 0 | 37
↳| 0 | 7.742000e+03 | 8.135000e+03 | 5.08%| unknown
0.9s| 1 | 0 | 3101 | - | 42M | 0 | 206 | 182 | 166 | 2 | 1 | 38
↳| 0 | 7.742241e+03 | 8.135000e+03 | 5.07%| unknown
0.9s| 1 | 0 | 3102 | - | 42M | 0 | 206 | 183 | 165 | 2 | 1 | 39
↳| 0 | 7.743000e+03 | 8.135000e+03 | 5.06%| unknown
0.9s| 1 | 0 | 3103 | - | 42M | 0 | 206 | 184 | 165 | 2 | 1 | 40
↳| 0 | 7.743000e+03 | 8.135000e+03 | 5.06%| unknown

```

(continues on next page)

(continued from previous page)

0.9s	1	0	3184	-	43M	0	206	184	167	4	2	40	↩
↩	0	7.744329e+03	8.135000e+03	5.04%	unknown								
0.9s	1	0	3186	-	43M	0	206	187	167	4	2	43	↩
↩	0	7.745000e+03	8.135000e+03	5.04%	unknown								
1.0s	1	0	3233	-	43M	0	206	187	169	6	3	43	↩
↩	0	7.745000e+03	8.135000e+03	5.04%	unknown								
1.0s	1	0	3252	-	45M	0	206	187	171	8	4	43	↩
↩	0	7.745123e+03	8.135000e+03	5.03%	unknown								
1.0s	1	0	3255	-	45M	0	206	192	171	8	4	48	↩
↩	0	7.746000e+03	8.135000e+03	5.02%	unknown								
1.0s	1	0	3290	-	45M	0	206	192	173	10	5	48	↩
↩	0	7.746000e+03	8.135000e+03	5.02%	unknown								
1.1s	1	0	3434	-	46M	0	206	192	166	12	6	49	↩
↩	0	7.746946e+03	8.135000e+03	5.01%	unknown								
1.1s	1	0	3435	-	46M	0	206	192	166	12	6	49	↩
↩	0	7.747000e+03	8.135000e+03	5.01%	unknown								
1.1s	1	0	3459	-	46M	0	206	192	167	13	7	49	↩
↩	0	7.747318e+03	8.135000e+03	5.00%	unknown								
1.1s	1	0	3460	-	46M	0	206	192	167	13	7	49	↩
↩	0	7.748000e+03	8.135000e+03	4.99%	unknown								
1.2s	1	0	3588	-	47M	0	206	192	168	14	8	49	↩
↩	0	7.748652e+03	8.135000e+03	4.99%	unknown								
time node left LP iter LP it/n mem/heur mdpt vars cons rows cuts													
↩ sepa confs strbr dualbound primalbound gap compl.													
1.2s	1	0	3589	-	47M	0	206	193	168	14	8	50	↩
↩	0	7.749000e+03	8.135000e+03	4.98%	unknown								
1.2s	1	0	3622	-	55M	0	206	193	165	15	9	50	↩
↩	0	7.749000e+03	8.135000e+03	4.98%	unknown								
1.2s	1	0	3736	-	55M	0	206	193	166	16	10	50	↩
↩	0	7.750062e+03	8.135000e+03	4.97%	unknown								
1.2s	1	0	3737	-	55M	0	206	193	166	16	10	50	↩
↩	0	7.751000e+03	8.135000e+03	4.95%	unknown								
1.2s	1	0	3759	-	55M	0	206	193	167	17	11	50	↩
↩	0	7.751000e+03	8.135000e+03	4.95%	unknown								
1.3s	1	0	3823	-	55M	0	206	193	168	18	12	50	↩
↩	0	7.751152e+03	8.135000e+03	4.95%	unknown								
1.3s	1	0	3824	-	55M	0	206	193	168	18	12	50	↩
↩	0	7.752000e+03	8.135000e+03	4.94%	unknown								
1.3s	1	0	3829	-	55M	0	206	193	161	19	13	50	↩
↩	0	7.752000e+03	8.135000e+03	4.94%	unknown								
1.3s	1	0	3836	-	55M	0	206	193	162	20	14	50	↩
↩	0	7.752000e+03	8.135000e+03	4.94%	unknown								
1.3s	1	0	3838	-	55M	0	206	193	163	21	15	50	↩
↩	0	7.752000e+03	8.135000e+03	4.94%	unknown								
1.3s	1	0	3874	-	55M	0	206	195	165	23	16	52	↩
↩	0	7.752000e+03	8.135000e+03	4.94%	unknown								
1.3s	1	0	3878	-	55M	0	206	195	166	24	17	53	↩
↩	0	7.752000e+03	8.135000e+03	4.94%	unknown								
2.0s	1	2	4001	-	55M	0	206	200	166	24	18	59	↩
↩	71	7.784907e+03	8.135000e+03	4.50%	unknown								
* 2.9s	59	21	6175	44.6	strongbr	11	206	251	158	45	1	110	↩
↩	494	7.846000e+03	8.099000e+03	3.22%	17.26%								

(continues on next page)

(continued from previous page)

```

* 3.0s| 94 | 26 | 6897 | 35.7 | LP | 18 | 206 | 262 | 151 | 54 | 2 | 121┐
↳ | 508 | 7.846000e+03 | 8.090000e+03 | 3.11%| 20.71%
time | node | left |LP iter|LP it/n|mem/heur|mdpt |vars |cons |rows |cuts┐
↳ |sepa|confs|strbr| dualbound | primalbound | gap | compl.
3.0s| 100 | 24 | 7010 | 34.7 | 85M | 18 | 206 | 268 | 146 | 54 | 0 | 127┐
↳ | 511 | 7.846000e+03 | 8.090000e+03 | 3.11%| 22.99%
3.3s| 200 | 34 | 9281 | 28.7 | 109M | 18 | 206 | 294 | 156 | 104 | 1 | 153┐
↳ | 539 | 7.868560e+03 | 8.090000e+03 | 2.81%| 35.07%
3.5s| 300 | 32 | 10971 | 24.8 | 109M | 18 | 206 | 307 | 146 | 134 | 0 | 166┐
↳ | 546 | 7.905000e+03 | 8.090000e+03 | 2.34%| 47.01%
3.8s| 400 | 28 | 12714 | 22.9 | 109M | 18 | 206 | 322 | 146 | 159 | 0 | 181┐
↳ | 557 | 7.927000e+03 | 8.090000e+03 | 2.06%| 58.37%
4.0s| 500 | 16 | 14489 | 21.9 | 109M | 18 | 206 | 328 | 148 | 196 | 0 | 187┐
↳ | 565 | 7.955492e+03 | 8.090000e+03 | 1.69%| 80.54%

SCIP Status      : problem is solved [optimal solution found]
Solving Time (sec) : 4.10
Solving Nodes    : 584 (total of 585 nodes in 2 runs)
Primal Bound     : +8.090000000000000e+03 (4 solutions)
Dual Bound      : +8.090000000000000e+03
Gap              : 0.00 %

```

Let's now walk through information that the log provides us. We'll break down this information into smaller bits.

Presolve Information

At the beginning of the run presolve information is output. The most important component is likely the final few lines of this portion of output. For the log above, from those lines we know that our problem after presolve has 232 variables and 137 constraints. 231 of those variables are binary with one variable being an implicit integer. 53 constraints are type knapsack, 6 are linear, and 78 are type logicor. The presolving time was 0.01s.

Branch-and-Bound Information

This section has the bulk of the solve information, and comes directly after the presolve section. It can easily be identified by its table like content that makes it easily machine readable. The columns of the output above are information on the following

Table 6: Label Summaries

Key	Full Description
time	total solution time
node	number of processed nodes
left	number of unprocessed nodes
LP iter	number of simplex iterations (see statistics for more accurate numbers)
LP it/n	average number of LP iterations since the last output line
mem/heur	total number of bytes in block memory or the creator name when a new incumbent solution was found
mdpt	maximal depth of all processed nodes
vars	number of variables in the problem
cons	number of globally valid constraints in the problem
rows	number of LP rows in current node
cuts	total number of cuts applied to the LPs
sepa	number of separation rounds performed at the current node
confs	total number of conflicts found in conflict analysis
strbr	total number of strong branching calls
dualbound	current global dual bound
primalbound	current primal bound
gap	current (relative) gap using $ primal - dual / \min(dual , primal)$
compl.	completion of search in percent (based on tree size estimation)

Note

When a new primal solution is found a letter or asterisk appears on the left side of the current row. An asterisk indicates that a primal solution has been found during the tree search, and a letter indicates that a primal heuristic has found a solution (letter maps to a specific heuristic)

The table shows the progress of the solver as it optimizes the problem. Each line snapshots a state of the optimization process, and thereby gives users frequent updates on the quality of the current best solution, how much memory is being used, and predicted amount of the tree search completed.

It should be mentioned that in the log file above there is a pause in between the branch-and-bound output and SCIP provides more presolve information. This is due to SCIP identifying that it is beneficial to start the branch-and-bound tree again but this time applying information it has learned to the beginning of the search process. In the example above this is explained by the lines:

```
(run 1, node 1) restarting after 26 global fixings of integer variables
(restart) converted 26 cuts from the global cut pool into linear constraints
```

Final Summarised Information

After the branch-and-bound search is complete, SCIP provides a small amount of summarised information that is most important for the majority of users. This includes the status (was the problem proven optimal, or was it infeasible, did we hit a time limit, etc), the total solving time, the amount of nodes explored in the tree (if restarts were used then nodes of the current tree differ from total nodes of all trees), the final primal bound (objective value of the best solution), the

dual bound (the strongest valid bound at the end of the solving process), and finally the gap (the relative difference between the primal and dual bound).

How to Redirect SCIP Output

If you do not want this information output to your terminal, then before calling `optimize` one can call the following function:

```
scip.hideOutput()
```

If you want to redirect your output to Python instead of terminal then one can use the function:

```
scip.redirectOutput()
```

Finally, if you'd like to write the log to a file while optimizing, then one can use the function:

```
scip.setLogfile(path_to_file)
```

SCIP Statistics

While much information is available from the log file or can be easily queried from the Model object, more specific information is often difficult to find, e.g., how many cuts of a certain type were applied? For this information one must use the statistics of SCIP. The statistics can be directly printed to terminal or can be written to a file with the following commands:

```
scip.printStatistics()
scip.writeStatistics(filename=path_to_file)
```

1.3.8 Branching Rules

For the following let us assume that a Model object is available, which is created as follows:

```
from pycscipopt import Model, Branchrule, SCIP_RESULT

scip = Model()
```

Contents

- *Branching Rules*
 - *What is Branching*
 - *Example Branching Rule*
 - *Strong Branching Information*

What is Branching

Branching is when an optimization problem is split into smaller subproblems. Traditionally this is done on an integer variable with a fractional LP solution, with two child nodes being created with constraints $x \geq \text{ceil}(\text{frac})$ and $x \leq \text{floor}(\text{frac})$. In SCIP, arbitrary amount of children nodes can be created, and the constraints added the created nodes can also be arbitrary. This is not going to be used in the examples below, but this should be kept in mind when considering your application of your branching rule.

Example Branching Rule

Here we will program a most infeasible branching rule. This rule selects the integer variable whose LP solution is most fractional.

```
import numpy as np

class MostInfBranchRule(Branchrule):

    def __init__(self, scip):
        self.scip = scip

    def branchexeclp(self, allowaddcons):

        # Get the branching candidates. Only consider the number of priority candidates.
        ↪ (they are sorted to be first)
        # The implicit integer candidates in general shouldn't be branched on. Unless
        ↪ specified by the user
        # npriocands and ncands are the same (npriocands are variables that have been
        ↪ designated as priorities)
        branch_cands, branch_cand_sols, branch_cand_fracs, ncands, npriocands,
        ↪ nimplcands = self.scip.getLPBranchCands()

        # Find the variable that is most fractional
        best_cand_idx = 0
        best_dist = np.inf
        for i in range(npriocands):
            if abs(branch_cand_fracs[i] - 0.5) <= best_dist:
                best_dist = abs(branch_cand_fracs[i] - 0.5)
                best_cand_idx = i

        # Branch on the variable with the largest score
        down_child, eq_child, up_child = self.model.branchVarVal(
            branch_cands[best_cand_idx], branch_cand_sols[best_cand_idx])

        return {"result": SCIP_RESULT.BRANCHED}
```

Let's talk about some features of this branching rule. Currently, we only explicitly programmed a single function, which is called `branchexeclp`. This is the function that gets called when branching on an LP optimal solution. While this is the main case, it is not the only case that SCIP handles. What if there was an LP error at the node, or you are given a set of external candidates? For more information on this please read [this page](#).

Now let's discuss what the function returned. We see that it returned a simple dictionary, with the "result" key and a `SCIP_RESULT`. This is because inside the function the child nodes have already been created, and the solver just needs to be made aware of this with the appropriate code.

In the case of something going wrong while branching (and you have not made the children nodes), just simply return `SCIP_RESULT:DIDNOTRUN`. This will then move on to the next branching rule with the next highest priority.

Note

Returning `SCIP_RESULT:DIDNOTRUN` for more complicated components of the branching rule line in `branchexecps` is completely encouraged. It is even strongly suggested if you are doing an LP specific branching rule.

Now we will finally see how to include the branching rule.

```
scip = Model()

most_inf_branch_rule = MostInfBranchRule(scip)
scip.includeBranchrule(most_inf_branch_rule, "mostinf", "custom most infeasible_
↳branching rule",
                        priority=100000000, maxdepth=-1, maxbounddist=1)
```

This function `includeBranchrule` takes the branching rule as an argument, the name (which will be visible in the statistics), a description, the priority (for your rule to be called by default it must be higher than the current highest, which can be quite large), the maxdepth of the branch and bound tree for which the rule still works (-1 for unlimited), and the maxbounddist (We recommend using 1 and to see SCIP documentation for an explanation).

Strong Branching Information

Now let's look at a more complicated example, namely one where we implement our own strong branching rule. The aim of this example is to provide a basic understanding of what functions are necessary to use strong branching or obtain some strong branching information.

Note

This example is not equivalent to the strong branching rule in SCIP. It ignores some of the more complicated interactions in a MIP solver for information found during strong branching. These include but are not strictly limited to:

- What happens if a new primal solution is found and the bound is larger than the cutoff bound?
- What happens if the bound for one of the children is above a cutoff bound?
- If probing is enabled then one would need to handle new-found bounds appropriately.

```
class StrongBranchingRule(Branchrule):

    def __init__(self, scip):
        self.scip = scip

    def branchexclp(self, allowaddcons):

        branch_cands, branch_cand_sols, branch_cand_fracs, ncands, npriocands,
↳nimplcands = self.scip.getLPBranchCands()

        # Initialise scores for each variable
        scores = [-self.scip.infinity() for _ in range(npriocands)]
        down_bounds = [None for _ in range(npriocands)]
        up_bounds = [None for _ in range(npriocands)]

        # Initialise placeholder values
        num_nodes = self.scip.getNNodes()
        lpobjval = self.scip.getLPObjVal()
        lperror = False
        best_cand_idx = 0

        # Start strong branching and iterate over the branching candidates
```

(continues on next page)

(continued from previous page)

```

self.scip.startStrongbranch()
for i in range(npriocands):

    # Check the case that the variable has already been strong branched on at
    ↪ this node.
    # This case occurs when events happen in the node that should be handled
    ↪ immediately.
    # When processing the node again (because the event did not remove it), there
    ↪ 's no need to duplicate work.
    if self.scip.getVarStrongbranchNode(branch_cands[i]) == num_nodes:
        down, up, downvalid, upvalid, _, lastlpobjval = self.scip.
    ↪ getVarStrongbranchLast(branch_cands[i])
        if downvalid:
            down_bounds[i] = down
        if upvalid:
            up_bounds[i] = up
        downgain = max([down - lastlpobjval, 0])
        upgain = max([up - lastlpobjval, 0])
        scores[i] = self.scip.getBranchScoreMultiple(branch_cands[i], [downgain,
    ↪ upgain])
        continue

    # Strong branch!
    down, up, downvalid, upvalid, downinf, upinf, downconflict, upconflict,
    ↪ lperror = self.scip.getVarStrongbranch(
        branch_cands[i], 200, idempotent=False)

    # In the case of an LP error handle appropriately (for this example we just
    ↪ break the loop)
    if lperror:
        break

    # In the case of both infeasible sub-problems cutoff the node
    if downinf and upinf:
        return {"result": SCIP_RESULT.CUTOFF}

    # Calculate the gains for each up and down node that strong branching
    ↪ explored
    if not downinf and downvalid:
        down_bounds[i] = down
        downgain = max([down - lpobjval, 0])
    else:
        downgain = 0
    if not upinf and upvalid:
        up_bounds[i] = up
        upgain = max([up - lpobjval, 0])
    else:
        upgain = 0

    # Update the pseudo-costs
    lpso1 = branch_cands[i].getLPSol()
    if not downinf and downvalid:

```

(continues on next page)

(continued from previous page)

```

        self.scip.updateVarPseudocost(branch_cands[i], -self.scip.frac(lpsol),
↪ downgain, 1)
        if not upinf and upvalid:
            self.scip.updateVarPseudocost(branch_cands[i], 1 - self.scip.frac(lpsol),
↪ upgain, 1)

        scores[i] = self.scip.getBranchScoreMultiple(branch_cands[i], [downgain,
↪ upgain])
        if scores[i] > scores[best_cand_idx]:
            best_cand_idx = i

        # End strong branching
        self.scip.endStrongbranch()

        # In the case of an LP error
        if lperror:
            return {"result": SCIP_RESULT.DIDNOTRUN}

        # Branch on the variable with the largest score
        down_child, eq_child, up_child = self.model.branchVarVal(
            branch_cands[best_cand_idx], branch_cands[best_cand_idx].getLPSol())

        # Update the bounds of the down node and up node. Some cols might not exist due_
↪ to pricing
        if self.scip.allColsInLP():
            if down_child is not None and down_bounds[best_cand_idx] is not None:
                self.scip.updateNodeLowerbound(down_child, down_bounds[best_cand_idx])
            if up_child is not None and up_bounds[best_cand_idx] is not None:
                self.scip.updateNodeLowerbound(up_child, up_bounds[best_cand_idx])

        return {"result": SCIP_RESULT.BRANCHED}

```

Note

In SCIP we must call `startStrongbranch` before doing any actual strong branching (which is done with the call `getVarStrongbranch`). When we're done with strong branching we must then also call `endStrongbranch`.

1.3.9 Cut Selector

For the following let us assume that a `Model` object is available, which is created as follows:

```

from pycipopt import Model, SCIP_RESULT
from pycipopt.scip import Cutsel

scip = Model()

```

Contents

- *Cut Selector*

- *What is a Cut Selector?*
- *Cut Selector Structure*
- *Example Cut Selector*
- *Things to Keep in Mind*

What is a Cut Selector?

A cutting plane (cut) selector is an algorithm that selects which cuts to add to the optimization problem. It is given a set of candidate cuts, and from the set must decide which subset to add.

Cut Selector Structure

A cut selector in PySCIPOpt takes the following structure:

```
class DummyCutsel(Cutsel):

    def cutselselect(self, cuts, forcedcuts, root, maxnselectedcuts):
        """
        :param cuts: the cuts which we want to select from. Is a list of scip Rows
        :param forcedcuts: the cuts which we must add. Is a list of scip Rows
        :param root: boolean indicating whether we are at the root node
        :param maxnselectedcuts: int which is the maximum amount of cuts that can be
        ↪selected
        :return: sorted cuts and forcedcuts
        """

        return {'cuts': sorted_cuts, 'nselectedcuts': n,
                'result': SCIP_RESULT.SUCCESS}
```

The class `DummyCutsel` inherits the necessary `Cutsel` class, and then programs the necessary function `cutselselect`. The docstrings of the `cutselselect` explain the input to the function. It is then up to the user to create some new ordering cuts, which we have represented by `sorted_cuts`. The returned value of `nselectedcuts` results in the first `nselectedcuts` of the `sorted_cuts` being added to the optimization problem. The `SCIP_RESULT` is there to indicate whether the algorithm was successful. See the appropriate documentation for more potential result codes.

To include a cut selector one would need to do something like the following code:

```
cutsel = DummyCutsel()
scip.includeCutsel(cutsel, 'name', 'description', 50000000)
```

The final argument of the `includeCutsel` function in the example above was the priority. If the priority is higher than all other cut selectors then it will be called first. In the case of some failure or non-success return code, then the second-highest priority cut selector is called and so on.

Example Cut Selector

In this example we will program a cut selector that selects the 10 most efficacious cuts. Efficacy is the standard measure for cut quality and can be calculated via SCIP directly.

```
class MaxEfficacyCutsel(Cutsel):
```

(continues on next page)

(continued from previous page)

```

def cutselselect(self, cuts, forcedcuts, root, maxnselectedcuts):
    """
    Selects the 10 cuts with largest efficacy.
    """

    scip = self.model

    scores = [0] * len(cuts)
    for i in range(len(scores)):
        scores[i] = scip.getCutEfficacy(cuts[i])

    rankings = sorted(range(len(cuts)), key=lambda x: scores[x], reverse=True)

    sorted_cuts = [cuts[rank] for rank in rankings]

    assert len(sorted_cuts) == len(cuts)

    return {'cuts': sorted_cuts, 'nselectedcuts': min(maxnselectedcuts, len(cuts),
↪10),
            'result': SCIP_RESULT.SUCCESS}

```

Things to Keep in Mind

Here are some things to keep in mind when programming your own custom cut selector.

- Do not change any of the actual cut information!
- Do not reorder the forcedcuts. They are provided as reference points to inform the selection process. They should not be edited or reordered.
- Only reorder cuts. Do not add any new cuts.

1.3.10 Separator (Cutting Planes)

For the following let us assume that a Model object is available, which is created as follows:

```

from pyscipopt import Model, Sepa, SCIP_RESULT

scip = Model()

```

Contents

- *Separator (Cutting Planes)*
 - *What is a Separator?*
 - *Gomory Mixed-Integer Cut Example*

What is a Separator?

A separator is an algorithm for generating cutting planes (often abbreviated as cuts). A cut is an inequality that does not remove any feasible solutions of the optimization problem but is intended to remove some fractional solutions from the relaxation. For the purpose of this introduction we restrict ourselves to linear cuts in this paper. A cut would then be denoted by an array of coefficients ($\alpha \in \mathbb{R}^n$) on each variable and a right-hand-side value ($\beta \in \mathbb{R}$).

$$\alpha^T \mathbf{x} \leq \beta$$

The purpose of a separator is to find cuts that tighten the relaxation of an optimization problem. Most commonly, cuts are found that separate the current fractional feasible solution to the relaxation, thereby making a tighter relaxation. For this reason algorithms that find cuts are often called separators.

Gomory Mixed-Integer Cut Example

In this example we show to generate one of the most prolific class of cutting planes, the Gomory mixed-integer (GMI) cut. This example looks complicated, and that's simply because it is. While in theory a GMI cut can be quickly written down, that assumes that your basis information is nicely accessible in the form that you want and that your problem is in standard form. For this reason the code needs to be quite substantial to construct the cuts from scratch.

Note

Separators are one of the most difficult components of a MIP solver to program due to their correctness being difficult to verify, and numerics being a constant issue.

```
class GMI(Sepa):

    def __init__(self):
        self.ncuts = 0

    def getGMIFromRow(self, cols, rows, binvrow, binvarow, primsol):
        """ Given the row (binvarow, binvrow) of the tableau, computes gomory cut

        :param primsol: is the rhs of the tableau row.
        :param cols: are the variables
        :param rows: are the slack variables
        :param binvrow: components of the tableau row associated to the basis inverse
        :param binvarow: components of the tableau row associated to the basis inverse *1

        The GMI is given by
        sum(f_j x_j, j in J_I s.t. f_j <= f_0) +
        sum((1-f_j)*f_0/(1 - f_0) x_j, j in J_I s.t. f_j > f_0) +
        sum(a_j x_j, j in J_C s.t. a_j >= 0) -
        sum(a_j*f_0/(1-f_0) x_j, j in J_C s.t. a_j < 0) >= f_0.
        where J_I are the integer non-basic variables and J_C are the continuous.
        f_0 is the fractional part of primsol
        a_j is the j-th coefficient of the row and f_j its fractional part
        Note: we create -% <= -f_0 !!
        Note: this formula is valid for a problem of the form Ax = b, x >= 0. Since we do1
        not have
        such problem structure in general, we have to (implicitly) transform whatever we1
        are given
```

(continues on next page)

(continued from previous page)

```

to that form. Specifically, non-basic variables at their lower bound are shifted_
↳so that the lower
bound is 0 and non-basic at their upper bound are complemented.
"""

# initialize
cutcoefs = [0] * len(cols)
cutrhs = 0

# get scip
scip = self.model

# Compute cut fractionality f0 and f0/(1-f0)
f0 = scip.frac(primsol)
ratiof0compl = f0/(1-f0)

# rhs of the cut is the fractional part of the LP solution for the basic variable
cutrhs = -f0

# Generate cut coefficients for the original variables
for c in range(len(cols)):
    col = cols[c]
    assert col is not None
    status = col.getBasisStatus()

    # Get simplex tableau coefficient
    if status == "lower":
        # Take coefficient if nonbasic at lower bound
        rowelem = binvarow[c]
    elif status == "upper":
        # Flip coefficient if nonbasic at upper bound: x --> u - x
        rowelem = -binvarow[c]
    else:
        # variable is nonbasic free at zero -> cut coefficient is zero, skip OR
        # variable is basic, skip
        assert status == "zero" or status == "basic"
        continue

    # Integer variables
    if col.isIntegral():
        # warning: because of numerics cutelem < 0 is possible (though the_
↳fractional part is, mathematically, always positive)
        # However, when cutelem < 0 it is also very close to 0, enough that_
↳isZero(cutelem) is true, so we ignore
        # the coefficient (see below)
        cutelem = scip.frac(rowelem)

        if cutelem > f0:
            # sum((1-f_j)*f_0/(1 - f_0) x_j, j in J_I s.t. f_j > f_0) +
            cutelem = -((1.0 - cutelem) * ratiof0compl)
        else:
            # sum(f_j x_j
            , j in J_I s.t. f_j <= f_0) +

```

(continues on next page)

(continued from previous page)

```

        cutelem = -cutelem
    else:
        # Continuous variables
        if rowelem < 0.0:
            # -sum(a_j*f_0/(1-f_0) x_j, j in J_C s.t. a_j < 0) >= f_0.
            cutelem = rowelem * ratioof0compl
        else:
            # sum(a_j x_j, j in J_C s.t. a_j >= 0) -
            cutelem = -rowelem

    # cut is define when variables are in [0, infty). Translate to general bounds
    if not scip.isZero(cutelem):
        if col.getBasisStatus() == "upper":
            cutelem = -cutelem
            cutrhs += cutelem * col.getUb()
        else:
            cutrhs += cutelem * col.getLb()
        # Add coefficient to cut in dense form
        cutcoefs[col.getLPPos()] = cutelem

    # Generate cut coefficients for the slack variables; skip basic ones
    for c in range(len(rows)):
        row = rows[c]
        assert row != None
        status = row.getBasisStatus()

        # free slack variable shouldn't appear
        assert status != "zero"

        # Get simplex tableau coefficient
        if status == "lower":
            # Take coefficient if nonbasic at lower bound
            rowelem = binvrow[row.getLPPos()]
            # But if this is a >= or ranged constraint at the lower bound, we have
            # to flip the row element
            if not scip.isInfinity(-row.getLhs()):
                rowelem = -rowelem
            elif status == "upper":
                # Take element if nonbasic at upper bound - see notes at beginning of
                # file: only nonpositive slack variables
                # can be nonbasic at upper, therefore they should be flipped twice and
                # we can take the element directly.
                rowelem = binvrow[row.getLPPos()]
            else:
                assert status == "basic"
                continue

        # if row is integral we can strengthen the cut coefficient
        if row.isIntegral() and not row.isModifiable():
            # warning: because of numerics cutelem < 0 is possible (though the
            # fractional part is, mathematically, always positive)
            # However, when cutelem < 0 it is also very close to 0, enough that

```

(continues on next page)

(continued from previous page)

```

→isZero(cutelem) is true (see later)
    cutelem = scip.frac(rowelem)

    if cutelem > f0:
        # sum((1-f_j)*f_0/(1-f_0) x_j, j in J_I s.t. f_j > f_0) +
        cutelem = -((1.0 - cutelem) * ratiof0compl)
    else:
        # sum(f_j x_j, j in J_I s.t. f_j <= f_0) +
        cutelem = -cutelem
else:
    # Continuous variables
    if rowelem < 0.0:
        # -sum(a_j*f_0/(1-f_0) x_j, j in J_C s.t. a_j < 0) >= f_0.
        cutelem = rowelem * ratiof0compl
    else:
        # sum(a_j x_j, j in J_C s.t. a_j >= 0) -
        cutelem = -rowelem

# cut is define in original variables, so we replace slack by its definition
if not scip.isZero(cutelem):
    # get lhs/rhs
    rlhs = row.getLhs()
    rrhs = row.getRhs()
    assert scip.isLE(rlhs, rrhs)
    assert not scip.isInfinity(rlhs) or not scip.isInfinity(rrhs)

    # If the slack variable is fixed, we can ignore this cut coefficient
    if scip.isFeasZero(rrhs - rlhs):
        continue

    # Unflip slack variable and adjust rhs if necessary: row at lower means
    →the slack variable is at its upper bound.
    # Since SCIP adds +1 slacks, this can only happen when constraints have
    →a finite lhs
    if row.getBasisStatus() == "lower":
        assert not scip.isInfinity(-rlhs)
        cutelem = -cutelem

    rowcols = row.getCols()
    rowvals = row.getVals()

    assert len(rowcols) == len(rowvals)

    # Eliminate slack variable: rowcols is sorted: [columns in LP, columns
    →not in LP]
    for i in range(row.getNLPNonz()):
        cutcoefs[rowcols[i].getLPPos()] -= cutelem * rowvals[i]

    act = scip.getRowLPActivity(row)
    rhsslack = rrhs - act
    if scip.isFeasZero(rhsslack):
        assert row.getBasisStatus() == "upper" # cutelem != 0 and row active

```

(continues on next page)

(continued from previous page)

```

→at upper bound -> slack at lower, row at upper
    cutrhs -= cutelem * (rrhs - row.getConstant())
    else:
        assert scip.isFeasZero(act - rlhs)
        cutrhs -= cutelem * (rlhs - row.getConstant())

    return cutcoefs, cutrhs

def sepaexeclp(self):
    result = SCIP_RESULT.DIDNOTRUN
    scip = self.model

    if not scip.isLPSolBasic():
        return {"result": result}

    # get LP data
    cols = scip.getLPColsData()
    rows = scip.getLPRowsData()

    # exit if LP is trivial
    if len(cols) == 0 or len(rows) == 0:
        return {"result": result}

    result = SCIP_RESULT.DIDNOTFIND

    # get basis indices
    basisind = scip.getLPBasisInd()

    # For all basic columns (not slacks) belonging to integer variables, try to
    →generate a gomory cut
    for i in range(len(rows)):
        tryrow = False
        c = basisind[i]

        if c >= 0:
            assert c < len(cols)
            var = cols[c].getVar()

            if var.vtype() != "CONTINUOUS":
                primsol = cols[c].getPrimsol()
                assert scip.getSolVal(None, var) == primsol

                if 0.005 <= scip.frac(primsol) <= 1 - 0.005:
                    tryrow = True

        # generate the cut!
        if tryrow:
            # get the row of B^-1 for this basic integer variable with fractional
            →solution value
            binvrow = scip.getLPBInvRow(i)

            # get the tableau row for this basic integer variable with fractional

```

(continues on next page)

(continued from previous page)

```

↪solution value
        binvarow = scip.getLPBInvARow(i)

        # get cut's coefficients
        cutcoefs, cutrhs = self.getGMIFromRow(cols, rows, binvrow, binvarow,
↪primsol)

        # add cut
        cut = scip.createEmptyRowSepsa(self, "gmi%d_x%d"%(self.ncuts,c if c >= 0,
↪else -c-1), lhs = None, rhs = cutrhs)
        scip.cacheRowExtensions(cut)

        for j in range(len(cutcoefs)):
            if scip.isZero(cutcoefs[j]): # maybe here we need isFeasZero
                continue
            scip.addVarToRow(cut, cols[j].getVar(), cutcoefs[j])

        if cut.getNNonz() == 0:
            assert scip.isFeasNegative(cutrhs)
            return {"result": SCIP_RESULT.CUTOFF}

        # Only take efficacious cuts, except for cuts with one non-zero
↪coefficient (= bound changes)
        # the latter cuts will be handled internally in sepastore.
        if cut.getNNonz() == 1 or scip.isCutEfficacious(cut):

            # flush all changes before adding the cut
            scip.flushRowExtensions(cut)

            infeasible = scip.addCut(cut, forcecut=True)
            self.ncuts += 1

            if infeasible:
                result = SCIP_RESULT.CUTOFF
            else:
                result = SCIP_RESULT.SEPARATED
            scip.releaseRow(cut)

        return {"result": result}

```

The GMI separator can then be included using the following code:

```

sepa = GMI()
scip.includeSepsa(sepa, "python_gmi", "generates gomory mixed integer cuts",
↪priority=1000, freq=1)

```

1.3.11 Heuristics

For the following let us assume that a Model object is available, which is created as follows:

```
from pyscipopt import Model, Heur, SCIP_RESULT, SCIP_HEURTIMING, SCIP_LPSOLSTAT

scip = Model()
```

Contents

- *Heuristics*
 - *What is a Heuristic?*
 - *Simple Rounding Heuristic Example*

What is a Heuristic?

A (primal) heuristic is an algorithm for finding a feasible solution to an optimization problem at lower computational costs than their exact counterparts but without any optimality guarantees. The reason that heuristics are implemented in exact optimization solvers are two-fold. It is advantageous for certain algorithms to have a good intermediate solution, and it is helpful for users that they can halt the solving process and access the current best solution.

Simple Rounding Heuristic Example

In this example we show how to implement a simple rounding heuristic in SCIP. The rounding heuristic will take all the fractional variables with integer requirements from the current relaxation solution, and attempt to round them to their nearest integer values.

```
class SimpleRoundingHeuristic(Heur):

    def heurexec(self, heurtiming, nodeinfeasible):

        scip = self.model
        result = SCIP_RESULT.DIDNOTRUN

        # This heuristic does not run if the LP status is not optimal
        lpsolstat = scip.getLPSolstat()
        if lpsolstat != SCIP_LPSOLSTAT.OPTIMAL:
            return {"result": result}

        # We haven't added handling of implicit integers to this heuristic
        if scip.getNImplVars() > 0:
            return {"result": result}

        # Get the current branching candidate, i.e., the current fractional variables,
        ↪ with integer requirements
        branch_cands, branch_cand_sols, branch_cand_fracs, ncands, npriocands,
        ↪ nimplcands = scip.getLPBranchCands()

        # Ignore if there are no branching candidates
        if ncands == 0:
            return {"result": result}

        # Create a solution that is initialised to the LP values
        sol = scip.createSol(self, initlp=True)
```

(continues on next page)

(continued from previous page)

```

# Now round the variables that can be rounded
for i in range(ncands):
    old_sol_val = branch_cand_sols[i]
    scip_var = branch_cands[i]
    may_round_up = scip_var.varMayRound(direction="up")
    may_round_down = scip_var.varMayRound(direction="down")
    # If we can round in both directions then round in objective function_
    ↪direction
    if may_round_up and may_round_down:
        if scip_var.getObj() >= 0.0:
            new_sol_val = scip.feasFloor(old_sol_val)
        else:
            new_sol_val = scip.feasCeil(old_sol_val)
    elif may_round_down:
        new_sol_val = scip.feasFloor(old_sol_val)
    elif may_round_up:
        new_sol_val = scip.feasCeil(old_sol_val)
    else:
        # The variable cannot be rounded. The heuristic will fail.
        continue

    # Set the rounded new solution value
    scip.setSolVal(sol, scip_var, new_sol_val)

# Now try the solution. Note: This will free the solution afterwards by default.
stored = scip.trySol(sol)

if stored:
    return {"result": SCIP_RESULT.FOUNDSOL}
else:
    return {"result": SCIP_RESULT.DIDNOTFIND}

```

To include the heuristic in the SCIP model one would use the following code:

```

heuristic = SimpleRoundingHeuristic()
scip.includeHeur(heuristic, "SimpleRounding", "custom heuristic implemented in python",
    ↪"Y",
    timingmask=SCIP_HEURTIMING.DURINGLLOOP)

```

Note

The `timingmask` is especially important when programming your own heuristic. See [here](#) for information on timing options and how they affect when the heuristic can be called. Note also that heuristics are, as other plugins, called in order of their priorities.

Note

When you create a SCIP solution object it is important that you eventually free the object. This is done by calling `scip.freeSol(sol)`, although this is not necessary when the solution has been passed to `scip.trySol(sol)`.

with `free=True` (default behavior).

1.3.12 Node Selector

For the following let us assume that a `Model` object is available, which is created as follows:

```
from pysciptopt import Model
from pysciptopt.scip import Nodesel

scip = Model()
```

Contents

- *Node Selector*
 - *What is a Node Selector?*
 - *Example Node Selector*

What is a Node Selector?

In the branch-and-bound tree an important question that must be answered is which node should currently be processed. That is, given a branch-and-bound tree in an intermediate state, select a leaf node of the tree that will be processed next (most likely branched on). In SCIP this problem has its own plug-in, and thus custom algorithms can easily be included into the solving process!

Example Node Selector

In this example we are going to implement a depth first search node selection strategy. There are two functions that we need to code ourselves when adding such a rule from python. The first is the strategy on which node to select from all the current leaves, and the other is a comparison function that decides which node is preferred from two candidates.

```
# Depth First Search Node Selector
class DFS(Nodesel):

    def __init__(self, scip, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.scip = scip

    def nodeselect(self):
        """Decide which of the leaves from the branching tree to process next"""
        selnode = self.scip.getPrioChild()
        if selnode is None:
            selnode = self.scip.getPrioSibling()
            if selnode is None:
                selnode = self.scip.getBestLeaf()

        return {"selnode": selnode}

    def nodecomp(self, node1, node2):
        """
```

(continues on next page)

(continued from previous page)

```

compare two leaves of the current branching tree

It should return the following values:

value < 0, if node 1 comes before (is better than) node 2
value = 0, if both nodes are equally good
value > 0, if node 1 comes after (is worse than) node 2.
"""
depth_1 = node1.getDepth()
depth_2 = node2.getDepth()
if depth_1 > depth_2:
    return -1
elif depth_1 < depth_2:
    return 1
else:
    lb_1 = node1.getLowerbound()
    lb_2 = node2.getLowerbound()
    if lb_1 < lb_2:
        return -1
    elif lb_1 > lb_2:
        return 1
    else:
        return 0

```

Note

In general when implementing a node selection rule you will commonly use either `getPrioChild` or `getBestChild`. The first returns the child of the current node with the largest node selection priority, as assigned by the branching rule. The second returns the best child of the current node with respect to the node selector's ordering relation as defined in `nodecomp`.

To include the node selector in your SCIP Model one would use the following code:

```

dfs_node_sel = DFS(scip)
scip.includeNodesel(dfs_node_sel, "DFS", "Depth First Search Nodesel.", 1000000, 1000000)

```

For a more complex example, see the [Hybrid Estimate Node Selector](#) on GitHub.

1.3.13 Lazy Constraints (via Constraint Handler)

For the following let us assume that a Model object is available, which is created as follows:

```

from pycscipopt import Model, quicksum, Conshdlr, SCIP_RESULT

scip = Model()

```

Contents

- *Lazy Constraints (via Constraint Handler)*

- *What are Lazy Constraints?*
- *Why use a Constraint Handler?*
- *TSP Subtour Elimination Constraint Example*

What are Lazy Constraints?

A lazy constraint is a constraint that the user believes improves computational times when removed from the optimization problem until it is violated. Generally, lazy constraints exist in an incredibly large amount, with most of them being trivially satisfied.

Why use a Constraint Handler?

SCIP does not have a lazy constraint plug-in, rather its definition of constraint is broad enough to naturally encompass lazy constraints already. Therefore, the user must simply create an appropriate constraint handler.

TSP Subtour Elimination Constraint Example

In this example we will examine a basic TSP integer programming formulation, where the exponential amount of subtour elimination constraints are treated as lazy constraints.

TSP (short for travelling salesman problem) is a classic optimization problem. Let $n \in \mathbb{N}$ be the number of nodes in a graph with vertex set $\mathcal{V} = \{1, \dots, n\}$ and assume that the graph is complete, i.e., any two vertices are connected by an edge. Each edge (i, j) has an associated cost $c_{i,j} \in \mathbb{R}$ and an associated binary variable $x_{i,j}$. A standard integer programming formulation for the problem is:

$$\begin{aligned}
 & \min \\
 & \sum_{i=1}^n \sum_{j=1}^n c_{i,j} x_{i,j} \\
 & \text{s.t.} \\
 & \sum_{i=1}^n x_{i,j} = 2, \quad \forall j \in \mathcal{V} \\
 & \sum_{i,j \in \mathcal{S}} x_{i,j} \leq |\mathcal{S}| - 1, \quad \forall \mathcal{S} \subset \mathcal{V}, |\mathcal{S}| \geq 2 \quad (*) \\
 & x_{i,j} \in \{0, 1\}, \quad \forall (i, j) \in \mathcal{V} \times \mathcal{V}
 \end{aligned}$$

In the above formulation, the second set of constraints (marked with an $*$) are called subtour elimination constraints. They are called such as a valid solution in absence of those constraints might consist of a collection of smaller cycles instead of a single large cycle. As the constraint set requires checking every subset of nodes, there are exponentially many. Moreover, we know that most of the constraints are probably unnecessary, because it is clear from the objective that a minimum tour does not exist with a mini-cycle of nodes that are extremely far away from each other. Therefore, we want to model them as lazy constraints!

For modelling these constraints using a constraint handler, the constraint handler needs to be able to answer the following questions:

- Is a given solution feasible?
- If the given solution is not feasible, can you do something to forbid this solution from further consideration?

We will now create the basic model containing all information aside from the constraint handler

```

import numpy as np
import networkx
n = 300
x = {}
c = {}
for i in range(n):
    x[i] = {}
    c[i] = {}
    for j in range(i + 1, n):
        x[i][j] = scip.addVar(vtype='B', name=f"x_{i}_{j}")
        c[i][j] = np.random.uniform(10)
scip.setObjective(quicksum(quicksum(c[i][j]*x[i][j] for j in range(i + 1, n)) for i in
↪range(n)), "minimize")
for i in range(n):
    scip.addCons(quicksum(x[i][j] for j in range(i + 1, n)) + quicksum(x[j][i] for j in
↪range(i-1, 0, -1)) == 2,
                  name=f"sum_in_out_{i}")

```

Now we will create the code on how to implement such a constraint handler.

```

# subtour elimination constraint handler
class SEC(Conshdlr):

    # method for creating a constraint of this constraint handler type
    def createCons(self, name, variables):
        model = self.model
        cons = model.createCons(self, name)

        # data relevant for the constraint; in this case we only need to know which
        # variables cannot form a subtour
        cons.data = {}
        cons.data['vars'] = variables
        return cons

    # find subtours in the graph induced by the edges {i,j} for which x[i][j] is positive
    # at the given solution; when solution is None, the LP solution is used
    def find_subtours(self, cons, solution = None):
        edges = []
        x = cons.data['vars']

        for i in list(x.keys()):
            for j in list(x[i].keys()):
                if self.model.getSolVal(solution, x[i][j]) > 0.5:
                    edges.append((i, j))

        G = networkx.Graph()
        G.add_edges_from(edges)
        components = list(networkx.connected_components(G))

        if len(components) == 1:
            return []
        else:

```

(continues on next page)

(continued from previous page)

```

        return components

# checks whether solution is feasible
def conscheck(self, constraints, solution, check_integrality,
              check_lp_rows, print_reason, completely, **results):

    # check if there is a violated subtour elimination constraint
    for cons in constraints:
        if self.find_subtours(cons, solution):
            return {"result": SCIP_RESULT.INFEASIBLE}

    # no violated constraint found -> feasible
    return {"result": SCIP_RESULT.FEASIBLE}

# enforces the LP solution: searches for subtours in the solution and adds
# adds constraints forbidding all the found subtours
def consenfolp(self, constraints, n_useful_conss, sol_infeasible):
    consadded = False

    for cons in constraints:
        subtours = self.find_subtours(cons)

        # if there are subtours
        if subtours:
            x = cons.data['vars']

            # add subtour elimination constraint for each subtour
            for S in subtours:
                print("Constraint added!")
                self.model.addCons(quicksum(x[i][j] for i in S for j in S if j>i) <=
→ len(S)-1)

                consadded = True

    if consadded:
        return {"result": SCIP_RESULT.CONSADDED}
    else:
        return {"result": SCIP_RESULT.FEASIBLE}

# this is rather technical and not relevant for the exercise. to learn more see
# https://scipopt.org/doc/html/CONS.php#CONS_FUNDAMENTALCALLBACKS
def conslock(self, constraint, locktype, nlockspos, nlocksneg):
    pass

```

In the above we've created our problem and custom constraint handler! We now need to actually add the constraint handler to the problem. After that, we can simply call `optimize` whenever we are ready. To add the constraint handler use something along the lines of the following:

```

# create the constraint handler
conshdlr = SEC()

```

(continues on next page)

(continued from previous page)

```
# Add the constraint handler to SCIP. We set check priority < 0 so that only integer
↳feasible solutions
# are passed to the conscheck callback
scip.includeConshdlr(conshdlr, "TSP", "TSP subtour eliminator", chckpriority = -10,
↳enfopriority = -10)

# create a subtour elimination constraint
cons = conshdlr.createCons("no_subtour_cons", x)

# add constraint to SCIP
scip.addPyCons(cons)
```

1.3.14 Event Handlers

For the following let us assume that a Model object is available, which is created as follows:

```
from pycscipopt import Model

scip = Model()
```

Contents

- *Event Handlers*
 - *SCIP Events*
 - *What's an Event Handler?*
 - *Adding Event Handlers with Callbacks*
 - *Adding Event Handlers with Classes*

SCIP Events

SCIP provides a number of events that can be used to interact with the solver. These events would describe a change in the model state before or during the solving process; For example a variable/constraint were added or a new best solution is found. The enum `pycscipopt.SCIPOPT_EVENTTYPE` provides a list of all available events.

What's an Event Handler?

Event handlers are used to react to events that occur during the solving process. They are registered with the solver and are called whenever an event occurs. The event handler can then react to the event by performing some action. For example, an event handler can be used to update the incumbent solution whenever a new best solution is found.

Adding Event Handlers with Callbacks

The easiest way to create an event handler is providing a callback function to the model using the `Model.attachEventHandlerCallback` method. The following is an example the prints the value of the objective function whenever a new best solution is found:

```
from pycscipopt import Model, SCIP_EVENTTYPE
```

(continues on next page)

(continued from previous page)

```
def print_obj_value(model, event):
    print("New best solution found with objective value: {}".format(model.getObjVal()))

m = Model()
m.attachEventHandlerCallback(print_obj_value, [SCIP_EVENTTYPE.BESTSOLFOUND])
m.optimize()
```

The callback function should have the following signature: `def callback(model, event)`. The first argument is the model object and the second argument is the event that occurred.

Adding Event Handlers with Classes

If you need to store additional data in the event handler, you can create a custom event handler class that inherits from `pyscipopt.Eventhdlr`, and then include it in the model using the `Model.includeEventHandler` method. The following is an example that stores the number of best solutions found:

```
from pyscipopt import Model, Eventhdlr, SCIP_EVENTTYPE

class BestSolCounter(Eventhdlr):
    def __init__(self, model):
        Eventhdlr.__init__(model)
        self.count = 0

    def eventinit(self):
        self.model.catchEvent(SCIP_EVENTTYPE.BESTSOLFOUND, self)

    def eventexit(self):
        self.model.dropEvent(SCIP_EVENTTYPE.BESTSOLFOUND, self)

    def eventexec(self, event):
        self.count += 1
        print("!!!![@BestSolCounter] New best solution found. Total best solutions_
↳found: {}".format(self.count))

m = Model()
best_sol_counter = BestSolCounter(m)
m.includeEventhdlr(best_sol_counter, "best_sol_event_handler", "Event handler that_
↳counts the number of best solutions found")
m.optimize()
assert best_sol_counter.count == 1
```

1.3.15 Irreducible Infeasible Subsystems (IIS)

For the following, let us assume that a Model object is available, which is created as follows:

```
from pyscipopt import Model, IISfinder, SCIP_RESULT
model = Model()
```

Contents

- *Irreducible Infeasible Subsystems (IIS)*
 - *What is an IIS?*
 - *Generating an IIS*
 - *The IIS Object*
 - *Creating a Custom IIS Finder*
 - * *Basic Structure*
 - * *Including Your Custom IIS Finder*
 - * *Key Methods in IISfinder*

What is an IIS?

It is a common issue for integer programming practitioners to (unexpectedly) encounter infeasible problems. Often it is desirable to better understand exactly why the problem is infeasible. Was it an error in the input data, was the underlying formulation incorrect, or was the model simply infeasible by construction?

A common tool for helping diagnose the reason for infeasibility is an **Irreducible Infeasible Subsystem (IIS)**. An IIS is a subset of constraints and variable bounds from the original problem that:

1. Remains infeasible when considered together
2. Cannot be further reduced without the subsystem becoming feasible

Practitioners can use IIS finders to narrow their focus onto a smaller, more manageable problem. Note, however, that there are potentially many different irreducible subsystems for a given infeasible problem, and that IIS finders may not provide a guarantee of an IIS of minimum size.

Generating an IIS

Let us create a simple infeasible model and then generate an IIS for it.

```
from pyscipopt import Model

m = Model()
x1 = m.addVar("x1", vtype="B")
x2 = m.addVar("x2", vtype="B")
x3 = m.addVar("x3", vtype="B")

# These four constraints cannot be satisfied simultaneously
m.addCons(x1 + x2 == 1, name="c1")
m.addCons(x2 + x3 == 1, name="c2")
m.addCons(x1 + x3 == 1, name="c3")
m.addCons(x1 + x2 + x3 <= 0, name="c4")

model.optimize()
iis = model.generateIIS()
```

When you run this code, SCIP will output a log showing the progress of finding the IIS:

```

presolving:
presolving (1 rounds: 1 fast, 0 medium, 0 exhaustive):
2 deleted vars, 2 deleted constraints, 0 added constraints, 0 tightened bounds, 0 added
→holes, 0 changed sides, 0 changed coefficients
0 implications, 0 cliques, 0 implied integral variables (0 bin, 0 int, 0 cont)
presolving detected infeasibility
Presolving Time: 0.00

SCIP Status      : problem is solved [infeasible]
Solving Time (sec) : 0.00
Solving Nodes    : 0
Primal Bound     : +1.0000000000000000e+20 (0 solutions)
Dual Bound      : +1.0000000000000000e+20
Gap             : 0.00 %

time(s)| node | cons | vars | bounds| infeasible
0.0|    0|    1|    3|    6|         no
0.0|    0|    2|    3|    6|         no
0.0|    0|    4|    3|    6|         no
0.0|    0|    3|    3|    6|         yes
0.0|    0|    2|    3|    6|         yes
0.0|    0|    2|    3|    6|         yes

IIS Status      : irreducible infeasible subsystem (IIS) found
IIS irreducible : yes
Generation Time (sec) : 0.01
Generation Nodes : 0
Num. Cons. in IIS : 2
Num. Vars. in IIS : 3
Num. Bounds in IIS : 6

```

Note

While an already optimized infeasible model is not required to use the IIS functionality, it is encouraged to call this functionality only after `model.optimize()`. Otherwise, SCIP will naturally optimize the base problem first to ensure that it is actually infeasible.

After SCIP finds that the model is infeasible, see that SCIP's IIS finders alternate between including constraints to make the problem feasible, and removing constraints to make the problem as small as possible. You see in the final statistics that the IIS is indeed irreducible, with 3 variables and 2 constraints.

The IIS Object

The IIS object returned by `generateIIS()` can be queried to access the following information:

- **time**: The CPU time spent finding the IIS
- **irreducible**: Boolean indicating if the IIS is irreducible
- **nodes**: Number of nodes explored during IIS generation
- **model**: A `Model` object containing the subscip with the IIS constraints

You can interact with the subscip to examine which constraints and variables are part of the IIS:

```

iis = model.generateIIS()
subscip = iis.getSubscip()

# Get constraints in the IIS
for cons in subscip.getConss():
    print(f"Constraint: {cons.name}")

# Get variables in the IIS
for var in subscip.getVars():
    print(f"Variable: {var.name}")

```

Creating a Custom IIS Finder

You may want to implement your own algorithm to find an IIS. PySCIPOpt supports this through the `IISfinder` class, which allows you to define custom logic for identifying infeasible subsystems.

Basic Structure

To create a custom IIS finder, inherit from the `IISfinder` class and implement the `iisfinderexec` method:

```

from pyscipopt import Model, IISfinder, SCIP_RESULT

class SimpleIISFinder(IISfinder):
    """
    A simple IIS finder that removes constraints one by one
    until the problem becomes feasible.
    """

    def iisfinderexec(self):
        subscip = self.iis.getSubscip()
        constraints = subscip.getConss()

        # Start with all constraints
        active_constraints = set(constraints)

        for cons in constraints:
            # Temporarily remove the constraint
            active_constraints.discard(cons)

            # Check if remaining constraints are still infeasible
            # (This would require setting up a sub-problem with only active_constraints)
            # For simplicity, we use the full solve approach here

            subscip.freeTransform()
            subscip.delCons(cons)
            subscip.optimize()

            if subscip.getStatus() == SCIP_STATUS.INFEASIBLE:
                # Still infeasible without this constraint
                # Keep it removed
                pass
            else:
                # Feasible without it, so constraint is needed in IIS

```

(continues on next page)

(continued from previous page)

```

        active_constraints.add(cons)
        # In practice, you'd recreate the subscip with all active constraints

    iis.markIrreducible()
    return {"result": SCIP_RESULT.SUCCESS}

```

Including Your Custom IIS Finder

To use your custom IIS finder, include it in the model before calling `generateIIS()`:

```

# Create model
model = Model()

# Add variables and constraints (infeasible problem)
x1 = model.addVar("x1", vtype="B")
x2 = model.addVar("x2", vtype="B")
x3 = model.addVar("x3", vtype="B")

model.addCons(x1 + x2 == 1, name="c1")
model.addCons(x2 + x3 == 1, name="c2")
model.addCons(x1 + x3 == 1, name="c3")

# Create and include the custom IIS finder
simple_iis = SimpleIISFinder()
model.includeIISfinder(
    simple_iis,
    name="simpleiis",
    desc="Simple greedy IIS finder",
    priority=1000000 # Higher priority means it will be used first
)

# Solve to verify infeasibility
model.optimize()

# Generate IIS using our custom finder
iis = model.generateIIS()

# Examine the result
print(f"\nIIS Information:")
print(f"  Time: {iis.getTime():.2f} seconds")
print(f"  Nodes: {iis.getNNodes()}")
print(f"  Irreducible: {iis.isSubscipIrreducible()}")
print(f"  Number of constraints: {iis.getSubscip().getNConss()}")

```

Key Methods in IISfinder

When implementing a custom IIS finder, you have access to several important methods:

- `subscip=self.iis.getSubscip()`: Get the sub-problem (Model) containing the candidate IIS
- `subscip.getConss()`: Get all constraints in the subscip
- `subscip.delCons(cons)`: Remove a constraint from the subscip

- `subscip.addCons(cons)`: Add a constraint back to the subscip
- `subscip.optimize()`: Solve the subscip
- `subscip.getStatus()`: Check if the subscip is infeasible

1.3.16 Hands On Tutorials

For more tutorials with PySCIPOpt, especially interactive ones, please see [SCIPDex](#). SCIPDex is a collection of interactive exercises to get you started (and more) with PySCIPOpt

1.4 Why SCIP?

Note

This page is written for a user that is primarily MILP focused.

“Why SCIP?” is an important question, and one that is in general answered by performance claims. To be clear, SCIP is performant. It is one of the leading open-source solvers. It manages to be competitive on a huge array of benchmarks, which include but are not limited to, mixed-integer linear programming, mixed-integer quadratic programming, mixed-integer semidefinite programming, mixed-integer non-linear programming, and pseudo-boolean optimization. This page will attempt to answer the question “Why SCIP?” without relying on a performance comparison. It will convey the scope of SCIP, how the general structure of SCIP works, and the natural advantages (also weaknesses) SCIP has compared to other mixed-integer optimizers.

So, why SCIP? SCIP (Solving Constraint Integer Programs). The two main points are that SCIP is likely much more general than you expect, and it also likely provides easy to use functionality that you didn’t know was possible.

1.4.1 Differences to Standard MILP Solvers

SCIP (Solving Constraint Integer Programs) is a constraint based solver, it is not a pure MILP solver. This claim is thrown around frequently, but it is difficult to actually understand the difference and the positive + negative outcomes this has on the solving process. Let’s consider a stock standard MILP definition.

$$\begin{aligned}
 &\min \\
 &\mathbf{c}^t x \\
 &\text{s.t.} \\
 &\mathbf{A}x \leq \mathbf{b} \\
 &x \in \mathbb{Z}^{|\mathcal{J}|} \times \mathbb{R}^{[n]/\mathcal{J}}, \quad \mathcal{J} \subseteq [n]
 \end{aligned}$$

When looking at such a problem, one probably thinks of matrices, e.g., the coefficient matrix $\mathbf{A}$. When thinking of the solving process, one probably then thinks of adding cuts and branching. Adding cuts could then be represented by adding additional rows to the problem, and branching represented by creating two subproblems with different variable bounds (the bounds also maybe represented as rows). SCIP’s design is inherently different from such an approach, and does not take a matrix view. SCIP considers how to handle each constraint individually by asking the corresponding constraint handler.

What are the ramifications of this choice? A clear negative ramification is that some matrix operations, many of which are important for quick MILP solving, are less clear on how they’d be implemented and are likely to be less efficient. A clear positive ramification is that now the jump to handling non-linear constraints is easier. If there is no restriction to such a matrix representation, and each constraint is handled individually, then why can the constraints not actually be arbitrary? That’s exactly one of the core strengths of SCIP! This can be seen in the [lazy constraint example](#), where

a single constraint is constructed that will ultimately create up to exponentially many additional linear constraints dynamically in the solving process. To emphasise this point further, one of the most common constraint handlers in SCIP is the integrality constraint handler. It is responsible for telling the solver how to enforce integrality for integer variables with currently fractional solutions. That is, a constraint handler enforces integrality, and therefore it can be transparently decided when to enforce integrality during the solving process.

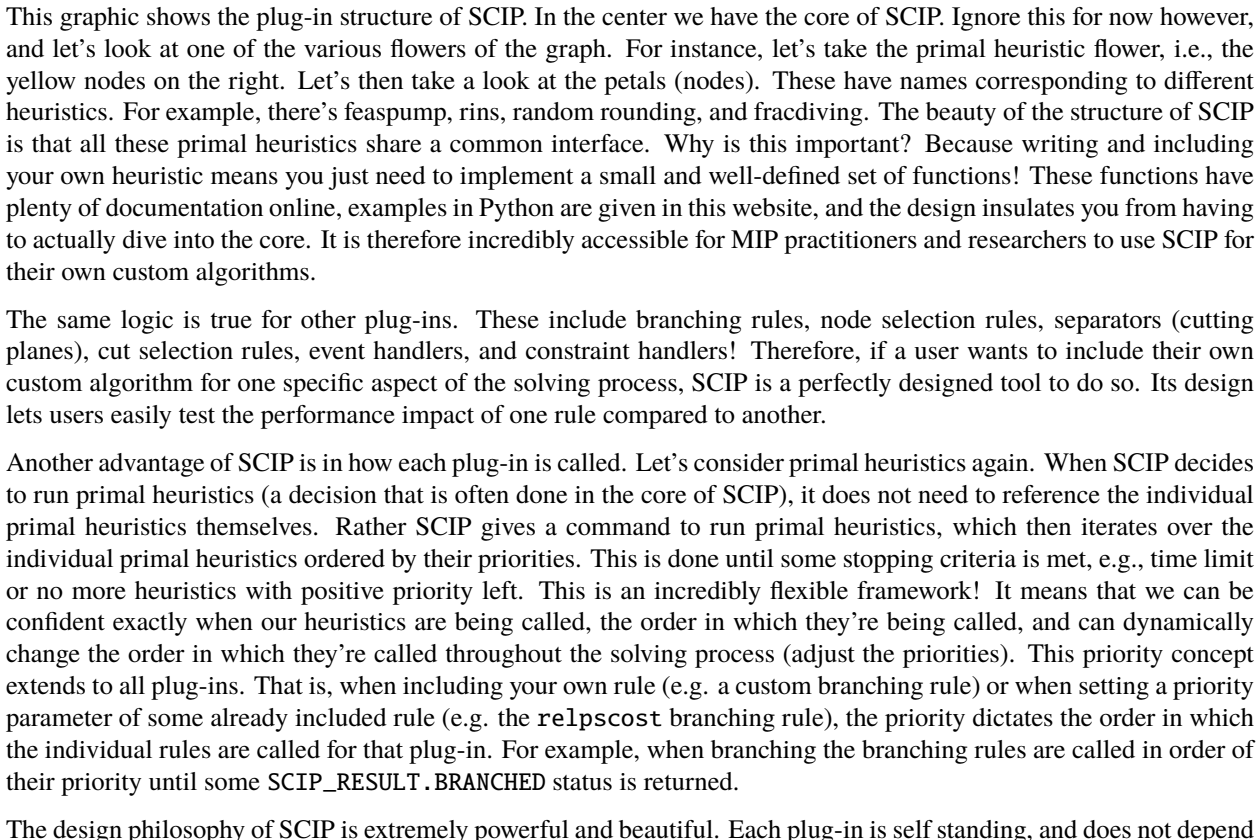
Most probably, when reading the above, an assumption has been made that when solving MILPs one needs access to an LP solver for solving the LP relaxations. SCIP is unusual here on two points. Firstly, SCIP does not have a built-in LP solver, but rather it has an LP interfaces that works with a huge variety of other external LP solvers. This has the huge benefit of separating the reliance on a single LP solver, and allows user to change the underlying LP solver if they wish. This also has a huge downside however, which is that communication with the LP solver is now more computationally costly, and some information from the LP solver may be inaccessible. The second unusual point is that SCIP does not need an LP solver to prove globally optimal solutions. There is an entire field of study parallel to MILP, namely CP (constraint programming). In CP, the problems are solved using propagation to tighten variable domains, and the splitting of the original problem into subproblems, i.e., branching. That is not to say that one should avoid LPs when solving MILPs, rather we claim that in the vast majority of cases the use of LP relaxations is extremely important for good computational performance. It is rather to say that SCIP gives the user a choice on the solving techniques used!

Now on to the final assumption. Above we mentioned that SCIP does not need to rely on an LP solver, and that it can use techniques from the CP community. What if we wanted to use another relaxation technique however? For instance some convex relaxation, but necessarily a linear relaxation? This can be done in SCIP! The functionality exists to write your own relaxator, and decide on what the relaxation of the problem should look like at each node of the tree.

SCIP is therefore a software at the intersection of mixed-integer programming and constraint programming. Its technique of treating the optimization problem as a set of constraints as opposed to an array of inequalities makes it able to naturally handle a wider class of optimization problems.

1.4.2 Modular Design and Plug-Ins

Maybe the biggest advantage of SCIP over other solvers, is for users in the MILP community that want to influence the default solving process in some way. Let's take a look at the following graphic:



on the other plug-ins. The core of SCIP therefore is heavily focused on deciding when to call each plug-in and how to handle the information returned by each plug-in. This philosophy does have one difficulty from a design point of view. Different rules are not supposed to communicate each other. All communication from one rule to another rule or from plug-in to another plug-in must go through the core of SCIP, and thereby have available functionality for all such rules of that plug-in. Much of the most common information is available and some level of communication happens constantly, but the result for users is that if they write two custom rules themselves (maybe for different plug-ins), these rules should not communicate unless general functions for all such rules are written. This is less of an issue for Python users (see the note below).

Note

Because we are using Python, communication between different self-written plug-ins is easy. Let's consider a branching plug-in and a node selection plug-in. If the branching plug-in in Python has access to the node selector object, then it can query information as desired.

The structure of SCIP is therefore extremely modular, extendable, and user-friendly.

1.5 Extending the Interface

PySCIPOpt covers many of the SCIP callable library methods, however it is not complete. One can extend the interface to encompass even more functionality that is possible with SCIP. The following will provide some directions on how this can be achieved.

Contents

- *Extending the Interface*
 - *General File Structure*
 - *Contribution Guidelines*
 - *Design Principles of PySCIPOpt*

1.5.1 General File Structure

The two most important files in PySCIPOpt are the `scip.pxd` and `scip.pxi`. These two files specify the public functions of SCIP that can be accessed from your python code.

To make PySCIPOpt aware of the public functions that one can access, one must add them to `scip.pxd`. There are two things that must be done in order to properly add the functions:

- Ensure any `enum`, `struct` or SCIP variable types are included in `scip.pxd`
- Add the prototype of the public function you wish to access to `scip.pxd`

After following the previous two steps, it is then possible to create functions in python that reference the SCIP public functions included in `scip.pxd`. This is achieved by modifying the `scip.pxi` file to add the functionality you require.

1.5.2 Contribution Guidelines

Code contributions are very welcome and should comply to a few rules:

- Read Design Principles of PySCIPOpt in the section below

- New features should be covered by tests and examples. Please extend the tests and example appropriately. Tests uses pytest and examples are meant to be accessible for PySCIOpt newcomers (even advanced examples).
- New code should be documented in the same style as the rest of the code.
- New features or bugfixes have to be documented in the CHANGELOG.
- New code should be [pep8-compliant](#). Help yourself with the [style guide checker](#).
- Before implementing a new PySCIOpt feature, check whether the feature exists in SCIP. If so, implement it as a pure wrapper, mimicking SCIP whenever possible. If the new feature does not exist in SCIP, but it is close to an existing one, consider if implementing that way is substantially convenient (e.g. Pythonic). If it does something completely different, you are welcome to pull your request and discuss the implementation.
- PySCIOpt uses [semantic versioning](#). Version number increase only happens on master and must be tagged to build a new PyPI release.

For general reference, we suggest:

- [SCIP documentation](#)
- [SCIP mailing list](#)
- [open and closed PySCIOpt issues](#)
- [SCIP/PySCIOpt Stack Exchange](#)

If you find this contributing guide unclear, please open an issue! :)

1.5.3 Design Principles of PySCIOpt

PySCIOpt is meant to be a fast-prototyping interface of the pure SCIP C API. By design, we distinguish different functions in PySCIOPT:

- pure wrapping functions of SCIP;
- convenience functions.

PySCIOpt wrappers of SCIP functions should act:

- with an expected behavior - and parameters, returns, attributes, ... - as close to SCIP as possible
- without “**breaking**” Python and the purpose for what the language it is meant.

Ideally speaking, we want every SCIP function to be wrapped in PySCIOpt.

Convenience functions are additional, non-detrimental features meant to help prototype the Python way. Since these functions are not in SCIP, we wish to limit them to prevent difference in features between SCIP and PySCIOPT, which are always difficult to maintain. A few convenience functions survive in PySCIOpt when keeping them is doubtless beneficial.

Admittedly, there is a middle ground where functions are not completely wrappers or just convenient. That is the case, for instance, of fundamental `Model` methods like `addCons` or `writeProblem`. We want to leave their development to negotiation.

1.6 Similar Software

Contents

- [Similar Software](#)

- *Alternate MIP Solvers (with Python interfaces)*
 - * *Open Source*
 - * *Closed Source*
- *General Modelling Frameworks (Solver Agnostic)*
- *Software using PySCIPOpt*
- *Additional SCIP Resources*

1.6.1 Alternate MIP Solvers (with Python interfaces)

In the following we will give a list of other mixed-integer optimizers with available python interfaces. As each solver has its own set of problem classes that it can solve we will use a table with reference keys to summarize these problem classes.

Note

This table is by no means complete.

Note

SCIP can solve of all the below problem classes (and many more).

Table 7: Label Summaries

Key	Feature
LP	Linear Programs
MILP	Mixed-Integer Linear Programs
QP	Quadratic Programs
MIQP	Mixed-Integer Quadratic Programs
QCP	Quadratically Constrained Programs
MIQCP	Mixed-Integer Quadratically Constrained Programs
MINLP	Mixed-Integer Nonlinear Programs
PB	Pseudo-Boolean Problems
SOCP	Second Order Cone Programming
SDP	Semidefinite Programming
MISDP	Mixed-Integer Semidefinite Programming

Open Source

- [HiGHS](#): O, LP, MILP, QP, MIQP
- [CLP / CBC](#): O, LP, MILP
- [GLOP](#): O, LP

Closed Source

- [CPLEX](#): LP, MILP, QP, MIQP, QCP, MIQCP, SOCP
- [Gurobi](#): LP, MILP, QP, MIQP, QCP, MIQCP, MINLP, PB, SOCP
- [Xpress](#): LP, MILP, QP, MIQP, QCP, MIQCP, MINLP, SOCP
- [COPT](#): LP, MILP, QP, MIQP, QCP, MIQCP, SOCP, SDP
- [MOSEK](#): LP, MILP, QP, MIQP, QCP, MIQCP, SOCP, SDP, MISDP

1.6.2 General Modelling Frameworks (Solver Agnostic)

This list will contain general modelling frameworks that allow you to use swap out SCIP for other mixed-integer optimizers. While we recommend using PySCIPOpt for the many features it provides, which are for the most part not available via general modelling frameworks, if you want to simply use a mixed-integer optimizer then a general modelling framework allows you to swap out the solver in a single line. This is a big advantage if you are uncertain of the solver that you want to use, you believe the solver might change at some point, or you want to compare the performance of different solvers.

- [Pyomo](#)
- [CVXPY](#)
- [LINOPY](#)
- [PULP](#)
- [PICOS](#)
- [AMPL Python API](#)

1.6.3 Software using PySCIPOpt

This is software that is built on PySCIPOpt

- [PyGCGOpt](#): An extension of SCIP, using generic decompositions for solving MIPs
- [GeCO](#): Generators for Combinatorial Optimization
- [scip-routing](#): An exact VRPTW solver in Python
- [PySCIPOpt-ML](#): Python interface to automatically formulate Machine Learning models into Mixed-Integer Programs
- [SCIP Book](#): Mathematical Optimization: Solving Problems using SCIP and Python

1.6.4 Additional SCIP Resources

- [SCIP Website](#)
- [SCIP Documentation](#)
- [SCIP GitHub](#)

1.7 Frequently Asked Questions

In this page we will provide some answers to commonly asked questions by users of PySCIPOpt.

1.7.1 How do I set parameter values?

See the appropriate section on [this page](#).

1.7.2 How do I know the order that the different plug-ins will be called?

Each rule for a given plug-in is called in order of their priority. To ensure that your custom rule is called first it must have a higher priority than all other rules of that plug-in.

1.7.3 Problems with dual values?

See the appropriate section on [this page](#). Short answer: SCIP cannot guarantee accurate precise dual values without certain parameter settings.

1.7.4 Constraints with both LHS and RHS (ranged constraints)

A ranged constraint takes the form:

```
lhs <= expression <= rhs
```

While these are supported, the Python syntax for chained comparisons can't be hijacked with operator overloading. Instead, parenthesis must be used when adding your own ranged constraints, e.g.,

```
lhs <= (expression <= rhs)
```

Alternatively, you may call `scip.chgRhs(cons, newrhs)` or `scip.chgLhs(cons, newlhs)` after the single-sided constraint has been created.

Note

Be careful of constant expressions being rearranged by Python when handling ranged constraints.

1.7.5 My model crashes when I make changes to it after optimizing

When dealing with an already optimized model, and you want to make changes, e.g., add a new constraint or change the objective, please use the following command:

```
scip.freeTransform()
```

Without calling this function the Model can only be queried in its post optimized state. This is because the transformed problem and all the previous solving information is not automatically deleted, and thus stops a new optimization call.

1.7.6 Why can I not add a non-linear objective?

SCIP does not support non-linear objectives. However, an equivalent optimization problem can easily be constructed by introducing a single new variable and a constraint. Please see [this page](#) for a guide.

1.8 Contributors

Contents

- *Contributors*
 - *List of SCIP Authors*
 - *List of PySCIPOpt Contributors*
 - *How to Cite PySCIPOpt / SCIP*

1.8.1 List of SCIP Authors

A list of current and past SCIP developers can be found [here](#).

1.8.2 List of PySCIPOpt Contributors

A list of all contributors to PySCIPOpt can be found [here](#).

1.8.3 How to Cite PySCIPOpt / SCIP

When using PySCIPOpt as part of your research, please cite the following paper:

```
@incollection{MaherMiltenbergerPedrosoRehfeldtSchwarzSerrano2016,
  author = {Stephen Maher and Matthias Miltenberger and Jo{\~{a}}o Pedro Pedroso and
↵Daniel Rehfeldt and Robert Schwarz and Felipe Serrano},
  title = {{PySCIPOpt}: Mathematical Programming in Python with the {SCIP} Optimization
↵Suite},
  booktitle = {Mathematical Software {\textendash} {ICMS} 2016},
  publisher = {Springer International Publishing},
  pages = {301--307},
  year = {2016},
  doi = {10.1007/978-3-319-42432-3_37},
}
```

Additionally, please cite the corresponding SCIP Optimization Suite report, which can be found [here](#).

1.9 API Reference

This page provides an auto-generated summary of PySCIPOpt's API.

1.9.1 SCIP Model

This is the main class of PySCIPOpt. Most functionality is accessible through functions of this class. All functions that require the SCIP object belong to this class.

Model API

class pycscipopt.**Model**

activateBenders(*benders*, *nsubproblems*)

Activates the Benders' decomposition plugin with the input name.

Parameters

- **benders** (*Benders*) – the Benders' decomposition to which the subproblem belongs to
- **nsubproblems** (*int*) – the number of subproblems in the Benders' decomposition

addBendersSubproblem(*benders, subproblem*)

Adds a subproblem to the Benders' decomposition given by the input name.

Parameters

- **benders** (*Benders*) – the Benders' decomposition to which the subproblem belongs to
- **subproblem** (*Model*) – the subproblem to add to the decomposition

addCoefKnapsack(*cons, var, weight*)

Adds coefficient to knapsack constraint (if it is not zero)

Parameters

- **cons** (*Constraint*) – knapsack constraint
- **var** (*Variable*) – variable of constraint entry
- **weight** (*float*) – coefficient of constraint entry

addCoefLinear(*cons, var, value*)

Adds coefficient to linear constraint (if it is not zero)

Parameters

- **cons** (*Constraint*) – linear constraint
- **var** (*Variable*) – variable of constraint entry
- **value** (*float*) – coefficient of constraint entry

addCons(*cons, name="", initial=True, separate=True, enforce=True, check=True, propagate=True, local=False, modifiable=False, dynamic=False, removable=False, stickingatnode=False*)

Add a linear or nonlinear constraint.

Parameters

- **cons** (*ExprCons*) – The expression constraint that is not yet an actual constraint
- **name** (*str, optional*) – the name of the constraint, generic name if empty (Default value = "")
- **initial** (*bool, optional*) – should the LP relaxation of constraint be in the initial LP? (Default value = True)
- **separate** (*bool, optional*) – should the constraint be separated during LP processing? (Default value = True)
- **enforce** (*bool, optional*) – should the constraint be enforced during node processing? (Default value = True)
- **check** (*bool, optional*) – should the constraint be checked for feasibility? (Default value = True)
- **propagate** (*bool, optional*) – should the constraint be propagated during node processing? (Default value = True)
- **local** (*bool, optional*) – is the constraint only valid locally? (Default value = False)
- **modifiable** (*bool, optional*) – is the constraint modifiable (subject to column generation)? (Default value = False)
- **dynamic** (*bool, optional*) – is the constraint subject to aging? (Default value = False)
- **removable** (*bool, optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = False)

- **stickingatnode** (*bool*, *optional*) – should the constraints always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The created and added Constraint object.

Return type

Constraint

addConsAnd(*vars*, *resvar*, *name*="", *initial*=True, *separate*=True, *enforce*=True, *check*=True, *propagate*=True, *local*=False, *modifiable*=False, *dynamic*=False, *removable*=False, *stickingatnode*=False)

Add an AND-constraint.

Parameters

- **vars** (*list of Variable*) – list of BINARY variables to be included (operators)
- **resvar** (*Variable*) – BINARY variable (resultant)
- **name** (*str*, *optional*) – name of the constraint (Default value = "")
- **initial** (*bool*, *optional*) – should the LP relaxation of constraint be in the initial LP? (Default value = True)
- **separate** (*bool*, *optional*) – should the constraint be separated during LP processing? (Default value = True)
- **enforce** (*bool*, *optional*) – should the constraint be enforced during node processing? (Default value = True)
- **check** (*bool*, *optional*) – should the constraint be checked for feasibility? (Default value = True)
- **propagate** (*bool*, *optional*) – should the constraint be propagated during node processing? (Default value = True)
- **local** (*bool*, *optional*) – is the constraint only valid locally? (Default value = False)
- **dynamic** (*bool*, *optional*) – is the constraint subject to aging? (Default value = False)
- **removable** (*bool*, *optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = False)
- **stickingatnode** (*bool*, *optional*) – should the constraint always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The newly created AND constraint

Return type

Constraint

addConsCardinality(*consvars*, *cardval*, *indvars*=None, *weights*=None, *name*="", *initial*=True, *separate*=True, *enforce*=True, *check*=True, *propagate*=True, *local*=False, *dynamic*=False, *removable*=False, *stickingatnode*=False)

Add a cardinality constraint that allows at most ‘cardval’ many nonzero variables.

Parameters

- **consvars** (*list of Variable*) – list of variables to be included
- **cardval** (*int*) – nonnegative integer

- **indvars** (*list of Variable or None, optional*) – indicator variables indicating which variables may be treated as nonzero in cardinality constraint, or None if new indicator variables should be introduced automatically (Default value = None)
- **weights** (*list of float or None, optional*) – weights determining the variable order, or None if variables should be ordered in the same way they were added to the constraint (Default value = None)
- **name** (*str, optional*) – name of the constraint (Default value = “”)
- **initial** (*bool, optional*) – should the LP relaxation of constraint be in the initial LP? (Default value = True)
- **separate** (*bool, optional*) – should the constraint be separated during LP processing? (Default value = True)
- **enforce** (*bool, optional*) – should the constraint be enforced during node processing? (Default value = True)
- **check** (*bool, optional*) – should the constraint be checked for feasibility? (Default value = True)
- **propagate** (*bool, optional*) – should the constraint be propagated during node processing? (Default value = True)
- **local** (*bool, optional*) – is the constraint only valid locally? (Default value = False)
- **dynamic** (*bool, optional*) – is the constraint subject to aging? (Default value = False)
- **removable** (*bool, optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = False)
- **stickingatnode** (*bool, optional*) – should the constraint always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The newly created Cardinality constraint

Return type

Constraint

addConsCoeff(*cons, var, coeff*)

Add coefficient to the constraint (if non-zero).

Parameters

- **cons** (*Constraint*) – Constraint to be changed
- **var** (*Variable*) – variable to be added
- **coeff** (*float*) – coefficient of new variable

addConsDisjunction(*conss, name="", initial=True, relaxcons=None, enforce=True, check=True, local=False, modifiable=False, dynamic=False*)

Add a disjunction constraint.

Parameters

- **conss** (*iterable of ExprCons*) – An iterable of constraint objects to be included initially in the disjunction. Currently, these must be expressions.
- **name** (*str, optional*) – the name of the disjunction constraint.
- **initial** (*bool, optional*) – should the LP relaxation of disjunction constraint be in the initial LP? (Default value = True)

- **relaxcons** (*None*, *optional*) – a conjunction constraint containing the linear relaxation of the disjunction constraint, or *None*. NOT YET SUPPORTED. (Default value = *None*)
- **enforce** (*bool*, *optional*) – should the constraint be enforced during node processing? (Default value = *True*)
- **check** (*bool*, *optional*) – should the constraint be checked for feasibility? (Default value = *True*)
- **local** (*bool*, *optional*) – is the constraint only valid locally? (Default value = *False*)
- **modifiable** (*bool*, *optional*) – is the constraint modifiable (subject to column generation)? (Default value = *False*)
- **dynamic** (*bool*, *optional*) – is the constraint subject to aging? (Default value = *False*)

Returns

The created disjunction constraint

Return type

Constraint

addConsElemDisjunction(*disj_cons*, *cons*)

Appends a constraint to a disjunction.

Parameters

- **disj_cons** (*Constraint*) – the disjunction constraint to append to.
- **cons** (*Constraint*) – the constraint to append

Returns

disj_cons – The disjunction constraint with *cons* appended.

Return type

Constraint

addConsIndicator(*cons*, *binvar=None*, *activeone=True*, *name=""*, *initial=True*, *separate=True*, *enforce=True*, *check=True*, *propagate=True*, *local=False*, *dynamic=False*, *removable=False*, *stickingatnode=False*)

Add an indicator constraint for the linear inequality *cons*.

The *binvar* argument models the redundancy of the linear constraint. A solution for which *binvar* is 1 must satisfy the constraint.

Parameters

- **cons** (*ExprCons*) – a linear inequality of the form “ $\leq$ ”
- **binvar** (*Variable*, *optional*) – binary indicator variable, or *None* if it should be created (Default value = *None*)
- **activeone** (*bool*, *optional*) – constraint should active if *binvar* is 1 (0 if *activeone* = *False*)
- **name** (*str*, *optional*) – name of the constraint (Default value = “”)
- **initial** (*bool*, *optional*) – should the LP relaxation of constraint be in the initial LP? (Default value = *True*)
- **separate** (*bool*, *optional*) – should the constraint be separated during LP processing? (Default value = *True*)
- **enforce** (*bool*, *optional*) – should the constraint be enforced during node processing? (Default value = *True*)

- **check** (*bool, optional*) – should the constraint be checked for feasibility? (Default value = True)
- **propagate** (*bool, optional*) – should the constraint be propagated during node processing? (Default value = True)
- **local** (*bool, optional*) – is the constraint only valid locally? (Default value = False)
- **dynamic** (*bool, optional*) – is the constraint subject to aging? (Default value = False)
- **removable** (*bool, optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = False)
- **stickingatnode** (*bool, optional*) – should the constraint always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The newly created Indicator constraint

Return type

Constraint

addConsKnapsack(*vars, weights, capacity, name="", initial=True, separate=True, enforce=True, check=True, modifiable=False, propagate=True, local=False, dynamic=False, removable=False, stickingatnode=False*)

Parameters

- **vars** (*list of Variable*) – list of variables to be included
- **weights** (*list of int*) – list of weights
- **capacity** (*int*) – capacity of the knapsack
- **name** (*str, optional*) – name of the constraint (Default value = "")
- **initial** (*bool, optional*) – should the LP relaxation of constraint be in the initial LP? (Default value = True)
- **separate** (*bool, optional*) – should the constraint be separated during LP processing? (Default value = True)
- **enforce** (*bool, optional*) – should the constraint be enforced during node processing? (Default value = True)
- **check** (*bool, optional*) – should the constraint be checked for feasibility? (Default value = True)
- **propagate** (*bool, optional*) – should the constraint be propagated during node processing? (Default value = True)
- **local** (*bool, optional*) – is the constraint only valid locally? (Default value = False)
- **dynamic** (*bool, optional*) – is the constraint subject to aging? (Default value = False)
- **removable** (*bool, optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = False)
- **stickingatnode** (*bool, optional*) – should the constraint always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The newly created knapsack constraint

Return type

Constraint

addConsLocal(*cons*, *validnode=None*)

Add a constraint to the current node.

Parameters

- **cons** (*Constraint*) – the constraint to add to the current node
- **validnode** (*Node* or *None*, *optional*) – more global node where cons is also valid. (Default=None)

addConsNode(*node*, *cons*, *validnode=None*)

Add a constraint to the given node.

Parameters

- **node** (*Node*) – node at which the constraint will be added
- **cons** (*Constraint*) – the constraint to add to the node
- **validnode** (*Node* or *None*, *optional*) – more global node where cons is also valid. (Default=None)

addConsOr(*vars*, *resvar*, *name=""*, *initial=True*, *separate=True*, *enforce=True*, *check=True*, *propagate=True*, *local=False*, *modifiable=False*, *dynamic=False*, *removable=False*, *stickingatnode=False*)

Add an OR-constraint.

Parameters

- **vars** (*list of Variable*) – list of BINARY variables to be included (operators)
- **resvar** (*Variable*) – BINARY variable (resultant)
- **name** (*str*, *optional*) – name of the constraint (Default value = "")
- **initial** (*bool*, *optional*) – should the LP relaxation of constraint be in the initial LP? (Default value = True)
- **separate** (*bool*, *optional*) – should the constraint be separated during LP processing? (Default value = True)
- **enforce** (*bool*, *optional*) – should the constraint be enforced during node processing? (Default value = True)
- **check** (*bool*, *optional*) – should the constraint be checked for feasibility? (Default value = True)
- **propagate** (*bool*, *optional*) – should the constraint be propagated during node processing? (Default value = True)
- **local** (*bool*, *optional*) – is the constraint only valid locally? (Default value = False)
- **dynamic** (*bool*, *optional*) – is the constraint subject to aging? (Default value = False)
- **removable** (*bool*, *optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = False)
- **stickingatnode** (*bool*, *optional*) – should the constraint always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The newly created OR constraint

Return type

Constraint

addConsSOS1(vars, weights=None, name="", initial=True, separate=True, enforce=True, check=True, propagate=True, local=False, dynamic=False, removable=False, stickingatnode=False)

Add an SOS1 constraint.

Parameters

- **vars** (list of [Variable](#)) – list of variables to be included
- **weights** (list of float or None, optional) – list of weights (Default value = None)
- **name** (str, optional) – name of the constraint (Default value = "")
- **initial** (bool, optional) – should the LP relaxation of constraint be in the initial LP? (Default value = True)
- **separate** (bool, optional) – should the constraint be separated during LP processing? (Default value = True)
- **enforce** (bool, optional) – should the constraint be enforced during node processing? (Default value = True)
- **check** (bool, optional) – should the constraint be checked for feasibility? (Default value = True)
- **propagate** (bool, optional) – should the constraint be propagated during node processing? (Default value = True)
- **local** (bool, optional) – is the constraint only valid locally? (Default value = False)
- **dynamic** (bool, optional) – is the constraint subject to aging? (Default value = False)
- **removable** (bool, optional) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = False)
- **stickingatnode** (bool, optional) – should the constraint always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The newly created SOS1 constraint

Return type

[Constraint](#)

addConsSOS2(vars, weights=None, name="", initial=True, separate=True, enforce=True, check=True, propagate=True, local=False, dynamic=False, removable=False, stickingatnode=False)

Add an SOS2 constraint.

Parameters

- **vars** (list of [Variable](#)) – list of variables to be included
- **weights** (list of float or None, optional) – list of weights (Default value = None)
- **name** (str, optional) – name of the constraint (Default value = "")
- **initial** (bool, optional) – should the LP relaxation of constraint be in the initial LP? (Default value = True)
- **separate** (bool, optional) – should the constraint be separated during LP processing? (Default value = True)
- **enforce** (bool, optional) – should the constraint be enforced during node processing? (Default value = True)

- **check** (*bool, optional*) – should the constraint be checked for feasibility? (Default value = True)
- **propagate** (*bool, optional*) – should the constraint be propagated during node processing? (Default value = True)
- **local** (*bool, optional*) – is the constraint only valid locally? (Default value = False)
- **dynamic** (*bool, optional*) – is the constraint subject to aging? (Default value = False)
- **removable** (*bool, optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = False)
- **stickingatnode** (*bool, optional*) – should the constraint always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The newly created SOS2 constraint

Return type

Constraint

addConsXor(*vars, rhsvar, name="", initial=True, separate=True, enforce=True, check=True, propagate=True, local=False, modifiable=False, dynamic=False, removable=False, stickingatnode=False*)

Add a XOR-constraint.

Parameters

- **vars** (*list of Variable*) – list of binary variables to be included (operators)
- **rhsvar** (*bool*) – BOOLEAN value, explicit True, False or bool(obj) is needed (right-hand side)
- **name** (*str, optional*) – name of the constraint (Default value = "")
- **initial** (*bool, optional*) – should the LP relaxation of constraint be in the initial LP? (Default value = True)
- **separate** (*bool, optional*) – should the constraint be separated during LP processing? (Default value = True)
- **enforce** (*bool, optional*) – should the constraint be enforced during node processing? (Default value = True)
- **check** (*bool, optional*) – should the constraint be checked for feasibility? (Default value = True)
- **propagate** (*bool, optional*) – should the constraint be propagated during node processing? (Default value = True)
- **local** (*bool, optional*) – is the constraint only valid locally? (Default value = False)
- **dynamic** (*bool, optional*) – is the constraint subject to aging? (Default value = False)
- **removable** (*bool, optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = False)
- **stickingatnode** (*bool, optional*) – should the constraint always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The newly created XOR constraint

Return type

Constraint

addConss(*conss*, *name*="", *initial*=True, *separate*=True, *enforce*=True, *check*=True, *propagate*=True, *local*=False, *modifiable*=False, *dynamic*=False, *removable*=False, *stickingatnode*=False)

Adds multiple constraints.

Each of the constraints is added to the model using `Model.addCons()`.

For all parameters, except *conss*, this method behaves differently depending on the type of the passed argument: 1. If the value is iterable, it must be of the same length as *conss*. For each constraint, `Model.addCons()` will be called with the value at the corresponding index. 2. Else, the (default) value will be applied to all of the constraints.

Parameters

- **conss** (*iterable of ExprCons*) – An iterable of constraint objects. Any iterable will be converted into a list before further processing.
- **name** (*str or iterable of str, optional*) – the name of the constraint, generic name if empty (Default value = "")
- **initial** (*bool or iterable of bool, optional*) – should the LP relaxation of constraint be in the initial LP? (Default value = True)
- **separate** (*bool or iterable of bool, optional*) – should the constraint be separated during LP processing? (Default value = True)
- **enforce** (*bool or iterable of bool, optional*) – should the constraint be enforced during node processing? (Default value = True)
- **check** (*bool or iterable of bool, optional*) – should the constraint be checked for feasibility? (Default value = True)
- **propagate** (*bool or iterable of bool, optional*) – should the constraint be propagated during node processing? (Default value = True)
- **local** (*bool or iterable of bool, optional*) – is the constraint only valid locally? (Default value = False)
- **modifiable** (*bool or iterable of bool, optional*) – is the constraint modifiable (subject to column generation)? (Default value = False)
- **dynamic** (*bool or iterable of bool, optional*) – is the constraint subject to aging? (Default value = False)
- **removable** (*bool or iterable of bool, optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = False)
- **stickingatnode** (*bool or iterable of bool, optional*) – should the constraints always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The created and added Constraint objects.

Return type

list of *Constraint*

addCut(*cut*, *forcecut*=False)

Adds cut to separation storage and returns whether cut has been detected to be infeasible for local bounds.

Parameters

- **cut** (*Row*) – The cut that will be added

- **forcecut** (*bool*, *optional*) – Whether the cut should be forced or not, i.e., selected no matter what

Returns

infeasible – Whether the cut has been detected to be infeasible from local bounds

Return type

bool

addExprNonlinear(*cons*, *expr*, *coef*)

Add $\text{coef} * \text{expr}$ to nonlinear constraint.

Parameters

- **cons** (*Constraint*)
- **expr** (*Expr* or *GenExpr*)
- **coef** (*float*)

addMatrixCons(*cons*: *ExprCons* | *MatrixExprCons*, *name*: *str* | *ndarray* = "", *initial*: *bool* | *ndarray* = *True*, *separate*: *bool* | *ndarray* = *True*, *enforce*: *bool* | *ndarray* = *True*, *check*: *bool* | *ndarray* = *True*, *propagate*: *bool* | *ndarray* = *True*, *local*: *bool* | *ndarray* = *False*, *modifiable*: *bool* | *ndarray* = *False*, *dynamic*: *bool* | *ndarray* = *False*, *removable*: *bool* | *ndarray* = *False*, *stickingatnode*: *bool* | *ndarray* = *False*)

Add a linear or nonlinear matrix constraint.

Parameters

- **cons** (*ExprCons* or *MatrixExprCons*) – The matrix expression constraint or expression constraint, that are not yet an actual constraint
- **name** (*str* or *np.ndarray*, *optional*) – the name of the matrix constraint, generic name if empty (Default value = "")
- **initial** (*bool* or *np.ndarray*, *optional*) – should the LP relaxation of constraint be in the initial LP? (Default value = *True*)
- **separate** (*bool* or *np.ndarray*, *optional*) – should the matrix constraint be separated during LP processing? (Default value = *True*)
- **enforce** (*bool* or *np.ndarray*, *optional*) – should the matrix constraint be enforced during node processing? (Default value = *True*)
- **check** (*bool* or *np.ndarray*, *optional*) – should the matrix constraint be checked for feasibility? (Default value = *True*)
- **propagate** (*bool* or *np.ndarray*, *optional*) – should the matrix constraint be propagated during node processing? (Default value = *True*)
- **local** (*bool* or *np.ndarray*, *optional*) – is the matrix constraint only valid locally? (Default value = *False*)
- **modifiable** (*bool* or *np.ndarray*, *optional*) – is the matrix constraint modifiable (subject to column generation)? (Default value = *False*)
- **dynamic** (*bool* or *np.ndarray*, *optional*) – is the matrix constraint subject to aging? (Default value = *False*)
- **removable** (*bool* or *np.ndarray*, *optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = *False*)

- **stickingatnode** (*bool* or *np.ndarray*, *optional*) – should the matrix constraints always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The created and added Constraint or MatrixConstraint.

Return type

Constraint or *MatrixConstraint*

addMatrixConsIndicator(*cons*: *ExprCons* | *MatrixExprCons*, *binvar*: *Variable* | *MatrixVariable* = *None*, *activeone*: *bool* | *ndarray* = *True*, *name*: *str* | *ndarray* = "", *initial*: *bool* | *ndarray* = *True*, *separate*: *bool* | *ndarray* = *True*, *enforce*: *bool* | *ndarray* = *True*, *check*: *bool* | *ndarray* = *True*, *propagate*: *bool* | *ndarray* = *True*, *local*: *bool* | *ndarray* = *False*, *dynamic*: *bool* | *ndarray* = *False*, *removable*: *bool* | *ndarray* = *False*, *stickingatnode*: *bool* | *ndarray* = *False*) → *MatrixConstraint*

Add an indicator matrix constraint for the linear inequality *cons*.

The *binvar* argument models the redundancy of the linear constraint. A solution for which *binvar* is 1 must satisfy the constraint.

Parameters

- **cons** (*ExprCons* or *MatrixExprCons*) – a linear inequality of the form “ $\leq$ ”.
- **binvar** (*Variable* or *MatrixVariable*, *optional*) – binary indicator variable / matrix variable, or *None* if it should be created. (Default value = *None*)
- **activeone** (*bool* or *np.ndarray*, *optional*) – the matrix constraint should be active if *binvar* is 1 (0 if *activeone* = *False*).
- **name** (*str* or *np.ndarray*, *optional*) – name of the matrix constraint. (Default value = “”)
- **initial** (*bool* or *np.ndarray*, *optional*) – should the LP relaxation of matrix constraint be in the initial LP? (Default value = *True*)
- **separate** (*bool* or *np.ndarray*, *optional*) – should the matrix constraint be separated during LP processing? (Default value = *True*)
- **enforce** (*bool* or *np.ndarray*, *optional*) – should the matrix constraint be enforced during node processing? (Default value = *True*)
- **check** (*bool* or *np.ndarray*, *optional*) – should the matrix constraint be checked for feasibility? (Default value = *True*)
- **propagate** (*bool* or *np.ndarray*, *optional*) – should the matrix constraint be propagated during node processing? (Default value = *True*)
- **local** (*bool* or *np.ndarray*, *optional*) – is the matrix constraint only valid locally? (Default value = *False*)
- **dynamic** (*bool* or *np.ndarray*, *optional*) – is the matrix constraint subject to aging? (Default value = *False*)
- **removable** (*bool* or *np.ndarray*, *optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = *False*)
- **stickingatnode** (*bool* or *np.ndarray*, *optional*) – should the matrix constraint always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = *False*)

Returns

The newly created Indicator MatrixConstraint object.

Return type

MatrixConstraint

addMatrixVar(*shape: int | Tuple, name: str | np.ndarray = "", vtype: str | np.ndarray = 'C', lb: int | float | np.ndarray | None = 0.0, ub: int | float | np.ndarray | None = None, obj: int | float | np.ndarray = 0.0, pricedVar: bool | np.ndarray = False, pricedVarScore: int | float | np.ndarray = 1.0*) → MatrixVariable

Create a new matrix of variable. Default matrix variables are non-negative and continuous.

Parameters

- **shape** (*int or tuple*) – the shape of the resultant MatrixVariable
- **name** (*str or np.ndarray, optional*) – name of the matrix variable, generic if empty (Default value = "")
- **vtype** (*str or np.ndarray, optional*) – type of the matrix variable: 'C' continuous, 'I' integer, 'B' binary, and 'M' implicit integer (Default value = 'C')
- **lb** (*float or np.ndarray or None, optional*) – lower bound of the matrix variable, use None for -infinity (Default value = 0.0)
- **ub** (*float or np.ndarray or None, optional*) – upper bound of the matrix variable, use None for +infinity (Default value = None)
- **obj** (*float or np.ndarray, optional*) – objective value of matrix variable (Default value = 0.0)
- **pricedVar** (*bool or np.ndarray, optional*) – is the matrix variable a pricing candidate? (Default value = False)
- **pricedVarScore** (*float or np.ndarray, optional*) – score of matrix variable in case it is priced, the higher the better (Default value = 1.0)

Return type

MatrixVariable

addObjoffset(*offset, solutions=False*)

Add constant offset to objective.

Parameters

- **offset** (*float*) – offset to add
- **solutions** (*bool, optional*) – add offset also to existing solutions (Default value = False)

addPoolCut(*row*)

If not already existing, adds row to global cut pool.

Parameters

row (*Row*)

addPyCons(*cons*)

Adds a customly created cons.

Parameters

cons (*Constraint*) – constraint to add

addRowDive(*row*)

Adds a row to the LP in current dive.

Parameters

row ([Row](#))

addRowExact(*rowexact*)

Adds an exact row to the LP.

Parameters

rowexact ([RowExact](#)) – The exact row to add.

addSol(*solution*, *free=True*)

Try to add a solution to the storage.

Parameters

- **solution** ([Solution](#)) – solution to store
- **free** (*bool*, *optional*) – should solution be freed afterwards? (Default value = True)

Returns

stored – stores whether given solution was good enough to keep

Return type

bool

addVar(*name=""*, *vtype='C'*, *lb=0.0*, *ub=None*, *obj=0.0*, *pricedVar=False*, *pricedVarScore=1.0*, *deletable=False*)

Create a new variable. Default variable is non-negative and continuous.

Parameters

- **name** (*str*, *optional*) – name of the variable, generic if empty (Default value = "")
- **vtype** (*str*, *optional*) – type of the variable: 'C' continuous, 'I' integer, 'B' binary, and 'M' implicit integer (Default value = 'C')
- **lb** (*float or None*, *optional*) – lower bound of the variable, use None for -infinity (Default value = 0.0)
- **ub** (*float or None*, *optional*) – upper bound of the variable, use None for +infinity (Default value = None)
- **obj** (*float*, *optional*) – objective value of variable (Default value = 0.0)
- **pricedVar** (*bool*, *optional*) – is the variable a pricing candidate? (Default value = False)
- **pricedVarScore** (*float*, *optional*) – score of variable in case it is priced, the higher the better (Default value = 1.0)

Return type

[Variable](#)

addVarLocks(*var*, *nlocksdown*, *nlocksup*)

Adds given values to lock numbers of variable for rounding.

Parameters

- **var** ([Variable](#)) – variable to adjust the locks for
- **nlocksdown** (*int*) – new number of down locks
- **nlocksup** (*int*) – new number of up locks

addVarLocksType(*var*, *locktype*, *nlocksdown*, *nlocksup*)

adds given values to lock numbers of type locktype of variable for rounding

Parameters

- **var** (*Variable*) – variable to adjust the locks for
- **locktype** (*SCIP_LOCKTYPE*) – type of variable locks
- **nlocksdown** (*int*) – modification in number of down locks
- **nlocksup** (*int*) – modification in number of up locks

addVarSOS1(*cons*, *var*, *weight*)

Add variable to SOS1 constraint.

Parameters

- **cons** (*Constraint*) – SOS1 constraint
- **var** (*Variable*) – new variable
- **weight** (*weight*) – weight of new variable

addVarSOS2(*cons*, *var*, *weight*)

Add variable to SOS2 constraint.

Parameters

- **cons** (*Constraint*) – SOS2 constraint
- **var** (*Variable*) – new variable
- **weight** (*weight*) – weight of new variable

addVarToRow(*row*, *var*, *value*)

Resolves variable to columns and adds them with the coefficient to the row.

Parameters

- **row** (*Row*) – Row in which the variable will be added
- **var** (*Variable*) – Variable which will be added to the row
- **value** (*float*) – Coefficient on the variable when placed in the row

allColsInLP()

Checks if all columns, i.e. every variable with non-empty column is present in the LP. This is not True when performing pricing for instance.

Return type

bool

allowNegSlackExact()

Returns whether negative slack is allowed in exact solving mode.

Return type

bool

appendVarSOS1(*cons*, *var*)

Append variable to SOS1 constraint.

Parameters

- **cons** (*Constraint*) – SOS1 constraint

- **var** (*Variable*) – variable to append

appendVarSOS2(*cons*, *var*)

Append variable to SOS2 constraint.

Parameters

- **cons** (*Constraint*) – SOS2 constraint
- **var** (*Variable*) – variable to append

applyCutsProbing()

Applies the cuts in the separation storage to the LP and clears the storage afterwards; this method can only be applied during probing; the user should resolve the probing LP afterwards in order to get a new solution. returns:

Returns

cutoff – whether an empty domain was created

Return type

bool

attachEventHandlerCallback(*callback*, *events*, *name*='eventhandler', *description*='')

Attach an event handler to the model using a callback function.

Parameters

- **callback** (*callable*) – The callback function to be called when an event occurs. The callback function should have the following signature: `callback(model, event)`
- **events** (*list of SCIP_EVENTTYPE*) – List of event types to attach the event handler to.
- **name** (*str, optional*) – Name of the event handler. If not provided, a unique default name will be generated.
- **description** (*str, optional*) – Description of the event handler. If not provided, an empty string will be used.

backtrackProbing(*probingdepth*)

Undoes all changes to the problem applied in probing up to the given probing depth.

Parameters

probingdepth (*int*) – probing depth of the node in the probing path that should be reactivated

branchLPExact()

Performs exact LP branching.

Return type

SCIP_RESULT

branchVar(*variable*)

Branch on a non-continuous variable.

Parameters

variable (*Variable*) – Variable to branch on

Returns

- *Node* – Node created for the down (left) branch
- *Node or None* – Node created for the equal child (middle child). Only exists if branch variable is integer

- *Node* – Node created for the up (right) branch

branchVarVal(*variable*, *value*)

Branches on variable using a value which separates the domain of the variable.

Parameters

- **variable** (*Variable*) – Variable to branch on
- **value** (*float*) – value to branch on

Returns

- *Node* – Node created for the down (left) branch
- *Node or None* – Node created for the equal child (middle child). Only exists if the branch variable is integer
- *Node* – Node created for the up (right) branch

cacheRowExtensions(*row*)

Informs row that all subsequent additions of variables to the row should be cached and not directly applied; after all additions were applied, `flushRowExtensions()` must be called; while the caching of row extensions is activated, information methods of the row give invalid results; caching should be used, if a row is build with `addVarToRow()` calls variable by variable to increase the performance.

Parameters

row (*Row*)

calcChildEstimate(*variable*, *targetvalue*)

Calculates an estimate for the objective of the best feasible solution contained in the subtree after applying the given branching; this estimate can be given to the `SCIPcreateChild()` call.

Parameters

- **variable** (*Variable*) – Variable to compute the estimate for
- **targetvalue** (*float*) – new value of the variable in the child node

Returns

objective estimate of the best solution in the subtree after applying the given branching

Return type

float

calcNodeselPriority(*variable*, *branchdir*, *targetvalue*)

Calculates the node selection priority for moving the given variable's LP value to the given target value; this node selection priority can be given to the `SCIPcreateChild()` call.

Parameters

- **variable** (*Variable*) – variable on which the branching is applied
- **branchdir** (*PY SCIP BRANCHDIR*) – type of branching that was performed
- **targetvalue** (*float*) – new value of the variable in the child node

Returns

node selection priority for moving the given variable's LP value to the given target value

Return type

int

catchEvent(*eventtype*, *eventhdlr*)

Catches a global (not variable or row dependent) event.

Parameters

- **eventtype** (*PY SCIP EVENTTYPE*)
- **eventhdlr** (*Eventhdlr*)

catchRowEvent(*row*, *eventtype*, *eventhdlr*)

Catches a row coefficient, constant, or side change event on the given row.

Parameters

- **row** (*Row*)
- **eventtype** (*PY SCIP EVENTTYPE*)
- **eventhdlr** (*Eventhdlr*)

catchVarEvent(*var*, *eventtype*, *eventhdlr*)

Catches an objective value or domain change event on the given transformed variable.

Parameters

- **var** (*Variable*)
- **eventtype** (*PY SCIP EVENTTYPE*)
- **eventhdlr** (*Eventhdlr*)

checkBendersSubproblemOptimality(*solution*, *probnumber*, *benders=None*)

Returns whether the subproblem is optimal w.r.t the master problem auxiliary variables.

Parameters

- **solution** (*Solution*) – the master problem solution that is being checked for optimamlity
- **probnumber** (*int*) – the problem number for which optimality is being checked
- **benders** (*Benders or None, optional*) – the Benders’ decomposition to which the subproblem belongs to

Returns

optimal – flag to indicate whether the current subproblem is optimal for the master

Return type

bool

checkQuadraticNonlinear(*cons*)

Returns if the given constraint is quadratic.

Parameters

- cons** (*Constraint*)

Return type

bool

checkSol(*solution*, *printreason=True*, *completely=False*, *checkbounds=True*, *checkintegrality=True*, *checklprows=True*, *original=False*)

Check given primal solution for feasibility without adding it to the storage.

Parameters

- **solution** (*Solution*) – solution to store

- **printreason** (*bool*, *optional*) – should all reasons of violations be printed? (Default value = True)
- **completely** (*bool*, *optional*) – should all violation be checked? (Default value = False)
- **checkbounds** (*bool*, *optional*) – should the bounds of the variables be checked? (Default value = True)
- **checkintegrality** (*bool*, *optional*) – has integrality to be checked? (Default value = True)
- **checklprows** (*bool*, *optional*) – have current LP rows (both local and global) to be checked? (Default value = True)
- **original** (*bool*, *optional*) – must the solution be checked against the original problem (Default value = False)

Returns

feasible – whether the given solution was feasible or not

Return type

bool

chgCapacityKnapsack(*cons*, *capacity*)

Change capacity of a knapsack constraint.

Parameters

- **cons** ([Constraint](#)) – knapsack constraint to change the capacity from
- **capacity** (*float* or *None*) – new capacity (set to None for +infinity)

chgCoefLinear(*cons*, *var*, *value*)

Changes coefficient of variable in linear constraint; deletes the variable if coefficient is zero; adds variable if not yet contained in the constraint This method may only be called during problem creation stage for an original constraint and variable. This method requires linear time to search for occurrences of the variable in the constraint data.

Parameters

- **cons** ([Constraint](#)) – linear constraint
- **var** ([Variable](#)) – variable of constraint entry
- **value** (*float*) – new coefficient of constraint entry

chgLhs(*cons*, *lhs*)

Change left-hand side value of a constraint.

Parameters

- **cons** ([Constraint](#)) – constraint to change the left-hand side from
- **lhs** (*float* or *None*) – new left-hand side (set to None for -infinity)

chgReoptObjective(*coeffs*, *sense*='minimize')

Establish the objective function as a linear expression.

Parameters

- **coeffs** (*list of float*) – the coefficients
- **sense** (*str*) – the objective sense (Default value = 'minimize')

chgRhs(*cons, rhs*)

Change right-hand side value of a constraint.

Parameters

- **cons** ([Constraint](#)) – constraint to change the right-hand side from
- **rhs** (*float or None*) – new right-hand side (set to None for +infinity)

chgRowLhsDive(*row, newlhs*)

Changes row lhs in current dive, change will be undone after diving ends, for permanent changes use SCIPchgRowLhs().

Parameters

- **row** ([Row](#))
- **newlhs** (*float*)

chgRowRhsDive(*row, newrhs*)

Changes row rhs in current dive, change will be undone after diving ends. For permanent changes use SCIPchgRowRhs().

Parameters

- **row** ([Row](#))
- **newrhs** (*float*)

chgVarBranchPriority(*var, priority*)

Sets the branch priority of the variable. Variables with higher branch priority are always preferred to variables with lower priority in selection of branching variable.

Parameters

- **var** ([Variable](#)) – variable to change priority of
- **priority** (*int*) – the new priority of the variable (the default branching priority is 0)

chgVarLb(*var, lb*)

Changes the lower bound of the specified variable.

Parameters

- **var** ([Variable](#)) – variable to change bound of
- **lb** (*float or None*) – new lower bound (set to None for -infinity)

chgVarLbDive(*var, newbound*)

Changes variable's current lb in current dive.

Parameters

- **var** ([Variable](#))
- **newbound** (*float*)

chgVarLbGlobal(*var, lb*)

Changes the global lower bound of the specified variable.

Parameters

- **var** ([Variable](#)) – variable to change bound of
- **lb** (*float or None*) – new lower bound (set to None for -infinity)

chgVarLbNode(*node*, *var*, *lb*)

Changes the lower bound of the specified variable at the given node.

Parameters

- **node** (**Node**) – Node at which the variable bound will be changed
- **var** (**Variable**) – variable to change bound of
- **lb** (*float* or *None*) – new lower bound (set to *None* for -infinity)

chgVarLbProbing(*var*, *lb*)

Changes the variable lower bound during probing mode.

Parameters

- **var** (**Variable**) – variable to change bound of
- **lb** (*float* or *None*) – new lower bound (set to *None* for -infinity)

chgVarObjDive(*var*, *newobj*)

Changes (column) variable's objective value in current dive.

Parameters

- **var** (**Variable**)
- **newobj** (*float*)

chgVarObjProbing(*var*, *newobj*)

Changes (column) variable's objective value during probing mode.

chgVarType(*var*, *vtype*)

Changes the type of a variable.

Parameters

- **var** (**Variable**) – variable to change type of
- **vtype** (*str*) – new variable type. 'C' or "CONTINUOUS", 'I' or "INTEGER", 'B' or "BINARY", and 'M' "IMPLINT".

chgVarUb(*var*, *ub*)

Changes the upper bound of the specified variable.

Parameters

- **var** (**Variable**) – variable to change bound of
- **lb** (*float* or *None*) – new upper bound (set to *None* for +infinity)

chgVarUbDive(*var*, *newbound*)

Changes variable's current ub in current dive.

Parameters

- **var** (**Variable**)
- **newbound** (*float*)

chgVarUbGlobal(*var*, *ub*)

Changes the global upper bound of the specified variable.

Parameters

- **var** (**Variable**) – variable to change bound of

- **lb** (*float or None*) – new upper bound (set to None for +infinity)

chgVarUbNode(*node, var, ub*)

Changes the upper bound of the specified variable at the given node.

Parameters

- **node** (*Node*) – Node at which the variable bound will be changed
- **var** (*Variable*) – variable to change bound of
- **lb** (*float or None*) – new upper bound (set to None for +infinity)

chgVarUbProbing(*var, ub*)

Changes the variable upper bound during probing mode.

Parameters

- **var** (*Variable*) – variable to change bound of
- **ub** (*float or None*) – new upper bound (set to None for +infinity)

computeBestSolSubproblems()

Solves the subproblems with the best solution to the master problem. Afterwards, the best solution from each subproblem can be queried to get the solution to the original problem. If the user wants to resolve the subproblems, they must free them by calling `freeBendersSubproblems()`

constructLP()

Makes sure that the LP of the current node is loaded and may be accessed through the LP information methods.

Returns

cutoff – Can the node be cutoff?

Return type

bool

copyLargeNeighborhoodSearch(*to_fix, fix_vals*) → *Model*

Creates a configured copy of the transformed problem and applies provided fixings intended for LNS heuristics.

Parameters

- **to_fix** (*List[Variable]*) – A list of variables to fix in the copy
- **fix_vals** (*List[Real]*) – A list of the values to which to fix the variables in the copy (care their order)

Returns

model – A model containing the created copy

Return type

Model

count()

Counts the number of feasible points of problem.

createChild(*nodeselprio, estimate*)

Create a child node of the focus node.

Parameters

- **nodeselprio** (*int*) – node selection priority of new node

- **estimate** (*float*) – estimate for (transformed) objective value of best feasible solution in subtree

Returns

the child which was created

Return type

Node

createCons(*conshdlr*, *name*, *initial=True*, *separate=True*, *enforce=True*, *check=True*, *propagate=True*, *local=False*, *modifiable=False*, *dynamic=False*, *removable=False*, *stickingatnode=False*)

Create a constraint of a custom constraint handler.

Parameters

- **conshdlr** (*Conshdlr*) – constraint handler
- **name** (*str*) – name of constraint handler
- **initial** (*bool*, *optional*) – (Default value = True)
- **separate** (*bool*, *optional*) – (Default value = True)
- **enforce** (*bool*, *optional*) – (Default value = True)
- **check** (*bool*, *optional*) – (Default value = True)
- **propagate** (*bool*, *optional*) – (Default value = True)
- **local** (*bool*, *optional*) – (Default value = False)
- **modifiable** (*bool*, *optional*) – (Default value = False)
- **dynamic** (*bool*, *optional*) – (Default value = False)
- **removable** (*bool*, *optional*) – (Default value = False)
- **stickingatnode** (*bool*, *optional*) – (Default value = False)

Return type

Constraint

createConsFromExpr(*cons*, *name=""*, *initial=True*, *separate=True*, *enforce=True*, *check=True*, *propagate=True*, *local=False*, *modifiable=False*, *dynamic=False*, *removable=False*, *stickingatnode=False*)

Create a linear or nonlinear constraint without adding it to the SCIP problem. This is useful for creating disjunction constraints without also enforcing the individual constituents. Currently, this can only be used as an argument to *.addConsElemDisjunction*. To add an individual linear/nonlinear constraint, prefer *.addCons()*.

Parameters

- **cons** (*ExprCons*) – The expression constraint that is not yet an actual constraint
- **name** (*str*, *optional*) – the name of the constraint, generic name if empty (Default value = '')
- **initial** (*bool*, *optional*) – should the LP relaxation of constraint be in the initial LP? (Default value = True)
- **separate** (*bool*, *optional*) – should the constraint be separated during LP processing? (Default value = True)
- **enforce** (*bool*, *optional*) – should the constraint be enforced during node processing? (Default value = True)

- **check** (*bool, optional*) – should the constraint be checked for feasibility? (Default value = True)
- **propagate** (*bool, optional*) – should the constraint be propagated during node processing? (Default value = True)
- **local** (*bool, optional*) – is the constraint only valid locally? (Default value = False)
- **modifiable** (*bool, optional*) – is the constraint modifiable (subject to column generation)? (Default value = False)
- **dynamic** (*bool, optional*) – is the constraint subject to aging? (Default value = False)
- **removable** (*bool, optional*) – should the relaxation be removed from the LP due to aging or cleanup? (Default value = False)
- **stickingatnode** (*bool, optional*) – should the constraint always be kept at the node where it was added, even if it may be moved to a more global node? (Default value = False)

Returns

The created Constraint object.

Return type

Constraint

createEmptyRowSepa(*sepa, name='row', lhs=0.0, rhs=None, local=True, modifiable=False, removable=True*)

Creates and captures an LP row without any coefficients from a separator.

Parameters

- **sepa** (*Sepa*) – separator that creates the row
- **name** (*str, optional*) – name of row (Default value = “row”)
- **lhs** (*float or None, optional*) – left hand side of row (Default value = 0)
- **rhs** (*float or None, optional*) – right hand side of row (Default value = None)
- **local** (*bool, optional*) – is row only valid locally? (Default value = True)
- **modifiable** (*bool, optional*) – is row modifiable during node processing (subject to column generation)? (Default value = False)
- **removable** (*bool, optional*) – should the row be removed from the LP due to aging or cleanup? (Default value = True)

Return type

Row

createEmptyRowUnspec(*name='row', lhs=0.0, rhs=None, local=True, modifiable=False, removable=True*)

Creates and captures an LP row without any coefficients from an unspecified source.

Parameters

- **name** (*str, optional*) – name of row (Default value = “row”)
- **lhs** (*float or None, optional*) – left hand side of row (Default value = 0)
- **rhs** (*float or None, optional*) – right hand side of row (Default value = None)
- **local** (*bool, optional*) – is row only valid locally? (Default value = True)
- **modifiable** (*bool, optional*) – is row modifiable during node processing (subject to column generation)? (Default value = False)

- **removable** (*bool, optional*) – should the row be removed from the LP due to aging or cleanup? (Default value = True)

Return type*Row***createOrigSol**(*heur=None*)

Create a new primal solution in the original space.

Parameters

heur (*Heur or None, optional*) – heuristic that found the solution (Default value = None)

Return type

Solution

createPartialSol(*heur=None*)

Create a partial primal solution, initialized to unknown values.

Parameters

heur (*Heur or None, optional*) – heuristic that found the solution (Default value = None)

Return type

Solution

createProbBasic(*problemName='model'*)

Create new problem instance with given name.

Parameters

problemName (*str, optional*) – name of model or problem (Default value = 'model')

createSol(*heur=None, initlp=False*)

Create a new primal solution in the transformed space.

Parameters

- **heur** (*Heur or None, optional*) – heuristic that found the solution (Default value = None)
- **initlp** (*bool, optional*) – Should the created solution be initialised to the current LP solution instead of all zeros

Return type

Solution

cutoffNode(*node*)

marks node and whole subtree to be cut off from the branch and bound tree.

Parameters

node (*Node*)

deactivatePricer(*pricer*)

Deactivate the given pricer.

Parameters

pricer (*Pricer*) – the pricer to deactivate

delCoefLinear(*cons, var*)

Deletes variable from linear constraint This method may only be called during problem creation stage for an original constraint and variable. This method requires linear time to search for occurrences of the variable in the constraint data.

Parameters

- **cons** ([Constraint](#)) – linear constraint
- **var** ([Variable](#)) – variable of constraint entry

delCons(*cons*)

Delete constraint from the model

Parameters

- **cons** ([Constraint](#)) – constraint to be deleted

delConsLocal(*cons*)

Delete constraint from the current node and its children.

Parameters

- **cons** ([Constraint](#)) – constraint to be deleted

delVar(*var*)

Delete a variable.

Parameters

- **var** ([Variable](#)) – the variable which shall be deleted

Returns**deleted** – Whether deleting was successful**Return type**

bool

disableDebugSol()

Disables the debug solution mechanism.

disablePropagation(*onlyroot=False*)

Disables propagation in SCIP to avoid modifying the original problem during transformation.

Parameters

- **onlyroot** (*bool*, *optional*) – use propagation when root processing is finished (Default value = False)

dropEvent(*eventtype*, *eventhdlr*)

Drops a global event (stops tracking the event).

Parameters

- **eventtype** (*PY SCIP EVENTTYPE*)
- **eventhdlr** (*Eventhdlr*)

dropRowEvent(*row*, *eventtype*, *eventhdlr*)

Drops a row coefficient, constant, or side change event (stops tracking the event) on the given row.

Parameters

- **row** ([Row](#))
- **eventtype** (*PY SCIP EVENTTYPE*)
- **eventhdlr** (*Eventhdlr*)

dropVarEvent(*var*, *eventtype*, *eventhdlr*)

Drops an objective value or domain change event (stops tracking the event) on the given transformed variable.

Parameters

- **var** (*Variable*)
- **eventtype** (*PY SCIP EVENTTYPE*)
- **eventhdlr** (*Eventhdlr*)

enableDebugSol()

Enables the debug solution mechanism, which allows tracing back the invalidation of a debug solution during the solution process of SCIP. It must be explicitly enabled for the SCIP data structure.

enableExactSolving(enable)

Enables or disables exact solving mode in SCIP.

Parameters

enable (*SCIP_Bool*) – Whether to enable exact solving mode (True) or disable it (False).

enableReoptimization(enable=True)

Include specific heuristics and branching rules for reoptimization.

Parameters

enable (*bool*, *optional*) – True to enable and False to disable

endDive()

Quits probing and resets bounds and constraints to the focus node's environment.

endProbing()

Quits probing and resets bounds and constraints to the focus node's environment.

endStrongbranch()

End strong branching. Needs to be called if startStrongBranching was called previously. Between these calls the user can access all strong branching functionality.

epsilon()

Retrieve epsilon for e.g. equality checks.

Return type

float

feasCeil(value)

Rounds value - feasibility tolerance up to the next integer.

Parameters

value (*float*)

Return type

float

feasFloor(value)

Rounds value + feasibility tolerance down to the next integer.

Parameters

value (*float*)

Return type

float

feasFrac(value)

Returns fractional part of value, i.e. $x - \text{floor}(x)$ in feasible tolerance: $x - \text{floor}(x + \text{feastol})$.

Parameters**value** (*float*)**Return type***float***feasRound**(*value*)

Rounds value to the nearest integer in feasibility tolerance.

Parameters**value** (*float*)**Return type***float***feastol**()

Retrieve feasibility tolerance.

Return type*float***fixVar**(*var*, *val*)Fixes the variable *var* to the value *val* if possible.**Parameters**

- **var** (*Variable*) – variable to fix
- **val** (*float*) – the fix value

Returns

- **infeasible** (*bool*) – Is the fixing infeasible?
- **fixed** (*bool*) – Was the fixing performed?

fixVarProbing(*var*, *fixedval*)

Fixes a variable at the current probing node.

Parameters

- **var** (*Variable*)
- **fixedval** (*float*)

flushRowExtensions(*row*)Flushes all cached row extensions after a call of `cacheRowExtensions()` and merges coefficients with equal columns into a single coefficient**Parameters****row** (*Row*)**frac**(*value*)Returns fractional part of value, i.e. $x - \text{floor}(x)$ in epsilon tolerance: $x - \text{floor}(x + \text{eps})$.**Parameters****value** (*float*)**Return type***float*

freeBendersSubproblems()

Deprecated: This method is no longer needed.

Benders subproblems are now automatically managed and freed by the master Model when it is deallocated. Calling this method explicitly has no effect.

Deprecated since version 5.7.0: This method is deprecated and will be removed in a future version. Sub-problem memory management is now handled automatically.

freeProb()

Frees problem and solution process data.

freeReoptSolve()

Frees all solution process data and prepares for reoptimization.

freeSol(solution)

Free given solution

Parameters

solution (*Solution*) – solution to be freed

freeTransform()

Frees all solution process data including presolving and transformed problem, only original problem is kept.

static from_ptr(capsule, take_ownership)

Create a Model from a given pointer.

Parameters

- **capsule** – The PyCapsule containing the SCIP pointer under the name “scip”
- **take_ownership** (*bool*) – Whether the newly created Model assumes ownership of the underlying Scip pointer (see `_freescip`)

Return type

Model

generateIIS()

Generates an Irreducible Infeasible Subsystem (IIS) from the current problem.

Return type

IIS

getActivity(cons, sol=None)

Retrieve activity of given constraint. Can only be called after solving is completed.

Parameters

- **cons** (*Constraint*) – linear constraint
- **sol** (*Solution or None, optional*) – solution to compute activity of, None to use current node’s solution (Default value = None)

Return type

float

getBendersAuxiliaryVar(probnumber, benders=None)

Returns the auxiliary variable that is associated with the input problem number

Parameters

- **probnnumber** (*int*) – the problem number for which the target variable belongs, -1 for master problem
- **benders** (*Benders or None, optional*) – the Benders' decomposition to which the subproblem variables belong to

Return type*Variable***getBendersSubproblem**(*probnnumber, benders=None*)

Returns a Model object that wraps around the SCIP instance of the subproblem. NOTE: This Model object is just a place holder and SCIP instance will not be freed when the object is destroyed.

Parameters

- **probnnumber** (*int*) – the problem number for subproblem that is required
- **benders** (*Benders or None, optional*) – the Benders' decomposition object that the subproblem belongs to (Default = None)

Return type*Model***getBendersVar**(*var, benders=None, probnnumber=-1*)

Returns the variable for the subproblem or master problem depending on the input probnnumber.

Parameters

- **var** (*Variable*) – the source variable for which the target variable is requested
- **benders** (*Benders or None, optional*) – the Benders' decomposition to which the subproblem variables belong to
- **probnnumber** (*int, optional*) – the problem number for which the target variable belongs, -1 for master problem

Return type*Variable* or None**getBestChild()**

Gets the best child of the focus node w.r.t. the node selection strategy.

Return type*Node***getBestLeaf()**

Gets the best leaf from the node queue w.r.t. the node selection strategy.

Return type*Node***getBestNode()**

Gets the best node from the tree (child, sibling, or leaf) w.r.t. the node selection strategy.

Return type*Node***getBestSibling()**

Gets the best sibling of the focus node w.r.t. the node selection strategy.

Return type*Node*

getBestSol()

Retrieve currently best known feasible primal solution.

Return type

Solution or None

getBestboundNode()

Gets the node with smallest lower bound from the tree (child, sibling, or leaf).

Return type

Node

getBipartiteGraphRepresentation(*prev_col_features=None, prev_edge_features=None, prev_row_features=None, static_only=False, suppress_warnings=False*)

This function generates the bipartite graph representation of an LP, which was first used in the following paper: @inproceedings{conf/nips/GasseCFCL19, title={Exact Combinatorial Optimization with Graph Convolutional Neural Networks}, author={Gasse, Maxime and Chételat, Didier and Ferroni, Nicola and Charlin, Laurent and Lodi, Andrea}, booktitle={Advances in Neural Information Processing Systems 32}, year={2019} } The exact features have been modified compared to the original implementation. This function is used mainly in the machine learning community for MIP. A user can only call it during the solving process, when there is an LP object. This means calling it from some user defined plugin on the Python side. An example plugin is a branching rule or an event handler, which is exclusively created to call this function. The user must then make certain to return the appropriate SCIP_RESULT (e.g. DIDNOTRUN)

Parameters

- **prev_col_features** (*list of list or None, optional*) – The list of column features previously returned by this function
- **prev_edge_features** (*list of list or None, optional*) – The list of edge features previously returned by this function
- **prev_row_features** (*list of list or None, optional*) – The list of row features previously returned by this function
- **static_only** (*bool, optional*) – Whether exclusively static features should be generated
- **suppress_warnings** (*bool, optional*) – Whether warnings should be suppressed

Returns

- **col_features** (*list of list*)
- **edge_features** (*list of list*)
- **row_features** (*list of list*)
- *dict* – The feature mappings for the columns, edges, and rows

getBranchScoreMultiple(*var, gains*)

Calculates the branching score out of the gain predictions for a branching with arbitrarily many children.

Parameters

- **var** (*Variable*) – variable to calculate the score for
- **gains** (*list of float*) – list of gains for each child.

Return type

float

getCapacityKnapsack(*cons*)

Retrieve capacity of a knapsack constraint.

Parameters

cons ([Constraint](#)) – knapsack constraint to get the capacity from

Return type

float

getChildren()

Gets the children of the focus node.

Return type

list of Nodes

getColRedCost(*col*)

Gets the reduced cost of the column in the current LP.

Parameters

col ([Column](#))

Return type

float

getCondition(*exact=False*)

Get the current LP's condition number.

Parameters

exact (*bool*, *optional*) – whether to get an estimate or the exact value (Default value = False)

Return type

float

getConsNVars(*constraint*)

Gets number of variables in a constraint.

Parameters

constraint ([Constraint](#)) – Constraint to get the number of variables from.

Return type

int

Raises

TypeError – If the associated constraint handler does not have this functionality

getConsVals(*constraint*)

Returns the value array of an arbitrary SCIP constraint that can be represented as a single linear constraint.

Parameters

constraint ([Constraint](#)) – Constraint to get the values from.

Return type

list of float

getConsVars(*constraint*)

Gets variables in a constraint.

Parameters

constraint ([Constraint](#)) – Constraint to get the variables from.

Return typelist of *Variable***getConss**(*transformed=True*)

Retrieve all constraints.

Parameters**transformed** (*bool*, *optional*) – get transformed variables instead of original (Default value = True)**Return type**list of *Constraint***getCurrentNode**()

Retrieve current node.

Return type*Node***getCutEfficacy**(*cut*, *sol=None*)Returns efficacy of the cut with respect to the given primal solution or the current LP solution: $e = -$ feasibility/norm**Parameters**

- **cut** (*Row*)
- **sol** (*Solution or None*, *optional*)

Return type

float

getCutLPsolCutoffDistance(*cut*, *sol*)

Returns row's cutoff distance in the direction of the given primal solution.

Parameters

- **cut** (*Row*)
- **sol** (*Solution*)

Return type

float

getCutoffbound()

Gets the cutoff bound of the transformed problem.

Return type

float

getDepth()

Retrieve the depth of the current node.

Return type

int

getDualMultiplier(*cons*)

DEPRECATED: Retrieve the dual solution to a linear constraint.

Parameters**cons** (*Constraint*) – linear constraint

Return type

float

getDualSolVal(*cons*, *boundconstraint=False*)

Returns dual solution value of a constraint.

Parameters

- **cons** (*Constraint*) – constraint to get the dual solution value of
- **boundconstraint** (*bool*, *optional*) – Decides whether to store a bool if the constraint is a bound constraint (default = False)

Return type

float

getDualbound()

Retrieve the best dual bound.

Return type

float

getDualboundRoot()

Retrieve the best root dual bound.

Return type

float

getDualfarkasKnapsack(*cons*)

Retrieve the dual farkas value to a knapsack constraint.

Parameters**cons** (*Constraint*) – knapsack constraint**Return type**

float

getDualfarkasLinear(*cons*)

Retrieve the dual farkas value to a linear constraint.

Parameters**cons** (*Constraint*) – linear constraint**Return type**

float

getDualsolKnapsack(*cons*)

Retrieve the dual solution to a knapsack constraint.

Parameters**cons** (*Constraint*) – knapsack constraint**Return type**

float

getDualsolLinear(*cons*)

Retrieve the dual solution to a linear constraint.

Parameters**cons** (*Constraint*) – linear constraint**Return type**

float

getGap()

Retrieve the gap, i.e. $\text{abs}((\text{primalbound} - \text{dualbound})/\min(\text{abs}(\text{primalbound}), \text{abs}(\text{dualbound})))$

Return type

float

getHeurTiming(*heurname*)

Get the timing of a heuristic

Parameters

heurname (*string*, name of the heuristic)

Returns

positions in the node solving loop where heuristic should be executed

Return type

PY SCIP HEURTIMING

getIIS()

Get the IIS object. Note: Needs to be called after generateIIS, or after a single execution of the iisfinderExec.

Return type

IIS

getLPBInvARow(*row*)

Gets a row from $B^{-1} * A$.

Parameters

row (*int*) – The row index of the inverse basis matrix multiplied by the coefficient matrix

Return type

list of float

getLPBInvRow(*row*)

Gets a row from the inverse basis matrix B^{-1}

Parameters

row (*int*) – The row index of the inverse basis matrix

Return type

list of float

getLPBasisInd()

Gets all indices of basic columns and rows: index $i \geq 0$ corresponds to column i , index $i < 0$ to row $-i-1$

Return type

list of int

getLPBranchCands()

Gets branching candidates for LP solution branching (fractional variables) along with solution values, fractionalities, and number of branching candidates; The number of branching candidates does NOT account for fractional implicit integer variables which should not be used for branching decisions. Fractional implicit integer variables are stored at the positions nlpccands to $\text{nlpccands} + \text{nfracimplvars} - 1$ branching rules should always select the branching candidate among the first nprlpccands of the candidate list

Returns

- *list of Variable* – list of variables of LP branching candidates
- *list of float* – list of LP candidate solution values
- *list of float* – list of LP candidate fractionalities

- *int* – number of LP branching candidates
- *int* – number of candidates with maximal priority
- *int* – number of fractional implicit integer variables

getLPColsData()

Retrieve current LP columns.

Return type

list of *Column*

getLPObjVal()

Gets objective value of current LP (which is the sum of column and loose objective value).

Return type

float

getLPRowsData()

Retrieve current LP rows.

Return type

list of *Row*

getLPSolstat()

Gets solution status of current LP.

Return type

SCIP_LPSOLSTAT

getLeaves()

Gets the leaves of the tree along with number of leaves.

Return type

list of Nodes

getLhs(*cons*)

Retrieve left-hand side value of a constraint.

Parameters

cons (*Constraint*) – linear or nonlinear constraint

Return type

float

getLinearConsIndicator(*cons*)

Get the linear constraint corresponding to the indicator constraint.

Parameters

cons (*Constraint*) – The indicator constraint

Return type

Constraint or None

getLocalEstimate(*original=False*)

Gets estimate of best primal solution w.r.t. original or transformed problem contained in current subtree.

Parameters

original (*bool*, *optional*) – get estimate of original or transformed problem (Default value = False)

Return type

float

getLowerbound()

Gets global lower (dual) bound of the transformed problem.

Return type

float

getMajorVersion()

Retrieve SCIP major version.

Return type

int

getMaxDepth()

Gets maximal depth of the branch-and-bound tree processed during solving (excluding probing nodes).

Return type

int

getMinorVersion()

Retrieve SCIP minor version.

Return type

int

getNBestSolsFound()

Gets number of feasible primal solutions found so far, that improved the primal bound at the time they were found.

Return type

int

getNBinVars()

Gets number of binary active problem variables.

Return type

int

getNChildren()

Gets number of children of focus node.

Return type

int

getNConss(*transformed=True*)

Retrieve number of all constraints.

Parameters

transformed (*bool*, *optional*) – get number of transformed variables instead of original
(Default value = True)

Return type

int

getNContVars()

Gets number of continuous active problem variables.

Return type

int

getNCountedSols()

Get number of feasible solution.

Return type

int

getNCuts()

Retrieve total number of cuts in storage.

Return type

int

getNCutsApplied()

Retrieve number of currently applied cuts.

Return type

int

getNFeasibleLeaves()

Retrieve number of leaf nodes processed with feasible relaxation solution.

Return type

int

getNImplVars()

Gets number of implicit integer active problem variables.

Return type

int

getNInfeasibleLeaves()

Gets number of infeasible leaf nodes processed.

Return type

int

getNIntVars()

Gets number of integer active problem variables.

Return type

int

getNLPBranchCands()

Gets number of branching candidates for LP solution branching (number of fractional variables)

Returns

number of LP branching candidates

Return type

int

getNLPCols()

Retrieve the number of columns currently in the LP.

Return type

int

getNLPIterations()

Returns the total number of LP iterations so far.

Return type

int

getNLPROws()

Retrieve the number of rows currently in the LP.

Return type
int

getNLPs()

Gets total number of LPs solved so far.

Return type
int

getNLeaves()

Gets number of leaves in the tree.

Return type
int

getNLimSolsFound()

Gets number of feasible primal solutions respecting the objective limit found so far.

Return type
int

getNNlRows()

Gets current number of nonlinear rows in SCIP's internal NLP.

Return type
int

getNNodeLPIterations()

Gets number of LP iterations used for solving node relaxations so far.

Return type
int

getNNodes()

Gets number of processed nodes in current run, including the focus node.

Return type
int

getNReaders()

Get number of currently available readers.

Return type
int

getNSepaRounds()

Retrieve the number of separation rounds that have been performed at the current node.

Return type
int

getNSiblings()

Gets number of siblings of focus node.

Return type
int

getNSols()

Gets number of feasible primal solutions stored in the solution storage in case the problem is transformed; in case the problem stage is SCIP_STAGE_PROBLEM, the number of solution in the original solution candidate storage is returned.

Return type
int

getNSolsFound()

Gets number of feasible primal solutions found so far.

Return type
int

getNStrongbranchLPiterations()

Gets number of LP iterations used for strong branching so far.

Return type
int

getNTotalNodes()

Gets number of processed nodes in all runs, including the focus node.

Return type
int

getNVars(*transformed=True*)

Retrieve number of variables in the problems.

Parameters
transformed (*bool*, *optional*) – Get transformed variables instead of original (Default value = True)

Return type
int

getNVarsAnd(*and_cons*)

Gets number of variables in and constraint.

Parameters
and_cons ([Constraint](#)) – AND constraint to get the number of variables from.

Return type
int

getNlRowActivityBounds(*nlrow*)

Gives the minimal and maximal activity of a nonlinear row w.r.t. the variable's bounds.

Parameters
nlrow ([NLRow](#))

Return type
tuple of float

getNlRowSolActivity(*nlrow*, *sol=None*)

Gives the activity of a nonlinear row for a given primal solution.

Parameters
• **nlrow** ([NLRow](#))

- **sol** (*Solution or None, optional*) – a primal solution, if None, then the current LP solution is used

Return type

float

getNlRowSolFeasibility(*nlrow, sol=None*)

Gives the feasibility of a nonlinear row for a given primal solution

Parameters

- **nlrow** (*NLRow*)
- **sol** (*Solution or None, optional*) – a primal solution, if None, then the current LP solution is used

Return type

bool

getNlRows()

Returns a list with the nonlinear rows in SCIP's internal NLP.

Return type

list of NLRow

getObjVal(*original=True*)

Retrieve the objective value of the best solution.

Parameters

original (*bool, optional*) – objective value in original space (Default value = True)

Return type

float

getObjective()

Retrieve objective function as Expr.

Return type

Expr

getObjectiveSense()

Retrieve objective sense.

Return type

str

getObjlimit()

Returns current limit on objective function.

Return type

float

getObjoffset(*original=True*)

Retrieve constant objective offset

Parameters

original (*bool, optional*) – offset of original or transformed problem (Default value = True)

Return type

float

getOpenNodes()

Access to all data of open nodes (leaves, children, and siblings).

Returns

- **leaves** (*list of Node*) – list of all open leaf nodes
- **children** (*list of Node*) – list of all open children nodes
- **siblings** (*list of Node*) – list of all open sibling nodes

getParam(name)

Get the value of a parameter of type int, bool, real, long, char or str.

Parameters

name (*str*) – name of parameter

Return type

object

getParams()

Gets the values of all parameters as a dict mapping parameter names to their values.

Returns

dict mapping parameter names to their values.

Return type

dict of str to object

getPlungeDepth()

Gets current plunging depth (successive selections of child/sibling nodes).

Return type

int

getPresolvingTime()

Returns the current presolving time in seconds.

Return type

float

getPrimalRay()

Gets primal ray causing unboundedness of the LP relaxation.

Return type

list of float

getPrimalRayVal(var)

Gets value of given variable in primal ray causing unboundedness of the LP relaxation.

Parameters

var (*Variable*)

Return type

float

getPrimalbound()

Retrieve the best primal bound.

Return type

float

getPrioChild()

Gets the best child of the focus node w.r.t. the node selection priority assigned by the branching rule.

Return type

Node

getPrioSibling()

Gets the best sibling of the focus node w.r.t. the node selection priority assigned by the branching rule.

Return type

Node

getProbName()

Retrieve problem name.

Return type

str

getProbingDepth()

Returns the current probing depth.

getPseudoBranchCands()

Gets branching candidates for pseudo solution branching (non-fixed variables) along with the number of candidates.

Returns

- *list of Variable* – list of variables of pseudo branching candidates
- *int* – number of pseudo branching candidates
- *int* – number of candidates with maximal priority

getReadingTime()

Retrieve the current reading time in seconds.

Return type

float

getResultantAnd(*and_cons*)

Gets the resultant variable in And constraint.

Parameters

and_cons (*Constraint*) – Constraint to get the resultant variable from.

Return type

Variable

getRhs(*cons*)

Retrieve right-hand side value of a constraint.

Parameters

cons (*Constraint*) – constraint to get the right-hand side from

Return type

float

getRowActivity(*row*)

Returns the activity of a row in the last LP or pseudo solution.

Parameters

row (*Row*)

Return type

float

getRowDualSol(*row*)

Gets the dual LP solution of a row.

Parameters**row** ([Row](#))**Return type**

float

getRowLPActivity(*row*)

Returns the activity of a row in the last LP solution.

Parameters**row** ([Row](#))**Return type**

float

getRowLinear(*cons*)

Retrieve the linear relaxation of the given linear constraint as a row. May return NULL if no LP row was yet created; the user must not modify the row!

Parameters**cons** ([Constraint](#)) – linear constraint to get the coefficients of**Return type**[Row](#)**getRowNumIntCols**(*row*)

Returns number of integral columns in the row.

Parameters**row** ([Row](#))**Return type**

int

getRowObjParallelism(*row*)

Returns 1 if the row is parallel, and 0 if orthogonal.

Parameters**row** ([Row](#))**Return type**

float

getRowParallelism(*row1*, *row2*, *orthofunc*=101)

Returns the degree of parallelism between hyperplanes. 1 if perfectly parallel, 0 if orthogonal. For two row vectors *v*, *w* the parallelism is calculated as: $\text{abs}(v \cdot w) / (\text{abs}(v) \cdot \text{abs}(w))$. 101 in this case is an 'e' (euclidean) in ASCII. The other acceptable input is 100 (d for discrete).

Parameters

- **row1** ([Row](#))
- **row2** ([Row](#))
- **orthofunc** (*int*, *optional*) – 101 (default) is an 'e' (euclidean) in ASCII. Alternate value is 100 (d for discrete)

Return type

float

getSiblings()

Gets the siblings of the focus node.

Return type

list of Nodes

getSlack(cons, sol=None, side=None)

Retrieve slack of given constraint. Can only be called after solving is completed.

Parameters

- **cons** (*Constraint*) – linear constraint
- **sol** (*Solution or None, optional*) – solution to compute slack of, None to use current node's solution (Default value = None)
- **side** (*str or None, optional*) – whether to use 'lhs' or 'rhs' for ranged constraints, None to return minimum (Default value = None)

Return type

float

getSlackVarIndicator(cons)

Get slack variable of an indicator constraint.

Parameters**cons** (*Constraint*) – The indicator constraint**Return type***Variable***getSolObjVal(sol, original=True)**

Retrieve the objective value of the solution.

Parameters

- **sol** (*Solution*)
- **original** (*bool, optional*) – objective value in original space (Default value = True)

Return type

float

getSolTime(sol)

Get clock time when this solution was found.

Parameters**sol** (*Solution*)**Return type**

float

getSolVal(sol, expr)

Retrieve value of given variable or expression in the given solution or in the LP/pseudo solution if sol == None

Parameters

- **sol** (*Solution*)
- **expr** (*Expr*) – polynomial expression to query the value of

Return type
float

Notes

A variable is also an expression.

getSols()

Retrieve list of all feasible primal solutions stored in the solution storage.

Return type
list of Solution

getSolvingTime()

Retrieve the current solving time in seconds.

Return type
float

getStage()

Retrieve current SCIP stage.

Return type
int

getStageName()

Returns name of current stage as string.

Return type
str

getStatus()

Retrieve solution status.

Returns
The status of SCIP.

Return type
str

getTechVersion()

Retrieve SCIP technical version.

Return type
int

getTermsQuadratic(*cons*)

Retrieve bilinear, quadratic, and linear terms of a quadratic constraint.

Parameters
cons ([Constraint](#))

Returns

- **bilinterms** (*list of tuple*)
- **quadterms** (*list of tuple*)
- **linterms** (*list of tuple*)

getTotalTime()

Retrieve the current total SCIP time in seconds, i.e. the total time since the SCIP instance has been created.

Return type

float

getTransformedCons(*cons*)

Retrieve transformed constraint.

Parameters

cons (*Constraint*)

Return type

Constraint

getTransformedVar(*var*)

Retrieve the transformed variable.

Parameters

var (*Variable*) – original variable to get the transformed of

Return type

Variable

getTreesizeEstimation()

Get an estimate of the final tree size.

Return type

float

getVal(*expr*: *Expr* | *MatrixExpr*)

Retrieve the value of the given variable or expression in the best known solution. Can only be called after solving is completed.

Parameters

expr (*Expr* or *MatrixExpr*) – polynomial expression to query the value of

Return type

float

Notes

A variable is also an expression.

getValsLinear(*cons*)

Retrieve the coefficients of a linear constraint

Parameters

cons (*Constraint*) – linear constraint to get the coefficients of

Return type

dict of str to float

getVarDict(*transformed=False*)

Gets dictionary with variables names as keys and current variable values as items.

Parameters

transformed (*bool*, *optional*) – Get transformed variables instead of original (Default value = False)

Return type

dict of str to float

getVarLbDive(*var*)

Returns variable's current lb in current dive.

Parameters

var (*Variable*)

Return type

float

getVarPseudocost(*var*, *branchdir*)

gets the variable's pseudo cost value for the given direction

Parameters

- **variable** (*Variable*) – problem variable
- **branchdir** (*PY SCIP BRANCHDIR*) – branching direction (downwards, or upwards)

Return type

float

getVarPseudocostScore(*var*, *solVal*)

gets the variable's pseudo cost score value for the given LP solution value

Parameters

- **variable** (*Variable*) – problem variable
- **solVal** (*float*) – difference of variable's new LP value - old LP value

Return type

float

getVarRedcost(*var*)

Retrieve the reduced cost of a variable.

Parameters

var (*Variable*) – variable to get the reduced cost of

Return type

float

getVarStrongbranch(*var*, *itlim*, *idempotent=False*, *integral=False*)

Strong branches and gets information on column variable.

Parameters

- **var** (*Variable*) – Variable to get strong branching information on
- **itlim** (*int*) – LP iteration limit for total strong branching calls
- **idempotent** (*bool*, *optional*) – Should SCIP's state remain the same after the call?
- **integral** (*bool*, *optional*) – Boolean on whether the variable is currently integer.

Returns

- **down** (*float*) – The dual bound of the LP after branching down on the variable
- **up** (*float*) – The dual bound of the LP after branchign up on the variable
- **downvalid** (*bool*) – stores whether the returned down value is a valid dual bound, or NULL
- **upvalid** (*bool*) – stores whether the returned up value is a valid dual bound, or NULL
- **downinf** (*bool*) – store whether the downwards branch is infeasible

- **upinf** (*bool*) – store whether the upwards branch is infeasible
- **downconflict** (*bool*) – store whether a conflict constraint was created for an infeasible downwards branch
- **upconflict** (*bool*) – store whether a conflict constraint was created for an infeasible upwards branch
- **lpperor** (*bool*) – whether an unresolved LP error occurred in the solving process

getVarStrongbranchLast(*var*)

Get the results of the last strong branching call on this variable (potentially was called at another node).

Parameters

var (*Variable*) – variable to get the previous strong branching information from

Returns

- **down** (*float*) – The dual bound of the LP after branching down on the variable
- **up** (*float*) – The dual bound of the LP after branchign up on the variable
- **downvalid** (*bool*) – stores whether the returned down value is a valid dual bound, or NULL
- **upvalid** (*bool*) – stores whether the returned up value is a valid dual bound, or NULL
- **solval** (*float*) – The solution value of the variable at the last strong branching call
- **lpobjval** (*float*) – The LP objective value at the time of the last strong branching call

getVarStrongbranchNode(*var*)

Get the node number from the last time strong branching was called on the variable.

Parameters

var (*Variable*) – variable to get the previous strong branching node from

Return type

int

getVarUbDive(*var*)

Returns variable's current ub in current dive.

Parameters

var (*Variable*)

Return type

float

getVars(*transformed=False*)

Retrieve all variables.

Parameters

transformed (*bool*, *optional*) – Get transformed variables instead of original (Default value = False)

Return type

list of *Variable*

getVarsAnd(*and_cons*)

Gets variables in AND constraint.

Parameters

and_cons (*Constraint*) – AND Constraint to get the variables from.

Return typelist of *Variable***getWeightsKnapsack(*cons*)**

Retrieve the coefficients of a knapsack constraint

Parameters**cons** (*Constraint*) – knapsack constraint to get the coefficients of**Return type**

dict of str to float

hasPrimalRay()

Returns whether a primal ray is stored that proves unboundedness of the LP relaxation.

Return type

bool

hideOutput(*quiet=True*)

Hide the output.

Parameters**quiet** (*bool*, *optional*) – hide output? (Default value = True)**inProbing()**

Returns whether we are in probing mode; probing mode is activated via startProbing() and stopped via endProbing().

Return type

bool

inRepropagation()

Returns if the current node is already solved and only propagated again.

Return type

bool

includeBenders(*benders*, *name*, *desc*, *priority=1*, *cutlp=True*, *cutpseudo=True*, *cutrelax=True*, *shareaux=False*)

Include a Benders' decomposition.

Parameters

- **benders** (*Benders*) – the Benders decomposition
- **name** (*str*) – the name
- **desc** (*str*) – the description
- **priority** (*int*, *optional*) – priority of the Benders' decomposition
- **cutlp** (*bool*, *optional*) – should Benders' cuts be generated from LP solutions
- **cutpseudo** (*bool*, *optional*) – should Benders' cuts be generated from pseudo solutions
- **cutrelax** (*bool*, *optional*) – should Benders' cuts be generated from relaxation solutions
- **shareaux** (*bool*, *optional*) – should the Benders' decomposition share the auxiliary variables of the highest priority Benders' decomposition

includeBendersDefaultCuts(*benders*)

Includes the default Benders' decomposition cuts to the custom Benders' decomposition plugin.

Parameters

benders (*Benders*) – the Benders' decomposition that the default cuts will be applied to

includeBenderscut(*benders*, *benderscut*, *name*, *desc*, *priority=1*, *islpcut=True*)

Include a Benders' decomposition cutting method

Parameters

- **benders** (*Benders*) – the Benders' decomposition that this cutting method is attached to
- **benderscut** (*Benderscut*) – the Benders' decomposition cutting method
- **name** (*str*) – the name
- **desc** (*str*) – the description
- **priority** (*int*, *optional*) – priority of the Benders' decomposition (Default = 1)
- **islpcut** (*bool*, *optional*) – is this cutting method suitable for generating cuts for convex relaxations? (Default = True)

includeBranchrule(*branchrule*, *name*, *desc*, *priority*, *maxdepth*, *maxbounddist*)

Include a branching rule.

Parameters

- **branchrule** (*Branchrule*) – branching rule
- **name** (*str*) – name of branching rule
- **desc** (*str*) – description of branching rule
- **priority** (*int*) – priority of branching rule
- **maxdepth** (*int*) – maximal depth level up to which this branching rule should be used (or -1)
- **maxbounddist** (*float*) – maximal relative distance from current node's dual bound to primal bound compared to best node's dual bound for applying branching rule (0.0: only on current best node, 1.0: on all nodes)

includeConshdlr(*conshdlr*, *name*, *desc*, *sepapriority=0*, *enfopriority=0*, *chkcpriority=0*, *sepaftereq=-1*, *propfreq=-1*, *eagerfreq=100*, *maxprerounds=-1*, *delaysepa=False*, *delayprop=False*, *needscons=True*, *proptiming=1*, *presoltiming=8*)

Include a constraint handler.

Parameters

- **conshdlr** (*Conshdlr*) – constraint handler
- **name** (*str*) – name of constraint handler
- **desc** (*str*) – description of constraint handler
- **sepapriority** (*int*, *optional*) – priority for separation (Default value = 0)
- **enfopriority** (*int*, *optional*) – priority for constraint enforcing (Default value = 0)
- **chkcpriority** (*int*, *optional*) – priority for checking feasibility (Default value = 0)
- **sepaftereq** (*int*, *optional*) – frequency for separating cuts; 0 = only at root node (Default value = -1)

- **propfreq** (*int*, *optional*) – frequency for propagating domains; 0 = only preprocessing propagation (Default value = -1)
- **eagerfreq** (*int*, *optional*) –
frequency for using all instead of only the useful constraints in separation,
propagation and enforcement; -1 = no eager evaluations, 0 = first only (Default value = 100)
- **maxprerounds** (*int*, *optional*) – maximal number of presolving rounds the constraint handler participates in (Default value = -1)
- **delaysepa** (*bool*, *optional*) – should separation method be delayed, if other separators found cuts? (Default value = False)
- **delayprop** (*bool*, *optional*) – should propagation method be delayed, if other propagators found reductions? (Default value = False)
- **needscons** (*bool*, *optional*) – should the constraint handler be skipped, if no constraints are available? (Default value = True)
- **proptiming** (*PY SCIP PROPTIMING*) –
positions in the node solving loop where propagation method of constraint handlers
should be executed (Default value = SCIP_PROPTIMING.BEFORELP)
- **presoltiming** (*PY SCIP PRESOLTIMING*) – timing mask of the constraint handler's presolving method (Default value = SCIP_PRESOLTIMING.MEDIUM)

includeCutsel (*cutsel*, *name*, *desc*, *priority*)

Include a cut selector.

Parameters

- **cutsel** (*Cutsel*) – cut selector
- **name** (*str*) – name of cut selector
- **desc** (*str*) – description of cut selector
- **priority** (*int*) – priority of the cut selector

includeDefaultPlugins()

Includes all default plug-ins into SCIP.

includeEventhdlr (*eventhdlr*, *name*, *desc*)

Include an event handler.

Parameters

- **eventhdlr** (*Eventhdlr*) – event handler
- **name** (*str*) – name of event handler
- **desc** (*str*) – description of event handler

includeHeur (*heur*, *name*, *desc*, *dispchar*, *priority=10000*, *freq=1*, *freqofs=0*, *maxdepth=-1*, *timingmask=1*, *usessubscip=False*)

Include a primal heuristic.

Parameters

- **heur** (*Heur*) – heuristic
- **name** (*str*) – name of heuristic

- **desc** (*str*) – description of heuristic
- **dispchar** (*str*) – display character of heuristic. Please use a single length string.
- **priority** (*int*, *optional*) – priority of the heuristic (Default value = 10000)
- **freq** (*int*, *optional*) – frequency for calling heuristic (Default value = 1)
- **freqofs** (*int*, *optional*) – frequency offset for calling heuristic (Default value = 0)
- **maxdepth** (*int*, *optional*) – maximal depth level to call heuristic at (Default value = -1)
- **timingmask** (*PY SCIP HEURTIMING*, *optional*) – positions in the node solvingreturn { 'result': SCIP_RESULT.SUCCESS, 'lowerbound': 10e4
- **executed** (*{ loop where heuristic should be }*) – (Default value = SCIP_HEURTIMING_BEFORENODE)
- **usessubscip** (*bool*, *optional*) – does the heuristic use a secondary SCIP instance? (Default value = False)

includeIISfinder(*iisfinder*, *name*, *desc*, *priority=10000*, *freq=1*)

Include an IIS (Irreducible Infeasible Set) finder handler.

Parameters

- **iisfinder** (*IISfinder*) – IIS finder
- **name** (*str*) – name of IIS finder
- **desc** (*str*) – description of IIS finder
- **priority** (*int*, *optional*) – priority of the IISfinder (#todo description)
- **freq** (*int*, *optional*) – frequency for calling IIS finder

includeNodesel(*nodesel*, *name*, *desc*, *stdpriority*, *memsavepriority*)

Include a node selector.

Parameters

- **nodesel** (*Nodesel*) – node selector
- **name** (*str*) – name of node selector
- **desc** (*str*) – description of node selector
- **stdpriority** (*int*) – priority of the node selector in standard mode
- **memsavepriority** (*int*) – priority of the node selector in memory saving mode

includePresol(*presol*, *name*, *desc*, *priority*, *maxrounds*, *timing=4*)

Include a presolver.

Parameters

- **presol** (*Presol*) – presolver
- **name** (*str*) – name of presolver
- **desc** (*str*) – description of presolver
- **priority** (*int*) – priority of the presolver (≥ 0 : before, < 0 : after constraint handlers)
- **maxrounds** (*int*) – maximal number of presolving rounds the presolver participates in (-1: no limit)

- **timing** (*PY SCIP PRESOLTIMING*, *optional*) – timing mask of presolver (Default value = *SCIP PRESOLTIMING_FAST*)

includePricer(*pricer*, *name*, *desc*, *priority=1*, *delay=True*)

Include a pricer.

Parameters

- **pricer** (*Pricer*) – pricer
- **name** (*str*) – name of pricer
- **desc** (*str*) – description of pricer
- **priority** (*int*, *optional*) – priority of pricer (Default value = 1)
- **delay** (*bool*, *optional*) – should the pricer be delayed until no other pricers or already existing problem variables with negative reduced costs are found? (Default value = True)

includeProp(*prop*, *name*, *desc*, *presolpriority*, *presolmaxrounds*, *proptiming*, *presoltiming=4*, *priority=1*, *freq=1*, *delay=True*)

Include a propagator.

Parameters

- **prop** (*Prop*) – propagator
- **name** (*str*) – name of propagator
- **desc** (*str*) – description of propagator
- **presolpriority** (*int*) – presolving priority of the propagator (≥ 0 : before, < 0 : after constraint handlers)
- **presolmaxrounds** (*int*) – maximal number of presolving rounds the propagator participates in (-1: no limit)
- **proptiming** (*SCIP_PROPTIMING*) – positions in the node solving loop where propagation method of constraint handlers should be executed
- **presoltiming** (*PY SCIP PRESOLTIMING*, *optional*) – timing mask of the constraint handler's presolving method (Default value = *SCIP PRESOLTIMING_FAST*)
- **priority** (*int*, *optional*) – priority of the propagator (Default value = 1)
- **freq** (*int*, *optional*) – frequency for calling propagator (Default value = 1)
- **delay** (*bool*, *optional*) – should propagator be delayed if other propagators have found reductions? (Default value = True)

includeReader(*reader*, *name*, *desc*, *ext*)

Include a reader.

Parameters

- **reader** (*Reader*) – reader
- **name** (*str*) – name of reader
- **desc** (*str*) – description of reader
- **ext** (*str*) – file extension of reader

includeRelax(*relax, name, desc, priority=10000, freq=1*)

Include a relaxation handler.

Parameters

- **relax** (*Relax*) – relaxation handler
- **name** (*str*) – name of relaxation handler
- **desc** (*str*) – description of relaxation handler
- **priority** (*int, optional*) – priority of the relaxation handler (negative: after LP, non-negative: before LP, Default value = 10000)
- **freq** (*int, optional*) – frequency for calling relaxation handler

includeSepa(*sepa, name, desc, priority=0, freq=10, maxbounddist=1.0, usessubscip=False, delay=False*)

Include a separator

Parameters

- **sepa** (*Seпа*) – separator
- **name** (*str*) – name of separator
- **desc** (*str*) – description of separator
- **priority** (*int, optional*) – priority of separator (≥ 0 : before, < 0 : after constraint handlers)
- **freq** (*int, optional*) – frequency for calling separator
- **maxbounddist** (*float, optional*) – maximal relative distance from current node's dual bound to primal bound compared to best node's dual bound for applying separation
- **usessubscip** (*bool, optional*) – does the separator use a secondary SCIP instance? (Default value = False)
- **delay** (*bool, optional*) – should separator be delayed, if other separators found cuts? (Default value = False)
- **sepa** – separator
- **name** – name of separator
- **desc** – description of separator
- **priority** – priority of separator (≥ 0 : before, < 0 : after constraint handlers) (default=0)
- **freq** – frequency for calling separator (default=10)
- **maxbounddist** – maximal relative distance from current node's dual bound to primal bound compared to best node's dual bound for applying separation. (default = 1.0)
- **usessubscip** – does the separator use a secondary SCIP instance? (Default value = False)
- **delay** – should separator be delayed if other separators found cuts? (Default value = False)

infinity()

Retrieve SCIP's infinity value.

Return type

int

initBendersDefault(*subproblems*)

Initialises the default Benders' decomposition with a dictionary of subproblems.

Parameters

subproblems (*Model* or *dict of object to Model*) – a single Model instance or dictionary of Model instances

interruptSolve()

Interrupt the solving process as soon as possible.

isAndConsSorted(*and_cons*)

Returns if the variables of the AND-constraint are sorted with respect to their indices.

Parameters

and_cons (*Constraint*) – Constraint to check.

Return type

bool

isCutEfficacious(*cut*, *sol=None*)

Returns whether the cut's efficacy with respect to the given primal solution or the current LP solution is greater than the minimal cut efficacy.

Parameters

- **cut** (*Row*)
- **sol** (*Solution or None, optional*)

Return type

float

isEQ(*val1*, *val2*)

Checks, if values are in range of epsilon.

Parameters

- **val1** (*float*)
- **val2** (*float*)

Return type

bool

isExact()

Returns whether exact solving mode is enabled in SCIP.

Return type

bool

isFeasEQ(*val1*, *val2*)

Returns if relative difference between val1 and val2 is in range of feasibility tolerance.

Parameters

- **val1** (*float*)
- **val2** (*float*)

Return type

bool

isFeasGE(*val1*, *val2*)

Returns whether relative difference of *val1* and *val2* is not lower than minus feasibility tolerance.

Parameters

- **val1** (*float*)
- **val2** (*float*)

Return type

bool

isFeasGT(*val1*, *val2*)

Returns whether relative difference between *val1* and *val2* is greater than feasibility tolerance.

Parameters

- **val1** (*float*)
- **val2** (*float*)

Return type

bool

isFeasIntegral(*value*)

Returns whether *value* is integral within the LP feasibility bounds.

Parameters

value (*float*)

Return type

bool

isFeasLE(*val1*, *val2*)

Returns whether relative difference between *val1* and *val2* is not greater than feasibility tolerance.

Parameters

- **val1** (*float*)
- **val2** (*float*)

Return type

bool

isFeasLT(*val1*, *val2*)

Returns whether relative difference between *val1* and *val2* is lower than minus feasibility tolerance.

Parameters

- **val1** (*float*)
- **val2** (*float*)

Return type

bool

isFeasNegative(*value*)

Returns whether *value* < -feastol.

Parameters

value (*float*)

Return type

bool

isFeasPositive(*value*)

Returns whether $\text{value} > \text{feastol}$.

Parameters

value (*float*)

Return type

bool

isFeasZero(*value*)

Returns whether $\text{abs}(\text{value}) < \text{feastol}$.

Parameters

value (*float*)

Return type

bool

isGE(*val1*, *val2*)

Returns whether $\text{val1} \geq \text{val2} - \text{eps}$.

Parameters

- **val1** (*float*)

- **val2** (*float*)

Return type

bool

isGT(*val1*, *val2*)

Returns whether $\text{val1} > \text{val2} + \text{eps}$.

Parameters

- **val1** (*float*)

- **val2** (*float*)

Return type

bool

isHugeValue(*val*)

Checks if value is huge and should be handled separately (e.g., in activity computation).

Parameters

val (*float*)

Return type

bool

isInfinity(*value*)

Returns whether value is SCIP's infinity.

Parameters

value (*float*)

Return type

bool

isLE(*val1*, *val2*)

Returns whether $\text{val1} \leq \text{val2} + \text{eps}$.

Parameters

- **val1** (*float*)

- **val2** (*float*)

Return type

bool

isLPSolBasic()

Returns whether the current LP solution is basic, i.e. is defined by a valid simplex basis.

Return type

bool

isLT(val1, val2)

Returns whether $\text{val1} < \text{val2} - \text{eps}$.

Parameters

- **val1** (*float*)

- **val2** (*float*)

Return type

bool

isNLPConstructed()

Returns whether SCIP's internal NLP has been constructed.

Return type

bool

isNegative(val)

Returns whether $\text{val} < -\text{eps}$.

Parameters

val (*float*)

Return type

bool

isObjChangedProbing()

Returns whether the objective function has changed during probing mode.

Return type

bool

isPositive(val)

Returns whether $\text{val} > \text{eps}$.

Parameters

val (*float*)

Return type

bool

isZero(value)

Returns whether $\text{abs}(\text{value}) < \text{eps}$.

Parameters

value (*float*)

Return type

bool

lpiGetIterations()

Get the iteration count of the last solved LP.

Return type

int

markDoNotAggrVar(var)

Marks a variable preventing it from being aggregated.

Parameters

var (*Variable*) – variable to mark

markDoNotMultaggrVar(var)

Marks a variable preventing it from being multi-aggregated.

Parameters

var (*Variable*) – variable to mark

newProbingNode()

Creates a new probing sub node, whose changes can be undone by backtracking to a higher node in the probing path with a call to backtrackProbing().

optimize()

Optimize the problem.

optimizeNogil()

Optimize the problem without GIL.

presolve()

Presolve the problem.

printBestSol(write_zeros=False)

Prints the best feasible primal solution.

Parameters

write_zeros (*bool*, *optional*) – include variables that are set to zero (Default = False)

printCons(constraint)

Print the constraint

Parameters

constraint (*Constraint*)

printExternalCodeVersions()

Print external code versions, e.g. symmetry, non-linear solver, lp solver.

printNlRow(nlrow)

Prints nonlinear row.

Parameters

nlrow (*NLRow*)

printProblem(ext='.cip', trans=False, genericnames=False)

Write current model/problem to standard output.

Parameters

- **ext** (*str*, *optional*) – the extension to be used (Default value = '.cip'). Should have an extension corresponding to one of the readable file formats, described in https://www.scipopt.org/doc/html/group__FILEREADERS.php.

- **trans** (*bool*, *optional*) – indicates whether the transformed problem is written to file (Default value = False)
- **genericnames** (*bool*, *optional*) – indicates whether the problem should be written with generic variable and constraint names (Default value = False)

printRow(*row*)

Prints row.

Parameters

row (*Row*)

printSol(*solution=None*, *write_zeros=False*)

Print the given primal solution.

Parameters

- **solution** (*Solution* or *None*, *optional*) – solution to print (default = None)
- **write_zeros** (*bool*, *optional*) – include variables that are set to zero (Default=False)

printStatistics(*filename=None*)

Print statistics.

Parameters

filename (*str*, *optional*) – name of the output file (Default = None)

printStatisticsJson()

Print statistics in JSON format.

printVersion()

Print version, copyright information and compile mode.

propagateProbing(*maxpropounds*)

Applies domain propagation on the probing sub problem, that was changed after `SCIPstartProbing()` was called; the propagated domains of the variables can be accessed with the usual bound accessing calls `SCIPvarGetLbLocal()` and `SCIPvarGetUbLocal()`; the propagation is only valid locally, i.e. the local bounds as well as the changed bounds due to `SCIPchgVarLbProbing()`, `SCIPchgVarUbProbing()`, and `SCIPfixVarProbing()` are used for propagation.

Parameters

maxpropounds (*int*) – maximal number of propagation rounds (Default value = -1, that is, no limit)

Returns

- **cutoff** (*bool*) – whether the probing node can be cutoff
- **ndomredsfound** (*int*) – number of domain reductions found

readParams(*file*)

Read an external parameter file.

Parameters

file (*str*) – file to read

readProblem(*filename*, *extension=None*)

Read a problem instance from an external file.

Parameters

- **filename** (*str*) – problem file name

- **extension** (*str* or *None*) – specify file extension/type (Default value = *None*)

readSol(*filename*)

Reads a given solution file, problem has to be transformed in advance.

Parameters

filename (*str*) – name of the input file

readSolFile(*filename*)

Reads a given solution file.

Solution is created but not added to storage/the model. Use ‘addSol’ OR ‘trySol’ to add it.

Parameters

filename (*str*) – name of the input file

Return type

Solution

redirectOutput()

Send output to python instead of terminal.

relax()

Relaxes the integrality restrictions of the model.

releaseRow(*row*)

Decreases usage counter of LP row, and frees memory if necessary.

Parameters

row (*Row*)

repropagateNode(*node*)

Marks the given node to be propagated again the next time a node of its subtree is processed.

resetParam(*name*)

Reset parameter setting to its default value

Parameters

name (*str*) – parameter to reset

resetParams()

Reset parameter settings to their default values.

restartSolve()

Restarts the solving process as soon as possible.

separateSol(*sol=None, pretendroot=False, allowlocal=True, onlydelayed=False*)

Separates the given primal solution or the current LP solution by calling the separators and constraint handlers’ separation methods; the generated cuts are stored in the separation storage and can be accessed with the methods `SCIPgetCuts()` and `SCIPgetNCuts()`; after evaluating the cuts, you have to call `SCIPclearCuts()` in order to remove the cuts from the separation storage; it is possible to call `SCIPseparateSol()` multiple times with different solutions and evaluate the found cuts afterwards.

Parameters

- **sol** (*Solution* or *None*, *optional*) – solution to separate, *None* to use current lp solution (Default value = *None*)
- **pretendroot** (*bool*, *optional*) – should the cut separators be called as if we are at the root node? (Default value = “False”)

- **allowlocal** (*bool*, *optional*) – should the separator be asked to separate local cuts (Default value = True)
- **onlydelayed** (*bool*, *optional*) – should only separators be called that were delayed in the previous round? (Default value = False)

Returns

- **delayed** (*bool*) – whether a separator was delayed
- **cutoff** (*bool*) – whether the node can be cut off

setBendersSubproblemIsConvex(*benders*, *probnumber*, *isconvex=True*)

Sets a flag indicating whether the subproblem is convex.

Parameters

- **benders** (*Benders*) – the Benders’ decomposition which contains the subproblem
- **probnumber** (*int*) – the problem number of the subproblem that the convexity will be set for
- **isconvex** (*bool*, *optional*) – flag to indicate whether the subproblem is convex (default=True)

setBoolParam(*name*, *value*)

Set a boolean-valued parameter.

Parameters

- **name** (*str*) – name of parameter
- **value** (*bool*) – value of parameter

setCharParam(*name*, *value*)

Set a char-valued parameter.

Parameters

- **name** (*str*) – name of parameter
- **value** (*str*) – value of parameter

setCheck(*cons*, *newCheck*)

Set “check” flag of a constraint.

Parameters

- **cons** (*Constraint*)
- **newCheck** (*bool*)

setEmphasis(*paraemphasis*, *quiet=True*)

Set emphasis settings

Parameters

- **paraemphasis** (*PY SCIP PARAMEMPHASIS*) – emphasis to set
- **quiet** (*bool*, *optional*) – hide output? (Default value = True)

setEnforced(*cons*, *newEnf*)

Set “enforced” flag of a constraint.

Parameters

- **cons** (*Constraint*)

- **newEnf** (*bool*)

setHeurTiming(*heurname, heurtiming*)

Set the timing of a heuristic

Parameters

- **heurname** (*string, name of the heuristic*)
- **heurtiming** (*PY SCIP_HEURTIMING*) – positions in the node solving loop where heuristic should be executed

setHeuristics(*setting*)

Set heuristics parameter settings.

Parameters

setting (*SCIP_PARAMSETTING*) – the parameter settings, e.g. SCIP_PARAMSETTING.OFF

setInitial(*cons, newInit*)

Set “initial” flag of a constraint.

Parameters

- **cons** (*Constraint*)
- **newInit** (*bool*)

setIntParam(*name, value*)

Set an int-valued parameter.

Parameters

- **name** (*str*) – name of parameter
- **value** (*int*) – value of parameter

setLogfile(*path*)

Sets the log file name for the currently installed message handler.

Parameters

path (*str or None*) – name of log file, or None (no log)

setLongintParam(*name, value*)

Set a long-valued parameter.

Parameters

- **name** (*str*) – name of parameter
- **value** (*int*) – value of parameter

setMaximize()

Set the objective sense to maximization.

setMinimize()

Set the objective sense to minimization.

setModifiable(*cons, newMod*)

Set “modifiable” flag of a constraint.

Parameters

- **cons** (*Constraint*)

- **newMod** (*bool*)

setObjIntegral()

Informs SCIP that the objective value is always integral in every feasible solution.

Notes

This function should be used to inform SCIP that the objective function is integral, helping to improve the performance. This is useful when using column generation. If no column generation (pricing) is used, SCIP automatically detects whether the objective function is integral or can be scaled to be integral. However, in any case, the user has to make sure that no variable is added during the solving process that destroys this property.

setObjective(*expr*, *sense*='minimize', *clear*='true')

Establish the objective function as a linear expression.

Parameters

- **expr** (*Expr* or *float*) – the objective function SCIP Expr, or constant value
- **sense** (*str*, *optional*) – the objective sense (“minimize” or “maximize”) (Default value = ‘minimize’)
- **clear** (*bool*, *optional*) – set all other variables objective coefficient to zero (Default value = ‘true’)

setObjlimit(*objlimit*)

Set a limit on the objective function. Only solutions with objective value better than this limit are accepted.

Parameters

- **objlimit** (*float*) – limit on the objective function

setParam(*name*, *value*)

Set a parameter with value in int, bool, real, long, char or str.

Parameters

- **name** (*str*) – name of parameter
- **value** (*object*) – value of parameter

setParams(*params*)

Sets multiple parameters at once.

Parameters

- **params** (*dict of str to object*) – dict mapping parameter names to their values.

setParamsCountsols()

Sets SCIP parameters such that a valid counting process is possible.

setPresolve(*setting*)

Set presolving parameter settings.

Parameters

- **setting** (*SCIP_PARAMSETTING*) – the parameter settings, e.g. SCIP_PARAMSETTING.OFF

setProbName(*name*)

Set problem name.

Parameters

- **name** (*str*)

setRealParam(*name*, *value*)

Set a real-valued parameter.

Parameters

- **name** (*str*) – name of parameter
- **value** (*float*) – value of parameter

setRelaxSolVal(*var*, *val*)

Sets the value of the given variable in the global relaxation solution.

Parameters

- **var** (*Variable*)
- **val** (*float*)

setRemovable(*cons*, *newRem*)

Set “removable” flag of a constraint.

Parameters

- **cons** (*Constraint*)
- **newRem** (*bool*)

setSeparating(*setting*)

Set separating parameter settings.

Parameters

setting (*SCIP_PARAMSETTING*) – the parameter settings, e.g. SCIP_PARAMSETTING.OFF

setSolVal(*solution*, *var*, *val*)

Set a variable in a solution.

Parameters

- **solution** (*Solution*) – solution to be modified
- **var** (*Variable*) – variable in the solution
- **val** (*float*) – value of the specified variable

setStringParam(*name*, *value*)

Set a string-valued parameter.

Parameters

- **name** (*str*) – name of parameter
- **value** (*str*) – value of parameter

setupBendersSubproblem(*probnumber*, *benders=None*, *solution=None*, *checktype=1*)

Sets up the Benders’ subproblem given the master problem solution.

Parameters

- **probnumber** (*int*) – the index of the problem that is to be set up
- **benders** (*Benders or None, optional*) – the Benders’ decomposition to which the subproblem belongs to
- **solution** (*Solution or None, optional*) – the master problem solution that is used for the set up, if None, then the LP solution is used

- **checktype** (*PY SCIP BENDERS FOTYPE*) – the type of solution check that prompted the solving of the Benders’ subproblems, either PY SCIP BENDERS FOTYPE: LP, RELAX, PSEUDO or CHECK. Default is LP.

solveBendersSubproblem(*probnumber, solvecip, benders=None, solution=None*)

Solves the Benders’ decomposition subproblem. The convex relaxation will be solved unless the parameter *solvecip* is set to True.

Parameters

- **probnumber** (*int*) – the index of the problem that is to be set up
- **solvecip** (*bool*) – whether the CIP of the subproblem should be solved. If False, then only the convex relaxation is solved.
- **benders** (*Benders or None, optional*) – the Benders’ decomposition to which the subproblem belongs to
- **solution** (*Solution or None, optional*) – the master problem solution that is used for the set up, if None, then the LP solution is used

Returns

- **infeasible** (*bool*) – returns whether the current subproblem is infeasible
- **objective** (*float or None*) – the objective function value of the subproblem, can be None

solveConcurrent()

Transforms, presolves, and solves problem using additional solvers which emphasize on finding solutions. WARNING: This feature is still experimental and prone to some errors.

solveDiveLP(*itlim=-1*)

Solves the LP of the current dive. No separation or pricing is applied.

Parameters

- **itlim** (*int, optional*) – maximal number of LP iterations to perform (Default value = -1, that is, no limit)

Returns

- **lperror** (*bool*) – whether an unresolved lp error occurred
- **cutoff** (*bool*) – whether the LP was infeasible or the objective limit was reached

solveProbingLP(*itlim=-1*)

Solves the LP at the current probing node (cannot be applied at preprocessing stage) no separation or pricing is applied.

Parameters

- **itlim** (*int*) – maximal number of LP iterations to perform (Default value = -1, that is, no limit)

Returns

- **lperror** (*bool*) – if an unresolved lp error occurred
- **cutoff** (*bool*) – whether the LP was infeasible or the objective limit was reached

sortAndCons(*and_cons*)

Sorts the variables of the AND-constraint with respect to their indices.

Parameters

- **and_cons** (*Constraint*) – Constraint to sort.

startDive()

Initiates LP diving. It allows the user to change the LP in several ways, solve, change again, etc, without affecting the actual LP. When endDive() is called, SCIP will undo all changes done and recover the LP it had before startDive.

startProbing()

Initiates probing, making methods SCIPnewProbingNode(), SCIPbacktrackProbing(), SCIPchgVarLbProbing(), SCIPchgVarUbProbing(), SCIPfixVarProbing(), SCIPpropagateProbing(), SCIPsolveProbingLP(), etc available.

startStrongbranch()

Start strong branching. Needs to be called before any strong branching. Must also later end strong branching. TODO: Propagation option has currently been disabled via Python. If propagation is enabled then strong branching is not done on the LP, but on additionally created nodes (has some overhead).

tightenVarLb(*var*, *lb*, *force*=False)

Tighten the lower bound in preprocessing or current node, if the bound is tighter.

Parameters

- **var** (*Variable*) – SCIP variable
- **lb** (*float*) – possible new lower bound
- **force** (*bool*, *optional*) – force tightening even if below bound strengthening tolerance (default = False)

Returns

- **infeasible** (*bool*) – Whether new domain is empty
- **tightened** (*bool*) – Whether the bound was tightened

tightenVarLbGlobal(*var*, *lb*, *force*=False)

Tighten the global lower bound, if the bound is tighter.

Parameters

- **var** (*Variable*) – SCIP variable
- **lb** (*float*) – possible new lower bound
- **force** (*bool*, *optional*) – force tightening even if below bound strengthening tolerance

Returns

- **infeasible** (*bool*) – Whether new domain is empty
- **tightened** (*bool*) – Whether the bound was tightened

tightenVarUb(*var*, *ub*, *force*=False)

Tighten the upper bound in preprocessing or current node, if the bound is tighter.

Parameters

- **var** (*Variable*) – SCIP variable
- **ub** (*float*) – possible new upper bound
- **force** (*bool*, *optional*) – force tightening even if below bound strengthening tolerance

Returns

- **infeasible** (*bool*) – Whether new domain is empty
- **tightened** (*bool*) – Whether the bound was tightened

tightenVarUbGlobal(*var, ub, force=False*)

Tighten the global upper bound, if the bound is tighter.

Parameters

- **var** (*Variable*) – SCIP variable
- **ub** (*float*) – possible new upper bound
- **force** (*bool, optional*) – force tightening even if below bound strengthening tolerance

Returns

- **infeasible** (*bool*) – Whether new domain is empty
- **tightened** (*bool*) – Whether the bound was tightened

to_ptr(*give_ownership*)

Return the underlying Scip pointer to the current Model.

Parameters

give_ownership (*bool*) – Whether the current Model gives away ownership of the underlying Scip pointer (see `_freescip`)

Returns

The underlying pointer to the current Model, wrapped in a PyCapsule under the name “scip”.

Return type

capsule

translateSubSol(*sub_model, sol, heur*) → Solution

Translates a solution of a model copy into a solution of the main model

Parameters

- **sub_model** (*Model*) – The python-wrapper of the subscip
- **sol** (*Solution*) – The python-wrapper of the solution of the subscip
- **heur** (*Heur*) – The python-wrapper of the heuristic that found the solution

Returns

solution – The corresponding solution in the main model

Return type

Solution

trySol(*solution, printreason=True, completely=False, checkbounds=True, checkintegrality=True, checklprows=True, free=True*)

Check given primal solution for feasibility and try to add it to the storage.

Parameters

- **solution** (*Solution*) – solution to store
- **printreason** (*bool, optional*) – should all reasons of violations be printed? (Default value = True)
- **completely** (*bool, optional*) – should all violation be checked? (Default value = False)
- **checkbounds** (*bool, optional*) – should the bounds of the variables be checked? (Default value = True)

- **checkintegrality** (*bool, optional*) – does integrality have to be checked? (Default value = True)
- **checklprows** (*bool, optional*) – do current LP rows (both local and global) have to be checked? (Default value = True)
- **free** (*bool, optional*) – should solution be freed? (Default value = True)

Returns

stored – whether given solution was feasible and good enough to keep

Return type

bool

updateBendersLowerbounds (*lowerbounds, benders=None*)

Updates the subproblem lower bounds for benders using the lowerbounds dict. If benders is None, then the default Benders' decomposition is updated.

Parameters

- **lowerbounds** (*dict of int to float*)
- **benders** (*Benders or None, optional*)

updateNodeLowerbound (*node, lb*)

If given value is larger than the node's lower bound (in transformed problem), sets the node's lower bound to the new value.

Parameters

- **node** (*Node*) – the node to update
- **lb** (*float*) – new bound (if greater) for the node

updateVarPseudocost (*var, valdelta, objdelta, weight*)

Updates the pseudo costs of the given variable and the global pseudo costs after a change of valdelta in the variable's solution value and resulting change of objdelta in the LP's objective value. Update is ignored if objdelts is infinite. Weight is in range (0, 1], and affects how it updates the global weighted sum.

Parameters

- **var** (*Variable*) – Variable whos pseudo cost will be updated
- **valdelta** (*float*) – The change in variable value (e.g. the fractional amount removed or added by branching)
- **objdelta** (*float*) – The change in objective value of the LP after valdelta change of the variable
- **weight** (*float*) – the weight in range (0,1] of how the update affects the stored weighted sum.

version()

Retrieve SCIP version.

Return type

float

writeBestSol (*filename='origprob.sol', write_zeros=False*)

Write the best feasible primal solution to a file.

Parameters

- **filename** (*str, optional*) – name of the output file (Default="origprob.sol")

- **write_zeros** (*bool*, *optional*) – include variables that are set to zero (Default=False)

writeBestTransSol(*filename*='transprob.sol', *write_zeros*=False)

Write the best feasible primal solution for the transformed problem to a file.

Parameters

- **filename** (*str*, *optional*) – name of the output file (Default="transprob.sol")
- **write_zeros** (*bool*, *optional*) – include variables that are set to zero (Default=False)

writeLP(*filename*='LP.lp')

Writes current LP to a file.

Parameters

filename (*str*, *optional*) – file name (Default value = "LP.lp")

writeMIP(*filename*, *genericnames*=False, *origobj*=False, *lazyconss*=False)

Writes MIP relaxation of the current branch-and-bound node to a file

Parameters

- **filename** (*str*) – name of the output file
- **genericnames** (*bool*, *optional*) – should generic names like *x_i* and *row_j* be used in order to avoid troubles with reserved symbols? (Default value = False)
- **origobj** (*bool*, *optional*) – should the original objective function be used (Default value = False)
- **lazyconss** (*bool*, *optional*) – output removable rows as lazy constraints? (Default value = False)

writeName(*var*)

Write the name of the variable to the std out.

Parameters

var (*Variable*)

writeParams(*filename*='param.set', *comments*=True, *onlychanged*=True, *verbose*=True)

Write parameter settings to an external file.

Parameters

- **filename** (*str*, *optional*) – file to be written (Default value = 'param.set')
- **comments** (*bool*, *optional*) – write parameter descriptions as comments? (Default value = True)
- **onlychanged** (*bool*, *optional*) – write only modified parameters (Default value = True)
- **verbose** (*bool*, *optional*) – indicates whether a success message should be printed

writeProblem(*filename*='model.cip', *trans*=False, *genericnames*=False, *verbose*=True)

Write current model/problem to a file.

Parameters

- **filename** (*str*, *optional*) – the name of the file to be used (Default value = 'model.cip'). Should have an extension corresponding to one of the readable file formats, described in https://www.scipopt.org/doc/html/group__FILEREADERS.php.
- **trans** (*bool*, *optional*) – indicates whether the transformed problem is written to file (Default value = False)

- **genericnames** (*bool*, *optional*) – indicates whether the problem should be written with generic variable and constraint names (Default value = False)
- **verbose** (*bool*, *optional*) – indicates whether a success message should be printed

writeSol(*solution*, *filename*='origprob.sol', *write_zeros*=False)

Write the given primal solution to a file.

Parameters

- **solution** (*Solution*) – solution to write
- **filename** (*str*, *optional*) – name of the output file (Default="origprob.sol")
- **write_zeros** (*bool*, *optional*) – include variables that are set to zero (Default=False)

writeStatistics(*filename*='origprob.stats')

Write statistics to a file.

Parameters

filename (*str*, *optional*) – name of the output file (Default = "origprob.stats")

writeStatisticsJson(*filename*='origprob.stats.json')

Write statistics to a JSON file.

Parameters

filename (*str*, *optional*) – name of the output file (Default = "origprob.stats.json")

writeTransSol(*solution*, *filename*='transprob.sol', *write_zeros*=False)

Write the given transformed primal solution to a file.

Parameters

- **solution** (*Solution*) – transformed solution to write
- **filename** (*str*, *optional*) – name of the output file (Default="transprob.sol")
- **write_zeros** (*bool*, *optional*) – include variables that are set to zero (Default=False)

1.9.2 SCIP Constraint

This class wraps a SCIP constraint object. It contains functions that can retrieve basic information that is entirely contained within the constraint object.

Constraint API

class pycipopt.Constraint

Base class holding a pointer to corresponding SCIP_CONS

getConshdlrName()

Return the constraint handler's name.

Return type

str

isActive()

Returns True iff constraint is active in the current node.

Return type

bool

isChecked()

Returns True if constraint should be checked for feasibility.

Return type

bool

isDynamic()

Returns True if constraint is subject to aging.

Return type

bool

isEnforced()

Returns True if constraint should be enforced during node processing.

Return type

bool

isInitial()

Returns True if the relaxation of the constraint should be in the initial LP.

Return type

bool

isKnapsack()

Returns True if constraint is a knapsack constraint. This is a special case of a linear constraint.

Return type

bool

isLinear()

Returns True if constraint is linear

Return type

bool

isLinearType()

Returns True if constraint can be represented as a linear constraint.

Return type

bool

isLocal()

Returns True if constraint is only locally valid or not added to any (sub)problem.

Return type

bool

isModifiable()

Returns True if constraint is modifiable (subject to column generation).

Return type

bool

isNonlinear()

Returns True if constraint is nonlinear.

Return type

bool

isOriginal()

Retrieve whether the constraint belongs to the original problem.

Return type
bool

isPropagated()

Returns True if constraint should be propagated during node processing.

Return type
bool

isRemovable()

Returns True if constraint's relaxation should be removed from the LP due to aging or cleanup.

Return type
bool

isSeparated()

Returns True if constraint should be separated during LP processing.

Return type
bool

isStickingAtNode()

Returns True if constraint is only locally valid or not added to any (sub)problem.

Return type
bool

1.9.3 SCIP Variable

This class wraps a SCIP variable object. It contains functions that can retrieve basic information that is entirely contained within the variable object.

Variable API

class pycscipopt.Variable

Is a linear expression and has SCIP_VAR*

getAvgSol()

Get the weighted average solution of variable in all feasible primal solutions found.

Return type
float

getCol()

Retrieve column of COLUMN variable.

Return type
Column

getImplType()

Returns the implied integral type of the variable

Return type
PY SCIP_IMPLINTTYPE

getIndex()

Retrieve the unique index of the variable.

Return type

int

getLPSol()

Retrieve the current LP solution value of variable.

Return type

float

getLbGlobal()

Retrieve global lower bound of variable.

Return type

float

getLbLocal()

Retrieve current lower bound of variable.

Return type

float

getLbOriginal()

Retrieve original lower bound of variable.

Return type

float

getNBranchings(*branchdir*)

returns the number of times a bound of the variable was changed in given direction due to branching

Parameters

branchdir (*PY SCIP BRANCHDIR*) – branching direction (downwards, or upwards)

Return type

int

getNBranchingsCurrentRun(*branchdir*)

returns the number of times a bound of the variable was changed in given direction due to branching in the current run

Parameters

branchdir (*PY SCIP BRANCHDIR*) – branching direction (downwards, or upwards)

Return type

int

getNLocksDown()

Returns the number of locks for rounding down.

Return type

int

getNLocksDownType(*locktype*)

Returns the number of locks for rounding down of a certain type.

Parameters

locktype (*SCIP LOCKTYPE*) – type of variable locks

Return type

int

getNLocksUp()

Returns the number of locks for rounding up.

Return type

int

getNLocksUpType(locktype)

Returns the number of locks for rounding up of a certain type.

Parameters**locktype** (*SCIP_LOCKTYPE*) – type of variable locks**Return type**

int

getObj()

Retrieve current objective value of variable.

Return type

float

getStatus()

Retrieve the variable status (ORIGINAL, LOOSE, COLUMN, FIXED, AGGREGATED, MULTAGGR, NEGATED)

Returns

“ORIGINAL”, “LOOSE”, “COLUMN”, “FIXED”, “AGGREGATED”, “MULTAGGR”, “NEGATED”

Return type

str

getUbGlobal()

Retrieve global upper bound of variable.

Return type

float

getUbLocal()

Retrieve current upper bound of variable.

Return type

float

getUbOriginal()

Retrieve original upper bound of variable.

Return type

float

isActive()

Retrieve whether the variable is active

Return type

bool

isBinary()

Returns whether variable is of BINARY type.

Return type

bool

isDeletable()

Returns whether variable is allowed to be deleted completely from the problem.

Return type

bool

isImpliedIntegral()

Returns whether variable is implied integral (weakly or strongly).

Return type

bool

isInLP()

Retrieve whether the variable is a COLUMN variable that is member of the current LP.

Return type

bool

isIntegral()

Returns whether variable is of INTEGER type.

Return type

bool

isNonImpliedIntegral()

Returns TRUE if the variable is integral, but not implied integral..

Return type

bool

isOriginal()

Retrieve whether the variable belongs to the original problem

Return type

bool

isRelaxationOnly()

returns whether a variable has been introduced to define a relaxation

These variables are only valid for the current SCIP solve round, they are not contained in any (checked) constraints, but may be used in cutting planes, for example. Relaxation-only variables are not copied by SCIPcopyVars and cuts that contain these variables are not added as linear constraints when restarting or transferring information from a copied SCIP to a SCIP. Also conflicts with relaxation-only variables are not generated at the moment. Relaxation-only variables do not appear in the objective.

Return type

bool

markRelaxationOnly()

marks that this variable has only been introduced to define a relaxation

The variable must not have a coefficient in the objective and must be deletable. If it is not marked deletable, it will be marked as deletable, which is only possible before the variable is added to a problem.

Return type

None

varMayRound(*direction*='down')

Checks whether it is possible to round variable up / down and stay feasible for the relaxation.

Parameters

direction (*str*) – “up” or “down”

Return type

bool

vtype()

Retrieve the variables type (BINARY, INTEGER, CONTINUOUS, or IMPLINT)

Returns

“BINARY”, “INTEGER”, “CONTINUOUS”, or “IMPLINT”

Return type

str

1.9.4 SCIP Row

This class wraps a SCIP row object. It contains functions that can retrieve basic information that is entirely contained within the row object.

Row API

class pycipopt.scip.Row

Base class holding a pointer to corresponding SCIP_ROW.

getAge()

Gets the age of the row. (The consecutive times the row has been non-active in the LP).

Return type

int

getBasisStatus()

Gets the basis status of a row in the LP solution.

Returns

Possible values are “lower”, “basic”, and “upper”

Return type

str

Raises

Exception – If SCIP returns an unknown or “zero” basis status

Notes

Returns basis status “basic” for rows not in the current SCIP LP.

getCols()

Gets list with columns of nonzero entries

Return type

list of *Column*

getConsOriginConshdlrtype()

Returns type of constraint handler that created the row.

Return type

str

getConstant()

Gets constant shift of row.

Return type
float

getDualfarkas()

Returns the dual Farkas solution of row.

Return type
float

getDualsol()

Returns the dual solution of row.

Return type
float

getLPPos()

Gets position of row in current LP, or -1 if it is not in LP.

Return type
int

getLhs()

Returns the left hand side of row.

Return type
float

getNLPNonz()

Get number of nonzero entries in row vector that correspond to columns currently in the SCIP LP.

Return type
int

getNNonz()

Get number of nonzero entries in row vector.

Return type
int

getNorm()

Gets Euclidean norm of row vector.

Return type
float

getOrigintype()

Returns type of origin that created the row.

Return type
PY SCIP_ROWORIGINTYPE

getRhs()

Returns the right hand side of row.

Return type
float

getVals()

Gets list with coefficients of nonzero entries.

Return type
list of int

isInGlobalCutpool()

Return TRUE iff row is a member of the global cut pool.

Return type
bool

isIntegral()

Returns TRUE iff the activity of the row (without the row's constant) is always integral in a feasible solution.

Return type
bool

isLocal()

Returns TRUE iff the row is only valid locally.

Return type
bool

isModifiable()

Returns TRUE iff row is modifiable during node processing (subject to column generation).

Return type
bool

isRemovable()

Returns TRUE iff row is removable from the LP (due to aging or cleanup).

Return type
bool

1.9.5 SCIP Column

This class wraps a SCIP column object. It contains functions that can retrieve basic information that is entirely contained within the column object.

Column API

class pyscipopt.scip.Column

Base class holding a pointer to corresponding SCIP_COL.

getAge()

Gets the age of the column, i.e., the total number of successive times a column was in the LP and was 0.0 in the solution.

Return type
int

getBasisStatus()

Gets the basis status of a column in the LP solution

Returns
Possible values are "lower", "basic", "upper", and "zero"

Return type

str

Raises**Exception** – If SCIP returns an unknown basis status**Notes**

Returns basis status “zero” for columns not in the current SCIP LP.

getLPPos()

Gets position of column in current LP, or -1 if it is not in LP.

Return type

int

getLb()

Gets lower bound of column.

Return type

float

getObjCoeff()

Gets objective value coefficient of a column.

Return type

float

getPrimsol()

Gets the primal LP solution of a column.

Return type

float

getUb()

Gets upper bound of column.

Return type

float

getVar()

Gets variable this column represents.

Return type*Variable***isIntegral()**

Returns whether the associated variable is of integral type (binary, integer, implicit integer).

Return type

bool

1.9.6 SCIP Node

This class wraps a SCIP node object. It contains functions that can retrieve basic information that is entirely contained within the node object.

Node API

class pysciptopt.scip.**Node**

Base class holding a pointer to corresponding SCIP_NODE

getAddedConss()

Retrieve all constraints added at this node.

Return type

list of *Constraint*

getDepth()

Retrieve depth of node.

Return type

int

getDomchg()

Retrieve domain changes for this node.

Return type

DomainChanges

getEstimate()

Retrieve the estimated value of the best feasible solution in subtree of the node.

Return type

float

getLowerbound()

Retrieve lower bound of node.

Return type

float

getNAddedConss()

Retrieve number of added constraints at this node.

Return type

int

getNDomchg()

Retrieve the number of bound changes due to branching, constraint propagation, and propagation.

Returns

- **nbranchings** (*int*)
- **nconsprop** (*int*)
- **nprop** (*int*)

getNParentBranchings()

Retrieve the number of variable branchings that were performed in the parent node to create this node.

Return type

int

getNumber()

Retrieve number of node.

Return type

int

getParent()

Retrieve parent node (or None if the node has no parent node).

Return type

Node

getParentBranchings()

Retrieve the set of variable branchings that were performed in the parent node to create this node.

Returns

- *list of Variable*
- *list of float*
- *list of int*

getType()

Retrieve type of node.

Return type

PY SCIP NODETYPE

isActive()

Is the node in the path to the current node?

Return type

bool

isPropagatedAgain()

Is the node marked to be propagated again?

Return type

bool

1.9.7 SCIP Event

This class wraps a SCIP event object. It contains functions that can retrieve basic information that is entirely contained within the event object.

Event API

class pysciptopt.scip.Event

Base class holding a pointer to corresponding SCIP_EVENT.

getName()

Gets name of event.

Return type

str

getNewBound()

Gets new bound for a bound change event.

Return type

float

getNode()

Gets node for a node or LP event.

Return type

Node

getOldBound()

Gets old bound for a bound change event.

Return type

float

getRow()

Gets row for a row event.

Return type

Row

getType()

Gets type of event.

Return type

PY SCIP_EVENTTYPE

getVar()

Gets variable for a variable event (var added, var deleted, var fixed, objective value or domain change, domain hole added or removed).

Return type

Variable

PYTHON MODULE INDEX

p

pyscipt, [67](#)

A

activateBenders() (*pyscipopt.Model method*), 67
 addBendersSubproblem() (*pyscipopt.Model method*), 67
 addCoefKnapsack() (*pyscipopt.Model method*), 68
 addCoefLinear() (*pyscipopt.Model method*), 68
 addCons() (*pyscipopt.Model method*), 68
 addConsAnd() (*pyscipopt.Model method*), 69
 addConsCardinality() (*pyscipopt.Model method*), 69
 addConsCoeff() (*pyscipopt.Model method*), 70
 addConsDisjunction() (*pyscipopt.Model method*), 70
 addConsElemDisjunction() (*pyscipopt.Model method*), 71
 addConsIndicator() (*pyscipopt.Model method*), 71
 addConsKnapsack() (*pyscipopt.Model method*), 72
 addConsLocal() (*pyscipopt.Model method*), 72
 addConsNode() (*pyscipopt.Model method*), 73
 addConsOr() (*pyscipopt.Model method*), 73
 addConss() (*pyscipopt.Model method*), 75
 addConsSOS1() (*pyscipopt.Model method*), 73
 addConsSOS2() (*pyscipopt.Model method*), 74
 addConsXor() (*pyscipopt.Model method*), 75
 addCut() (*pyscipopt.Model method*), 76
 addExprNonlinear() (*pyscipopt.Model method*), 77
 addMatrixCons() (*pyscipopt.Model method*), 77
 addMatrixConsIndicator() (*pyscipopt.Model method*), 78
 addMatrixVar() (*pyscipopt.Model method*), 79
 addObjoffset() (*pyscipopt.Model method*), 79
 addPoolCut() (*pyscipopt.Model method*), 79
 addPyCons() (*pyscipopt.Model method*), 79
 addRowDive() (*pyscipopt.Model method*), 79
 addRowExact() (*pyscipopt.Model method*), 80
 addSol() (*pyscipopt.Model method*), 80
 addVar() (*pyscipopt.Model method*), 80
 addVarLocks() (*pyscipopt.Model method*), 80
 addVarLocksType() (*pyscipopt.Model method*), 81
 addVarSOS1() (*pyscipopt.Model method*), 81
 addVarSOS2() (*pyscipopt.Model method*), 81
 addVarToRow() (*pyscipopt.Model method*), 81
 allColsInLP() (*pyscipopt.Model method*), 81
 allowNegSlackExact() (*pyscipopt.Model method*), 81

appendVarSOS1() (*pyscipopt.Model method*), 81
 appendVarSOS2() (*pyscipopt.Model method*), 82
 applyCutsProbing() (*pyscipopt.Model method*), 82
 attachEventHandlerCallback() (*pyscipopt.Model method*), 82

B

backtrackProbing() (*pyscipopt.Model method*), 82
 branchLPExact() (*pyscipopt.Model method*), 82
 branchVar() (*pyscipopt.Model method*), 82
 branchVarVal() (*pyscipopt.Model method*), 83

C

cacheRowExtensions() (*pyscipopt.Model method*), 83
 calcChildEstimate() (*pyscipopt.Model method*), 83
 calcNodeselPriority() (*pyscipopt.Model method*), 83
 catchEvent() (*pyscipopt.Model method*), 83
 catchRowEvent() (*pyscipopt.Model method*), 84
 catchVarEvent() (*pyscipopt.Model method*), 84
 checkBendersSubproblemOptimality() (*pyscipopt.Model method*), 84
 checkQuadraticNonlinear() (*pyscipopt.Model method*), 84
 checkSol() (*pyscipopt.Model method*), 84
 chgCapacityKnapsack() (*pyscipopt.Model method*), 85
 chgCoefLinear() (*pyscipopt.Model method*), 85
 chgLhs() (*pyscipopt.Model method*), 85
 chgReoptObjective() (*pyscipopt.Model method*), 85
 chgRhs() (*pyscipopt.Model method*), 85
 chgRowLhsDive() (*pyscipopt.Model method*), 86
 chgRowRhsDive() (*pyscipopt.Model method*), 86
 chgVarBranchPriority() (*pyscipopt.Model method*), 86
 chgVarLb() (*pyscipopt.Model method*), 86
 chgVarLbDive() (*pyscipopt.Model method*), 86
 chgVarLbGlobal() (*pyscipopt.Model method*), 86
 chgVarLbNode() (*pyscipopt.Model method*), 86
 chgVarLbProbing() (*pyscipopt.Model method*), 87
 chgVarObjDive() (*pyscipopt.Model method*), 87
 chgVarObjProbing() (*pyscipopt.Model method*), 87

chgVarType() (*pyscipopt.Model* method), 87
 chgVarUb() (*pyscipopt.Model* method), 87
 chgVarUbDive() (*pyscipopt.Model* method), 87
 chgVarUbGlobal() (*pyscipopt.Model* method), 87
 chgVarUbNode() (*pyscipopt.Model* method), 88
 chgVarUbProbing() (*pyscipopt.Model* method), 88
 Column (*class in pyscipopt.scip*), 146
 computeBestSolSubproblems() (*pyscipopt.Model* method), 88
 Constraint (*class in pyscipopt*), 138
 constructLP() (*pyscipopt.Model* method), 88
 copyLargeNeighborhoodSearch() (*pyscipopt.Model* method), 88
 count() (*pyscipopt.Model* method), 88
 createChild() (*pyscipopt.Model* method), 88
 createCons() (*pyscipopt.Model* method), 89
 createConsFromExpr() (*pyscipopt.Model* method), 89
 createEmptyRowSepa() (*pyscipopt.Model* method), 90
 createEmptyRowUnspec() (*pyscipopt.Model* method), 90
 createOrigSol() (*pyscipopt.Model* method), 91
 createPartialSol() (*pyscipopt.Model* method), 91
 createProbBasic() (*pyscipopt.Model* method), 91
 createSol() (*pyscipopt.Model* method), 91
 cutoffNode() (*pyscipopt.Model* method), 91

D

deactivatePricer() (*pyscipopt.Model* method), 91
 delCoefLinear() (*pyscipopt.Model* method), 91
 delCons() (*pyscipopt.Model* method), 92
 delConsLocal() (*pyscipopt.Model* method), 92
 delVar() (*pyscipopt.Model* method), 92
 disableDebugSol() (*pyscipopt.Model* method), 92
 disablePropagation() (*pyscipopt.Model* method), 92
 dropEvent() (*pyscipopt.Model* method), 92
 dropRowEvent() (*pyscipopt.Model* method), 92
 dropVarEvent() (*pyscipopt.Model* method), 92

E

enableDebugSol() (*pyscipopt.Model* method), 93
 enableExactSolving() (*pyscipopt.Model* method), 93
 enableReoptimization() (*pyscipopt.Model* method), 93
 endDive() (*pyscipopt.Model* method), 93
 endProbing() (*pyscipopt.Model* method), 93
 endStrongbranch() (*pyscipopt.Model* method), 93
 epsilon() (*pyscipopt.Model* method), 93
 Event (*class in pyscipopt.scip*), 149

F

feasCeil() (*pyscipopt.Model* method), 93
 feasFloor() (*pyscipopt.Model* method), 93
 feasFrac() (*pyscipopt.Model* method), 93

feasRound() (*pyscipopt.Model* method), 94
 feastol() (*pyscipopt.Model* method), 94
 fixVar() (*pyscipopt.Model* method), 94
 fixVarProbing() (*pyscipopt.Model* method), 94
 flushRowExtensions() (*pyscipopt.Model* method), 94
 frac() (*pyscipopt.Model* method), 94
 freeBendersSubproblems() (*pyscipopt.Model* method), 94
 freeProb() (*pyscipopt.Model* method), 95
 freeReoptSolve() (*pyscipopt.Model* method), 95
 freeSol() (*pyscipopt.Model* method), 95
 freeTransform() (*pyscipopt.Model* method), 95
 from_ptr() (*pyscipopt.Model* static method), 95

G

generateIIS() (*pyscipopt.Model* method), 95
 getActivity() (*pyscipopt.Model* method), 95
 getAddedConss() (*pyscipopt.scip.Node* method), 148
 getAge() (*pyscipopt.scip.Column* method), 146
 getAge() (*pyscipopt.scip.Row* method), 144
 getAvgSol() (*pyscipopt.Variable* method), 140
 getBasisStatus() (*pyscipopt.scip.Column* method), 146
 getBasisStatus() (*pyscipopt.scip.Row* method), 144
 getBendersAuxiliaryVar() (*pyscipopt.Model* method), 95
 getBendersSubproblem() (*pyscipopt.Model* method), 96
 getBendersVar() (*pyscipopt.Model* method), 96
 getBestboundNode() (*pyscipopt.Model* method), 97
 getBestChild() (*pyscipopt.Model* method), 96
 getBestLeaf() (*pyscipopt.Model* method), 96
 getBestNode() (*pyscipopt.Model* method), 96
 getBestSibling() (*pyscipopt.Model* method), 96
 getBestSol() (*pyscipopt.Model* method), 96
 getBipartiteGraphRepresentation() (*pyscipopt.Model* method), 97
 getBranchScoreMultiple() (*pyscipopt.Model* method), 97
 getCapacityKnapsack() (*pyscipopt.Model* method), 97
 getChildren() (*pyscipopt.Model* method), 98
 getCol() (*pyscipopt.Variable* method), 140
 getColRedCost() (*pyscipopt.Model* method), 98
 getCols() (*pyscipopt.scip.Row* method), 144
 getCondition() (*pyscipopt.Model* method), 98
 getConshdlrName() (*pyscipopt.Constraint* method), 138
 getConsNVars() (*pyscipopt.Model* method), 98
 getConsOriginConshdlrtype() (*pyscipopt.scip.Row* method), 144
 getConss() (*pyscipopt.Model* method), 99
 getConstant() (*pyscipopt.scip.Row* method), 144
 getConsVals() (*pyscipopt.Model* method), 98

getConsVars() (*pyscipopt.Model* method), 98
 getCurrentNode() (*pyscipopt.Model* method), 99
 getCutEfficacy() (*pyscipopt.Model* method), 99
 getCutLPsolCutoffDistance() (*pyscipopt.Model* method), 99
 getCutoffbound() (*pyscipopt.Model* method), 99
 getDepth() (*pyscipopt.Model* method), 99
 getDepth() (*pyscipopt.scip.Node* method), 148
 getDomchg() (*pyscipopt.scip.Node* method), 148
 getDualbound() (*pyscipopt.Model* method), 100
 getDualboundRoot() (*pyscipopt.Model* method), 100
 getDualfarkas() (*pyscipopt.scip.Row* method), 145
 getDualfarkasKnapsack() (*pyscipopt.Model* method), 100
 getDualfarkasLinear() (*pyscipopt.Model* method), 100
 getDualMultiplier() (*pyscipopt.Model* method), 99
 getDualsol() (*pyscipopt.scip.Row* method), 145
 getDualsolKnapsack() (*pyscipopt.Model* method), 100
 getDualsolLinear() (*pyscipopt.Model* method), 100
 getDualSolVal() (*pyscipopt.Model* method), 100
 getEstimate() (*pyscipopt.scip.Node* method), 148
 getGap() (*pyscipopt.Model* method), 100
 getHeurTiming() (*pyscipopt.Model* method), 101
 getIIS() (*pyscipopt.Model* method), 101
 getImplType() (*pyscipopt.Variable* method), 140
 getIndex() (*pyscipopt.Variable* method), 140
 getLb() (*pyscipopt.scip.Column* method), 147
 getLbGlobal() (*pyscipopt.Variable* method), 141
 getLbLocal() (*pyscipopt.Variable* method), 141
 getLbOriginal() (*pyscipopt.Variable* method), 141
 getLeaves() (*pyscipopt.Model* method), 102
 getLhs() (*pyscipopt.Model* method), 102
 getLhs() (*pyscipopt.scip.Row* method), 145
 getLinearConsIndicator() (*pyscipopt.Model* method), 102
 getLocalEstimate() (*pyscipopt.Model* method), 102
 getLowerbound() (*pyscipopt.Model* method), 102
 getLowerbound() (*pyscipopt.scip.Node* method), 148
 getLPBasisInd() (*pyscipopt.Model* method), 101
 getLPBInvARow() (*pyscipopt.Model* method), 101
 getLPBInvRow() (*pyscipopt.Model* method), 101
 getLPBranchCands() (*pyscipopt.Model* method), 101
 getLPColsData() (*pyscipopt.Model* method), 102
 getLPObjVal() (*pyscipopt.Model* method), 102
 getLPPos() (*pyscipopt.scip.Column* method), 147
 getLPPos() (*pyscipopt.scip.Row* method), 145
 getLPRowsData() (*pyscipopt.Model* method), 102
 getLPsol() (*pyscipopt.Variable* method), 141
 getLPsolstat() (*pyscipopt.Model* method), 102
 getMajorVersion() (*pyscipopt.Model* method), 103
 getMaxDepth() (*pyscipopt.Model* method), 103
 getMinorVersion() (*pyscipopt.Model* method), 103
 getNAddedConss() (*pyscipopt.scip.Node* method), 148
 getName() (*pyscipopt.scip.Event* method), 149
 getNBestSolsFound() (*pyscipopt.Model* method), 103
 getNBinVars() (*pyscipopt.Model* method), 103
 getNBranchings() (*pyscipopt.Variable* method), 141
 getNBranchingsCurrentRun() (*pyscipopt.Variable* method), 141
 getNChildren() (*pyscipopt.Model* method), 103
 getNConss() (*pyscipopt.Model* method), 103
 getNContVars() (*pyscipopt.Model* method), 103
 getNCountedSols() (*pyscipopt.Model* method), 103
 getNCuts() (*pyscipopt.Model* method), 104
 getNCutsApplied() (*pyscipopt.Model* method), 104
 getNDomchg() (*pyscipopt.scip.Node* method), 148
 getNewBound() (*pyscipopt.scip.Event* method), 149
 getNFeasibleLeaves() (*pyscipopt.Model* method), 104
 getNImplVars() (*pyscipopt.Model* method), 104
 getNInfeasibleLeaves() (*pyscipopt.Model* method), 104
 getNIntVars() (*pyscipopt.Model* method), 104
 getNLeaves() (*pyscipopt.Model* method), 105
 getNLimSolsFound() (*pyscipopt.Model* method), 105
 getNLocksDown() (*pyscipopt.Variable* method), 141
 getNLocksDownType() (*pyscipopt.Variable* method), 141
 getNLocksUp() (*pyscipopt.Variable* method), 142
 getNLocksUpType() (*pyscipopt.Variable* method), 142
 getNLPBranchCands() (*pyscipopt.Model* method), 104
 getNLPCols() (*pyscipopt.Model* method), 104
 getNLPIterations() (*pyscipopt.Model* method), 104
 getNLPNonz() (*pyscipopt.scip.Row* method), 145
 getNLPRows() (*pyscipopt.Model* method), 104
 getNLPs() (*pyscipopt.Model* method), 105
 getNlRowActivityBounds() (*pyscipopt.Model* method), 106
 getNlRows() (*pyscipopt.Model* method), 107
 getNlRowSolActivity() (*pyscipopt.Model* method), 106
 getNlRowSolFeasibility() (*pyscipopt.Model* method), 107
 getNNlRows() (*pyscipopt.Model* method), 105
 getNNodelPIterations() (*pyscipopt.Model* method), 105
 getNNodes() (*pyscipopt.Model* method), 105
 getNNonz() (*pyscipopt.scip.Row* method), 145
 getNode() (*pyscipopt.scip.Event* method), 149
 getNorm() (*pyscipopt.scip.Row* method), 145
 getNParentBranchings() (*pyscipopt.scip.Node* method), 148
 getNReaders() (*pyscipopt.Model* method), 105
 getNSepaRounds() (*pyscipopt.Model* method), 105
 getNSiblings() (*pyscipopt.Model* method), 105
 getNSols() (*pyscipopt.Model* method), 105

- getNSolsFound() (*pyscipopt.Model* method), 106
 getNStrongbranchLPIterations() (*pyscipopt.Model* method), 106
 getNTotalNodes() (*pyscipopt.Model* method), 106
 getNumber() (*pyscipopt.scip.Node* method), 148
 getNVars() (*pyscipopt.Model* method), 106
 getNVarsAnd() (*pyscipopt.Model* method), 106
 getObj() (*pyscipopt.Variable* method), 142
 getObjCoeff() (*pyscipopt.scip.Column* method), 147
 getObjective() (*pyscipopt.Model* method), 107
 getObjectiveSense() (*pyscipopt.Model* method), 107
 getObjlimit() (*pyscipopt.Model* method), 107
 getObjoffset() (*pyscipopt.Model* method), 107
 getObjVal() (*pyscipopt.Model* method), 107
 getOldBound() (*pyscipopt.scip.Event* method), 150
 getOpenNodes() (*pyscipopt.Model* method), 107
 getOrigintype() (*pyscipopt.scip.Row* method), 145
 getParam() (*pyscipopt.Model* method), 108
 getParams() (*pyscipopt.Model* method), 108
 getParent() (*pyscipopt.scip.Node* method), 148
 getParentBranchings() (*pyscipopt.scip.Node* method), 149
 getPlungeDepth() (*pyscipopt.Model* method), 108
 getPresolvingTime() (*pyscipopt.Model* method), 108
 getPrimalbound() (*pyscipopt.Model* method), 108
 getPrimalRay() (*pyscipopt.Model* method), 108
 getPrimalRayVal() (*pyscipopt.Model* method), 108
 getPrimsol() (*pyscipopt.scip.Column* method), 147
 getPrioChild() (*pyscipopt.Model* method), 108
 getPrioSibling() (*pyscipopt.Model* method), 109
 getProbingDepth() (*pyscipopt.Model* method), 109
 getProbName() (*pyscipopt.Model* method), 109
 getPseudoBranchCands() (*pyscipopt.Model* method), 109
 getReadingTime() (*pyscipopt.Model* method), 109
 getResultantAnd() (*pyscipopt.Model* method), 109
 getRhs() (*pyscipopt.Model* method), 109
 getRhs() (*pyscipopt.scip.Row* method), 145
 getRow() (*pyscipopt.scip.Event* method), 150
 getRowActivity() (*pyscipopt.Model* method), 109
 getRowDualSol() (*pyscipopt.Model* method), 110
 getRowLinear() (*pyscipopt.Model* method), 110
 getRowLPActivity() (*pyscipopt.Model* method), 110
 getRowNumIntCols() (*pyscipopt.Model* method), 110
 getRowObjParallelism() (*pyscipopt.Model* method), 110
 getRowParallelism() (*pyscipopt.Model* method), 110
 getSiblings() (*pyscipopt.Model* method), 111
 getSlack() (*pyscipopt.Model* method), 111
 getSlackVarIndicator() (*pyscipopt.Model* method), 111
 getSolObjVal() (*pyscipopt.Model* method), 111
 getSols() (*pyscipopt.Model* method), 112
 getSolTime() (*pyscipopt.Model* method), 111
 getSolVal() (*pyscipopt.Model* method), 111
 getSolvingTime() (*pyscipopt.Model* method), 112
 getStage() (*pyscipopt.Model* method), 112
 getStageName() (*pyscipopt.Model* method), 112
 getStatus() (*pyscipopt.Model* method), 112
 getStatus() (*pyscipopt.Variable* method), 142
 getTechVersion() (*pyscipopt.Model* method), 112
 getTermsQuadratic() (*pyscipopt.Model* method), 112
 getTotalTime() (*pyscipopt.Model* method), 112
 getTransformedCons() (*pyscipopt.Model* method), 113
 getTransformedVar() (*pyscipopt.Model* method), 113
 getTreesizeEstimation() (*pyscipopt.Model* method), 113
 getType() (*pyscipopt.scip.Event* method), 150
 getType() (*pyscipopt.scip.Node* method), 149
 getUb() (*pyscipopt.scip.Column* method), 147
 getUbGlobal() (*pyscipopt.Variable* method), 142
 getUbLocal() (*pyscipopt.Variable* method), 142
 getUbOriginal() (*pyscipopt.Variable* method), 142
 getVal() (*pyscipopt.Model* method), 113
 getVals() (*pyscipopt.scip.Row* method), 145
 getValsLinear() (*pyscipopt.Model* method), 113
 getVar() (*pyscipopt.scip.Column* method), 147
 getVar() (*pyscipopt.scip.Event* method), 150
 getVarDict() (*pyscipopt.Model* method), 113
 getVarLbDive() (*pyscipopt.Model* method), 113
 getVarPseudocost() (*pyscipopt.Model* method), 114
 getVarPseudocostScore() (*pyscipopt.Model* method), 114
 getVarRedcost() (*pyscipopt.Model* method), 114
 getVars() (*pyscipopt.Model* method), 115
 getVarsAnd() (*pyscipopt.Model* method), 115
 getVarStrongbranch() (*pyscipopt.Model* method), 114
 getVarStrongbranchLast() (*pyscipopt.Model* method), 115
 getVarStrongbranchNode() (*pyscipopt.Model* method), 115
 getVarUbDive() (*pyscipopt.Model* method), 115
 getWeightsKnapsack() (*pyscipopt.Model* method), 116
- ## H
- hasPrimalRay() (*pyscipopt.Model* method), 116
 hideOutput() (*pyscipopt.Model* method), 116
- ## I
- includeBenders() (*pyscipopt.Model* method), 116
 includeBenderscut() (*pyscipopt.Model* method), 117
 includeBendersDefaultCuts() (*pyscipopt.Model* method), 116
 includeBranchrule() (*pyscipopt.Model* method), 117
 includeConshdlr() (*pyscipopt.Model* method), 117

- `includeCutsel()` (*pyscipopt.Model* method), 118
 - `includeDefaultPlugins()` (*pyscipopt.Model* method), 118
 - `includeEventhdlr()` (*pyscipopt.Model* method), 118
 - `includeHeur()` (*pyscipopt.Model* method), 118
 - `includeIISfinder()` (*pyscipopt.Model* method), 119
 - `includeNodesel()` (*pyscipopt.Model* method), 119
 - `includePresol()` (*pyscipopt.Model* method), 119
 - `includePricer()` (*pyscipopt.Model* method), 120
 - `includeProp()` (*pyscipopt.Model* method), 120
 - `includeReader()` (*pyscipopt.Model* method), 120
 - `includeRelax()` (*pyscipopt.Model* method), 120
 - `includeSeps()` (*pyscipopt.Model* method), 121
 - `infinity()` (*pyscipopt.Model* method), 121
 - `initBendersDefault()` (*pyscipopt.Model* method), 121
 - `inProbing()` (*pyscipopt.Model* method), 116
 - `inRepropagation()` (*pyscipopt.Model* method), 116
 - `interruptSolve()` (*pyscipopt.Model* method), 122
 - `isActive()` (*pyscipopt.Constraint* method), 138
 - `isActive()` (*pyscipopt.scip.Node* method), 149
 - `isActive()` (*pyscipopt.Variable* method), 142
 - `isAndConsSorted()` (*pyscipopt.Model* method), 122
 - `isBinary()` (*pyscipopt.Variable* method), 142
 - `isChecked()` (*pyscipopt.Constraint* method), 138
 - `isCutEfficacious()` (*pyscipopt.Model* method), 122
 - `isDeletable()` (*pyscipopt.Variable* method), 143
 - `isDynamic()` (*pyscipopt.Constraint* method), 139
 - `isEnforced()` (*pyscipopt.Constraint* method), 139
 - `isEQ()` (*pyscipopt.Model* method), 122
 - `isExact()` (*pyscipopt.Model* method), 122
 - `isFeasEQ()` (*pyscipopt.Model* method), 122
 - `isFeasGE()` (*pyscipopt.Model* method), 122
 - `isFeasGT()` (*pyscipopt.Model* method), 123
 - `isFeasIntegral()` (*pyscipopt.Model* method), 123
 - `isFeasLE()` (*pyscipopt.Model* method), 123
 - `isFeasLT()` (*pyscipopt.Model* method), 123
 - `isFeasNegative()` (*pyscipopt.Model* method), 123
 - `isFeasPositive()` (*pyscipopt.Model* method), 123
 - `isFeasZero()` (*pyscipopt.Model* method), 124
 - `isGE()` (*pyscipopt.Model* method), 124
 - `isGT()` (*pyscipopt.Model* method), 124
 - `isHugeValue()` (*pyscipopt.Model* method), 124
 - `isImpliedIntegral()` (*pyscipopt.Variable* method), 143
 - `isInfinity()` (*pyscipopt.Model* method), 124
 - `isInGlobalCutpool()` (*pyscipopt.scip.Row* method), 146
 - `isInitial()` (*pyscipopt.Constraint* method), 139
 - `isInLP()` (*pyscipopt.Variable* method), 143
 - `isIntegral()` (*pyscipopt.scip.Column* method), 147
 - `isIntegral()` (*pyscipopt.scip.Row* method), 146
 - `isIntegral()` (*pyscipopt.Variable* method), 143
 - `isKnapsack()` (*pyscipopt.Constraint* method), 139
 - `isLE()` (*pyscipopt.Model* method), 124
 - `isLinear()` (*pyscipopt.Constraint* method), 139
 - `isLinearType()` (*pyscipopt.Constraint* method), 139
 - `isLocal()` (*pyscipopt.Constraint* method), 139
 - `isLocal()` (*pyscipopt.scip.Row* method), 146
 - `isLPSolBasic()` (*pyscipopt.Model* method), 125
 - `isLT()` (*pyscipopt.Model* method), 125
 - `isModifiable()` (*pyscipopt.Constraint* method), 139
 - `isModifiable()` (*pyscipopt.scip.Row* method), 146
 - `isNegative()` (*pyscipopt.Model* method), 125
 - `isNLPConstructed()` (*pyscipopt.Model* method), 125
 - `isNonImpliedIntegral()` (*pyscipopt.Variable* method), 143
 - `isNonlinear()` (*pyscipopt.Constraint* method), 139
 - `isObjChangedProbing()` (*pyscipopt.Model* method), 125
 - `isOriginal()` (*pyscipopt.Constraint* method), 139
 - `isOriginal()` (*pyscipopt.Variable* method), 143
 - `isPositive()` (*pyscipopt.Model* method), 125
 - `isPropagated()` (*pyscipopt.Constraint* method), 140
 - `isPropagatedAgain()` (*pyscipopt.scip.Node* method), 149
 - `isRelaxationOnly()` (*pyscipopt.Variable* method), 143
 - `isRemovable()` (*pyscipopt.Constraint* method), 140
 - `isRemovable()` (*pyscipopt.scip.Row* method), 146
 - `isSeparated()` (*pyscipopt.Constraint* method), 140
 - `isStickingAtNode()` (*pyscipopt.Constraint* method), 140
 - `isZero()` (*pyscipopt.Model* method), 125
- ## L
- `lpiGetIterations()` (*pyscipopt.Model* method), 125
- ## M
- `markDoNotAggrVar()` (*pyscipopt.Model* method), 126
 - `markDoNotMultaggrVar()` (*pyscipopt.Model* method), 126
 - `markRelaxationOnly()` (*pyscipopt.Variable* method), 143
 - `Model` (class in *pyscipopt*), 67
 - `module`
pyscipopt, 67
- ## N
- `newProbingNode()` (*pyscipopt.Model* method), 126
 - `Node` (class in *pyscipopt.scip*), 148
- ## O
- `optimize()` (*pyscipopt.Model* method), 126
 - `optimizeNogil()` (*pyscipopt.Model* method), 126
- ## P
- `presolve()` (*pyscipopt.Model* method), 126

printBestSol() (*pyscipopt.Model* method), 126
 printCons() (*pyscipopt.Model* method), 126
 printExternalCodeVersions() (*pyscipopt.Model*
 method), 126
 printNlRow() (*pyscipopt.Model* method), 126
 printProblem() (*pyscipopt.Model* method), 126
 printRow() (*pyscipopt.Model* method), 127
 printSol() (*pyscipopt.Model* method), 127
 printStatistics() (*pyscipopt.Model* method), 127
 printStatisticsJson() (*pyscipopt.Model* method),
 127
 printVersion() (*pyscipopt.Model* method), 127
 propagateProbing() (*pyscipopt.Model* method), 127
 pyscipopt
 module, 67

R

readParams() (*pyscipopt.Model* method), 127
 readProblem() (*pyscipopt.Model* method), 127
 readSol() (*pyscipopt.Model* method), 128
 readSolFile() (*pyscipopt.Model* method), 128
 redirectOutput() (*pyscipopt.Model* method), 128
 relax() (*pyscipopt.Model* method), 128
 releaseRow() (*pyscipopt.Model* method), 128
 repropagateNode() (*pyscipopt.Model* method), 128
 resetParam() (*pyscipopt.Model* method), 128
 resetParams() (*pyscipopt.Model* method), 128
 restartSolve() (*pyscipopt.Model* method), 128
 Row (class in *pyscipopt.scip*), 144

S

separateSol() (*pyscipopt.Model* method), 128
 setBendersSubproblemIsConvex() (*pyscipopt.Model*
 method), 129
 setBoolParam() (*pyscipopt.Model* method), 129
 setCharParam() (*pyscipopt.Model* method), 129
 setCheck() (*pyscipopt.Model* method), 129
 setEmphasis() (*pyscipopt.Model* method), 129
 setEnforced() (*pyscipopt.Model* method), 129
 setHeuristics() (*pyscipopt.Model* method), 130
 setHeurTiming() (*pyscipopt.Model* method), 130
 setInitial() (*pyscipopt.Model* method), 130
 setIntParam() (*pyscipopt.Model* method), 130
 setLogfile() (*pyscipopt.Model* method), 130
 setLongintParam() (*pyscipopt.Model* method), 130
 setMaximize() (*pyscipopt.Model* method), 130
 setMinimize() (*pyscipopt.Model* method), 130
 setModifiable() (*pyscipopt.Model* method), 130
 setObjective() (*pyscipopt.Model* method), 131
 setObjIntegral() (*pyscipopt.Model* method), 131
 setObjlimit() (*pyscipopt.Model* method), 131
 setParam() (*pyscipopt.Model* method), 131
 setParams() (*pyscipopt.Model* method), 131

setParamsCountsols() (*pyscipopt.Model* method),
 131
 setPresolve() (*pyscipopt.Model* method), 131
 setProbName() (*pyscipopt.Model* method), 131
 setRealParam() (*pyscipopt.Model* method), 131
 setRelaxSolVal() (*pyscipopt.Model* method), 132
 setRemovable() (*pyscipopt.Model* method), 132
 setSeparating() (*pyscipopt.Model* method), 132
 setSolVal() (*pyscipopt.Model* method), 132
 setStringParam() (*pyscipopt.Model* method), 132
 setupBendersSubproblem() (*pyscipopt.Model*
 method), 132
 solveBendersSubproblem() (*pyscipopt.Model*
 method), 133
 solveConcurrent() (*pyscipopt.Model* method), 133
 solveDiveLP() (*pyscipopt.Model* method), 133
 solveProbingLP() (*pyscipopt.Model* method), 133
 sortAndCons() (*pyscipopt.Model* method), 133
 startDive() (*pyscipopt.Model* method), 133
 startProbing() (*pyscipopt.Model* method), 134
 startStrongbranch() (*pyscipopt.Model* method), 134

T

tightenVarLb() (*pyscipopt.Model* method), 134
 tightenVarLbGlobal() (*pyscipopt.Model* method),
 134
 tightenVarUb() (*pyscipopt.Model* method), 134
 tightenVarUbGlobal() (*pyscipopt.Model* method),
 135
 to_ptr() (*pyscipopt.Model* method), 135
 translateSubSol() (*pyscipopt.Model* method), 135
 trySol() (*pyscipopt.Model* method), 135

U

updateBendersLowerbounds() (*pyscipopt.Model*
 method), 136
 updateNodeLowerbound() (*pyscipopt.Model* method),
 136
 updateVarPseudocost() (*pyscipopt.Model* method),
 136

V

Variable (class in *pyscipopt*), 140
 varMayRound() (*pyscipopt.Variable* method), 143
 version() (*pyscipopt.Model* method), 136
 vtype() (*pyscipopt.Variable* method), 144

W

writeBestSol() (*pyscipopt.Model* method), 136
 writeBestTransSol() (*pyscipopt.Model* method), 137
 writeLP() (*pyscipopt.Model* method), 137
 writeMIP() (*pyscipopt.Model* method), 137
 writeName() (*pyscipopt.Model* method), 137

`writeParams()` (*pyscipopt.Model* method), 137
`writeProblem()` (*pyscipopt.Model* method), 137
`writeSol()` (*pyscipopt.Model* method), 138
`writeStatistics()` (*pyscipopt.Model* method), 138
`writeStatisticsJson()` (*pyscipopt.Model* method),
138
`writeTransSol()` (*pyscipopt.Model* method), 138